

國立成功大學

電機工程學系

碩士論文

利用互補碼方式之多層展頻浮水印演算法

Multi-layer Spread Spectrum Watermarking by
Complementary Codes

研究生：游凱名 Student: Kai-Ming Yu

指導教授：邱瀝毅 Advisor: Lih-Yih Chiou

共同指導教授：雷曉方 Co-Advisor: Sheau-Fang Lei

Department of Electrical Engineering

National Cheng Kung University

Tainan, Taiwan, R.O.C.

Thesis for Master of Science

July, 2019

中華民國一零八年七月

國立成功大學

碩士論文

利用互補碼方式之多層展頻浮水印演算法
Multi-layer Spread Spectrum Watermarking by
Complementary Codes

研究生：游凱名

本論文業經審查及口試合格特此證明

論文考試委員：

審曉芳 姚書農

郭致宏 邱滙毅

指導教授：審曉芳 邱滙毅

系(所)主管：楊宏清

中華民國 108 年 7 月 9 日

中文摘要

利用互補碼方式之多層展頻浮水印演算法

游凱名¹ 邱瀝毅² 雷曉方³

國立成功大學電機工程研究所

浮水印演算法在近期的研究中蔚為流行，原因是因為網路與科技的發展，資訊傳播的速度提升，導致惡意散播與盜版的猖獗，而個人智慧財產也因此遭受挑戰，故發展出浮水印的演算法以用於保護個人智慧財產，而本論文研究也針對此方面提出一個更安全並且運算複雜度更低的演算法。

本論文探討之演算法為展頻浮水印演算法，展頻浮水印的發展先是著重於浮水印的解出正確率並且提出改善，而後則是面臨容量上的問題，在 2015 年與 2018 年時，學者 Yong Xiang 等人提出了分別兩種不同之高容量嵌入演算法，但後者的演算法中存在着安全性上的問題，於是同年另一名學者徐詠翔提出了使用互補碼法，做為浮水印嵌入的基礎，並且也有效的提升了安全性及維持原有的解出率。本論文研究提出在解出率不變的情形下，藉由互補碼的排列產生序列，並且分層將浮水印資訊嵌入各層序列當中，達到多層浮水印的效果，提供多重使用者共同擁有同一智慧財產之音檔，本論文演算法不僅提供多重使用者使用，也降低了演算法複雜度、提升偽隨機序列產生速度以及浮水印萃取正確率。

本研究演算法，經由實驗並且使用常見的數位信號處理進行攻擊，最後以相同感知音訊品質評估為基準與其他演算法做比較，最後以位元錯誤率(Bit Error Ratio)進行實驗的分析，實驗之結果顯示出本論文之演算法擁有較好的位元錯誤率。對於演算法偽隨機序列產生複雜度及比例因數選擇，也實驗了不同演算法的產生時間，實驗結果證明本研究提出之演算法降低了偽隨機序列組數以及產生序列的運算複雜度，同時保有較低的比例因數計算時間。

關鍵字：展頻浮水印(Spread Spectrum Watermarking)、互補碼(Complementary Codes)、多層(Multi-layer)

¹作者 ²指導教授 ³共同指導教授

EXTENDED ABSTRACT

Multi-layer Spread Spectrum Watermarking by Complementary Codes

Kai-Ming Yu¹ Lih-Yih Chiou² Sheau-Fang Lei³

Department of Electrical Engineering National Cheng Kung University

SUMMARY

Many spread spectrum (SS) based audio watermarking schemes to protect copyright have been proposed but they all have the host signal interference problem and capacity problem. The major drawback of SS-based audio watermarking methods is the low embedding capacity. In the past, a number of spread spectrum (SS) based audio watermarking methods have been proposed to deal with the host signal interference problem. In recent years, Yong-Xiang and Iynkaran Natgunanathan proposed a novel SS-based audio watermarking method using Gram-Schmidt orthogonalization method to generate orthogonal pseudonoise (PN) sequences. This method can embed larger number of watermark bit but it has security problem. However, since this method has security problem, Yong-Siang Syu then proposed new method which use complementary codes to be the PN sequences that process lower computational cost and high security but it maintains high embedding capacity. In this thesis, we proposed a spread spectrum audio watermarking scheme using the same complementary codes to be the PN sequences but it can embed multi-layer watermark which available to multiple users. By adjusting scaling factor, which not only process lower bit error ratio than Yong-Siang's method but also maintains security and capacity. The performance of the proposed SS-based audio watermarking method is demonstrated by simulation results

Key Words: Audio watermarking, spread spectrum, complementary codes, multi-layer

INTRODUCTION

The development of the internet and technology, the speed of information dissemination has increased, leading to malicious transmission and piracy, and personal intellectual property is also challenged. Therefore, we need to prevent illegal use of unauthorized copyright that information hiding technologies have attracted much attention. To solve above copyright problem, digital watermarking and fingerprinting are quite important solution. Digital watermarking is a technology to hide some copyright related information such as user ID, user password and transaction details into the audio file. When copyright issues occur, we can use this embedded file to extract information which in the audio to figure out this ownership issues.

The development of watermarking algorithms to the present day have many categories, such as Least Significant Bit (LSB) [1], phase-coding [2], spread spectrum [2], echo-hiding [3] and others. In this thesis, we will focus on the spread spectrum audio watermarking algorithm. SS-based audio watermarking spreads the information throughout the spectrum of the host signal to embed watermark. During the extraction process, we calculate the correlation between spreading sequence and watermarked signal. However, SS-based audio watermarking has two problems, one is host signal interference problem which could reduce the robustness of watermark extraction another is low embedding capacity problem which in order to achieve high robustness of watermark extraction leads

to low embedding capacity. To deal with host signal interference problem there have several methods such as improved spread spectrum [4,5], multiplicative spread spectrum [6,7], cross spread spectrum [8], improved cross spread spectrum [8].

In order to create embedding capacity, Yong-Xiang and Iynkaran Natgunanathan proposed new SS-based audio watermarking method [9,10]. Both methods generate difference orthonormal pseudo-noise sequences embed watermark into the DCT audio segments that apply discrete cosine transform (DCT) to the host signal to get. To generate PN-sequence the method [9] create a suitable length sequence and the rotationally shifting it to formed the rest number of PN-sequence, but its robustness of watermark extraction degraded under severe noise attacks. In method [10], it uses Gram-Schmidt orthogonalization method to generate PN-sequence. This method enhances the robustness against severe attacks and maintain high embedding capacity, but the disadvantage is that it has security problem because the PN-sequence is similar to identity matrix. In order to figure out this security problem, Yong-Siang Syu propose another method that generate PN-sequence by complementary codes [11]. This method although figure out the security problem but its computational cost of scaling factor is much more than method [10].

In this thesis, the orthogonal PN-sequence will generate by using the same complementary codes [12,13] with [11] but the difference section is we have multi-layer watermark can embed. Moreover, the proposed method not only process lower bit error rate than [11] method but also maintain security and capacity.

MATERIALS AND METHODS

The SS-based audio watermarking divided into two parts: watermark embedding and watermark extraction. Figure1 shows the block diagram of the proposed embedding method and Figure2 shows block diagram of the proposed extraction method.

At the watermark embedding first, we need to apply discrete cosine transform (DCT) to transform the host signal $x(n)$ to DCT coefficients $X(k)$. The host signal $x(n)$ is transformed into do DCT coefficients $X(k)$ according to fomular:

$$Y(k) = E(m) \sum_{n=0}^{L-1} x(n) \cos\left(\frac{m\pi(2n+1)}{2L}\right), \quad m = 0, 1, \dots, L-1 \quad (1)$$

$$E(m) = \begin{cases} \frac{1}{\sqrt{L}}, & \text{if } m = 0 \\ \sqrt{\frac{2}{L}}, & \text{if } 1 \leq m < L \end{cases} \quad (2)$$

After transformation, the corresponding discrete cosine coefficients M_{low} is obtained according to the desired starting frequency f_{low} using following functions:

$$\begin{aligned} f_{step} &= \frac{f_s}{L} \\ [M_{low}] &= [\text{ceil}(\frac{f_{low}}{f_{step}})] \end{aligned} \quad (3)$$

In this thesis we select f_{low} between 1000Hz~2000Hz, starting at 1000Hz, an objective difference grade (ODG) is calculated per 100Hz to find the starting frequency of the best objective difference grade.

Then we will show the production process of PN-sequences using the algorithm of complementary codes [10, 11, 12]. In the beginning we assumed A, B and D to be a $N \times N$ orthogonal matrix, the elements of matrix are complex and the norm of them are 1.

$$\begin{aligned} A &= \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_N \end{bmatrix}, \quad |a_{ij}| = 1, \text{ for } i, j = 1, 2, \dots, N \\ B &= \begin{bmatrix} B_1 \\ B_2 \\ \vdots \\ B_N \end{bmatrix}, \quad |b_{ij}| = 1, \text{ for } i, j = 1, 2, \dots, N \\ D &= \begin{bmatrix} D_1 \\ D_2 \\ \vdots \\ D_N \end{bmatrix}, \quad |d_{ij}| = 1, \text{ for } i, j = 1, 2, \dots, N \end{aligned} \quad (4)$$

Then we can use matrix A and B to create N sequences of length N^2 by:

$$\begin{aligned} C_1 &= (b_{11}A_1, b_{12}A_2, \dots, b_{1N}A_N) = (c_{11}, c_{12}, \dots, c_{1N^2}) \\ C_2 &= (b_{21}A_1, b_{22}A_2, \dots, b_{2N}A_N) = (c_{21}, c_{22}, \dots, c_{2N^2}) \\ &\vdots \\ C_N &= (b_{N1}A_1, b_{N2}A_2, \dots, b_{NN}A_N) = (c_{N1}, c_{N2}, \dots, c_{NN^2}) \end{aligned} \quad (5)$$

After create C sequence, we use C sequence and orthogonal matrix D to obtain the N^2 sequences of length N^2 by following functions:

$$\begin{aligned} E_{ij} &= [c_{i1}d_{j1}, c_{i2}d_{j2}, \dots, c_{iN}d_{jN}, c_{i(N+1)}d_{j1}, \dots, c_{i(2N)}d_{jN}, \dots, c_{i(N^2-N+1)}d_{j1}, \dots, c_{iN^2}d_{jN}] \\ &= [e_{ij1}, e_{ij2}, \dots, e_{ijN^2}] \text{ for } i, j = 1, 2, \dots, N \end{aligned} \quad (6)$$

Where i denote i th row and j denote j th column of A, B and D . Follow the functions of (5) and (6) we know every complementary code have N subcodes and each subcode' length is N^2 . Combine the sequences generated by (6) to obtain N complementary codes of the length N^3 by

$$\begin{aligned} E_1 &= [E_{11}, E_{12}, \dots, E_{1N}] \\ E_2 &= [E_{21}, E_{22}, \dots, E_{2N}] \\ &\vdots \\ E_N &= [E_{N1}, E_{N2}, \dots, E_{NN}] \end{aligned} \quad (7)$$

Howere, if the columns of the codes are not enough to use, we can use

$$\begin{aligned} F_i &= [e_{i11}, e_{i21}, \dots, e_{iN1}, e_{i12}, e_{i22}, \dots, e_{iN2}, \dots, e_{i1N^{r-1}}, e_{i2N^{r-1}}, \dots, e_{iNN^{r-1}}] \\ &= [f_{i1}, f_{i2}, \dots, f_{iN^r}] \text{ for } i = 1, 2, \dots, N \end{aligned} \quad (8)$$

to replace (5) sequences, then we can repeat (6) and (7) to calculate sequences of extended column.

Although we solve the column numbers problem, we have sequence numbers problem. To deal whit this problem, we rewrite (6):

$$\begin{aligned} E_{11} &= T_1 = [T_{11}, T_{12}, \dots, T_{1N^2}] \\ E_{12} &= T_2 = [T_{21}, T_{22}, \dots, T_{2N^2}] \\ E_{13} &= T_3 = [T_{31}, T_{32}, \dots, T_{3N^2}] \\ E_{14} &= T_4 = [T_{41}, T_{42}, \dots, T_{4N^2}] \\ &\vdots \\ E_{1N} &= T_N = [T_{N1}, T_{N2}, \dots, T_{NN^2}] \\ &\vdots \\ E_{NN} &= T_{N^2} = [T_{N^21}, T_{N^22}, \dots, T_{N^2N^2}] \end{aligned} \quad (9)$$

and we can increase the number of sequences by:

$$\begin{aligned} S_1 &= [T_{11}, T_{21}, T_{12}, T_{22}, \dots, T_{1l}, T_{2l}] = [S_{11}, S_{12}, \dots, S_{1(2l)}] \\ S_2 &= [T_{11}, \overline{T_{21}}, \overline{T_{12}}, \overline{T_{22}}, \dots, T_{1l}, \overline{T_{2l}}] = [S_{21}, S_{22}, \dots, S_{2(2l)}] \\ S_3 &= [T_{21}, T_{11}, T_{22}, T_{12}, \dots, T_{2l}, T_{1l}] = [S_{31}, S_{32}, \dots, S_{3(2l)}] \\ S_4 &= [T_{21}, \overline{T_{11}}, \overline{T_{22}}, \overline{T_{12}}, \dots, T_{2l}, \overline{T_{1l}}] = [S_{41}, S_{42}, \dots, S_{4(2l)}] \\ &\vdots \\ S_{4j+1} &= [T_{(2j+1)1}, T_{(2j+2)1}, T_{(2j+1)2}, T_{(2j+2)2}, \dots, T_{(2j+1)l}, T_{(2j+2)l}] \\ &= [S_{(4j+1)1}, S_{(4j+1)2}, \dots, S_{(4j+1)(2l)}] \\ S_{4j+2} &= [T_{(2j+1)1}, \overline{T_{(2j+1)1}}, T_{(2j+1)2}, \overline{T_{(2j+1)2}}, \dots, T_{(2j+1)l}, \overline{T_{(2j+1)l}}] \\ &= [S_{(4j+2)1}, S_{(4j+2)2}, \dots, S_{(4j+2)(2l)}] \\ S_{4j+3} &= [T_{(2j+2)1}, T_{(2j+1)1}, T_{(2j+2)2}, T_{(2j+1)2}, \dots, T_{(2j+2)l}, T_{(2j+1)l}] \\ &= [S_{(4j+3)1}, S_{(4j+3)2}, \dots, S_{(4j+3)(2l)}] \\ S_{4j+4} &= [T_{(2j+2)1}, \overline{T_{(2j+1)1}}, T_{(2j+2)2}, \overline{T_{(2j+1)2}}, \dots, T_{(2j+2)l}, \overline{T_{(2j+1)l}}] \\ &= [S_{(4j+4)1}, S_{(4j+4)2}, \dots, S_{(4j+4)(2l)}] \\ &\vdots \\ S_{2l-3} &= [T_{(l-1)1}, T_{l1}, T_{(l-1)2}, T_{l2}, \dots, T_{(l-1)l}, T_{ll}] = [S_{(2l-3)1}, S_{(2l-3)2}, \dots, S_{(2l-3)(2l)}] \end{aligned}$$

$$\begin{aligned}
S_{2l-2} &= [T_{(l-1)1}, \overline{T_{l1}}, T_{(l-1)2}, \overline{T_{l2}}, \dots, T_{(l-1)l}, \overline{T_{ll}}] = [S_{(2l-2)1}, S_{(2l-2)2}, \dots, S_{(2l-2)(2l)}] \\
S_{2l-1} &= [T_{l1}, T_{(l-1)1}, T_{l2}, T_{(l-1)2}, \dots, T_{ll}, T_{(l-1)l}] = [S_{(2l-1)1}, S_{(2l-1)2}, \dots, S_{(2l-1)(2l)}] \\
S_{2l} &= [T_{l1}, \overline{T_{(l-1)1}}, T_{l2}, \overline{T_{(l-1)2}}, \dots, T_{ll}, \overline{T_{(l-1)l}}] = [S_{(2l)1}, S_{(2l)2}, \dots, S_{(2l)(2l)}]
\end{aligned} \tag{10}$$

With above function we can generate mutually orthogonal PN-sequences. According to [15], we can rearrange complete complementary codes like:

$$\begin{aligned}
V^{(0)} &= (C_0^{(0)}, C_1^{(0)}, \dots, C_{n-1}^{(0)}) \\
V^{(1)} &= (C_0^{(1)}, C_1^{(1)}, \dots, C_{n-1}^{(1)}) \\
&\vdots \\
V^{(m-1)} &= (C_0^{(m-1)}, C_1^{(m-1)}, \dots, C_{n-1}^{(m-1)})
\end{aligned} \tag{11}$$

In order to embed watermark bits we need to create basic sequence [16] and then we need to shift it. A basic sequence $S^{(i)}$ is defined as:

$$S^{(i)} \stackrel{\text{def}}{=} \sum_{j=0}^{n-1} C_j^{(i)} Z^{-jT} \tag{12}$$

for any integer i satisfying $0 \leq i \leq m-1$. Z^{-jT} represents the $C_j^{(i)}$ shift by jT chips. Then we should embed watermark $d_t^{(i)} \stackrel{\text{def}}{=} (d_0^{(i)}, d_1^{(i)}, \dots, d_{B^{(i)}-1}^{(i)}) \in \{+1, -1\}^{B^{(i)}}$, $B^{(i)}$ denote the number of watermark. The convoluted sequence [16] $W^{(i)}$ is calculated as:

$$W^{(i)} \stackrel{\text{def}}{=} \sum_{t=0}^{B^{(i)}} \alpha_t d_t^{(i)} S^{(i)} Z^{-t} \tag{13}$$

where α_t is scaling factor, decided by Table 1.

Finally, we want to embed multi-layer water the embedded sequence W_N' is modulated as:

$$W_N' = \sum_{i=0}^{m-1} W^{(i)} \tag{14}$$

the embedded signal in DCT-domain will be

$$\tilde{X} = \beta Y + \beta W_N' \tag{15}$$

where β is the factor to adjust ODG to -0.7. Y denotes the host signal through DCT.

In the watermark extraction, we apply DCT to obtain DCT-domain embedded signal $\tilde{X}$.
The we can compute every watermarks of each layer by (15) and (16) as

$$R_{S^{(i)} \tilde{X} Z^{-j}} = \sum_{j=0}^{B^{(i)}-1} S^{(i)} \tilde{X} Z^{-j} \quad (15)$$

$$d_w^{(i)} = \text{sgn}(R_{S^{(i)} \tilde{X} Z^w}) \quad \text{and} \quad \text{sgn}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x \leq 0 \end{cases} \quad (16)$$

Table 1

Selection of α

Input: Embedded frequency range of original signal X_i , Basic sequence , watermark bit d

Output: α_i

Step:

1.

Compute

$$R = X_i S_i Z^{-i}$$

2.

If $R < 0$, $R = R - I$

3.

$$\alpha = \text{ceil}(R) ./ \text{CCC number}(n) * \text{CCC bits}(l) * d_{(i)}$$

4.

If $\alpha > 0$, $\alpha = \text{abs}(\min(\alpha))/10$

5.

$$A = \text{abs}(\alpha)$$

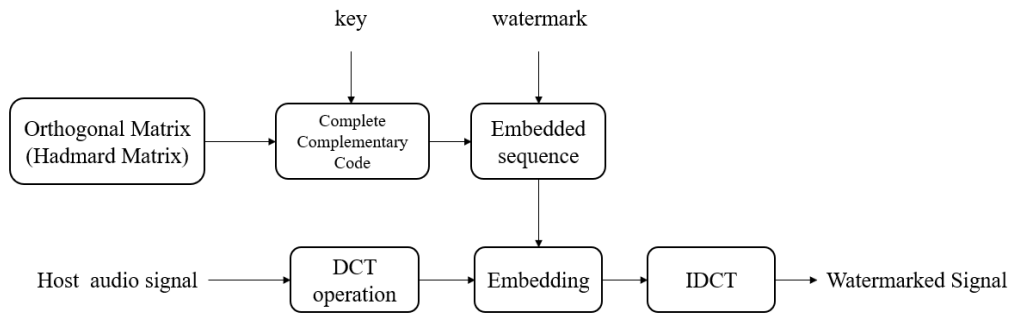


Figure 1. Block diagram of the proposed embedding method

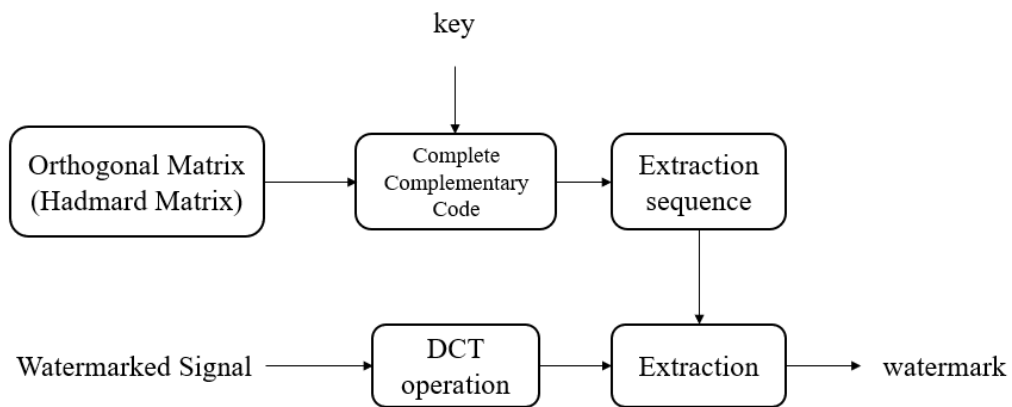


Figure 2. Block diagram of the proposed extraction method

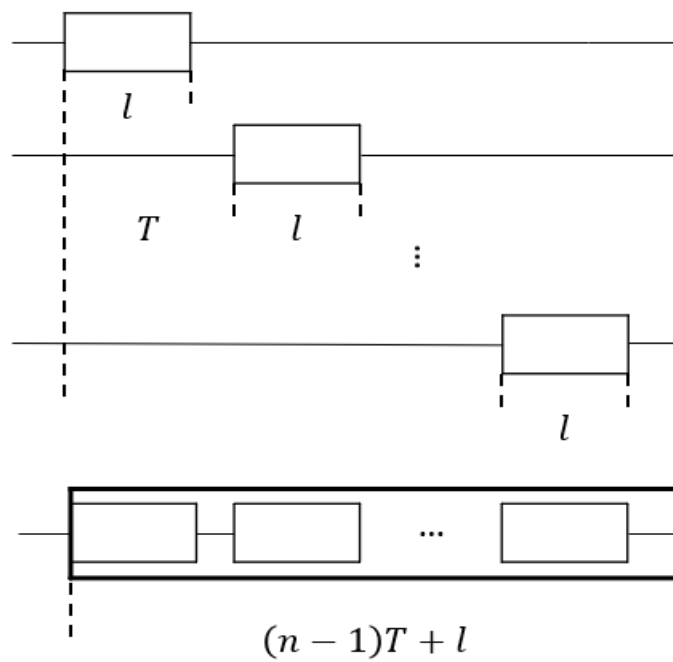


Figure 3. The basic sequence of complementary codes

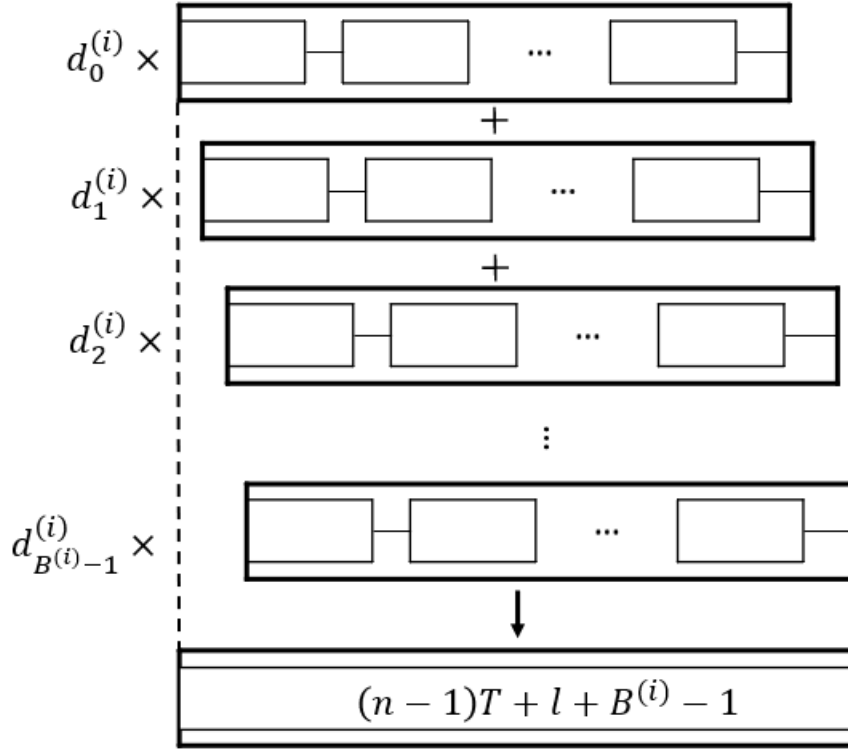


Figure 4. The convoluted sequence

RESULTS AND DISSCUSSION

In this section, we will use 10 randomly selected audio files to be the host signal, then we will embed the same watermark at each algorithm. In order to measure the robustness of watermark extraction we apply the bits error ratio and we set the same perceptual quality with ODG = -0.7. Table 2 shows the average bits error ratio comparison of method [9]、[10] and [11]. Table 3 shows the computational time taken of generating PN sequence. Table 4 shows the computational time taken of scaling factor.

$$BER(\%) = \frac{\text{Number of wrong bits}}{\text{Number of watermark bits}}$$

(16)

Table 2. Average Bits Error Ratio of the propose method and method [9] [10] and [11]

algorithm	Circular Shift [9]	Gram-Schmidt [10]	Complete Complementary Code [11]	Proposed
Test attack	Average Bits Error Ratio (%)			
Closed-Loop	0	1.6931	0.2646	0
Noise(15dB)	2.0900	1.7990	0.3704	0
Noise(10dB)	10.2384	2.5397	0.9788	0.0481
Noise(5dB)	20.0529	7.4868	5.8995	0.5769
Noise(0dB)	33.0003	17.6984	15.3439	2.8606
Amplitude (1.2)	3.9683	1.6931	0.2646	0
Amplitude (1.8)	0	1.8519	0.2646	0
Re-quantization	0	1.6138	0.2646	0
Low-pass Filter $f_c = 8000Hz$	0	1.6931	0.2646	0
High-pass Filter $f_c = 100Hz$	6.2646	1.6931	0.2646	0
AAC (128kbps)	0.8201	2.0900	0.3704	0
AAC (96kbps)	1.2169	1.6402	0.4233	0

Table 3 The computational time taken of generating PN sequence

	Circular Shift	Gram-Schmidt	Complete Complementary Code
Computation time (s)	0.0157	4.6171	0.077

Table 4 The computational time taken of scaling factor

	Method in [9]	Method in [10]	Method in [11]	Proposed
Computation time (s)	0.0117	0.0007	14.689	0.2539

CONCLUSION

In this thesis, although we use the method to generate the PN-sequence with [11] but the difference between those is that in this thesis, we have multi-layer watermark can embed. Moreover, the proposed method not only process lower bit error ratio than [9][10] and [11] method but also maintain security and capacity.

Reference

- [1] D. Lam, Audio watermarking. COMPSYS401A Project, The University of Auckland, 2003
- [2] W. Bender, D. Gruhl, N. Morimoto, A. Lu, Techniques for data hiding. IBM Systems Journal. vol. 35, nos 3&4, pp. 313-336, 1996.
- [3] h, H.O., et al. New echo embedding technique for robust and imperceptible audio watermarking. In Acoustics, Speech, and Signal Processing, 2001. Proceedings. (ICASSP'01). 2001 IEEE International Conference on 2001. IEEE.
- [4] H. S. Malvar and D. A. Florencio, "Improved spread spectrum: A new modulation technique for robust watermarking," *IEEE Trans. Signal Process.*, vol. 51, no. 4, pp. 898-905, April. 2003.
- [5] Zhang P, Xu S, Yang H.: Robust and transparent audio watermarking based on improved spread spectrum and psychoacoustic masking[C]. Information Science and Technology (ICIST), In: 2012 IEEE International Conference, 640-643 (2012)
- [6] A. Valizadeh and Z.J Wang. A framework of multiplicative spread spectrum embedding for data hiding: performance, decoder and signature design. in Global Telecommunications Conference, 2009. GLOBECOM 2009. IEEE. 2009. IEEE.
- [7] A. Valizadeh and Z.J. Wang, Correlation-and-bit-aware spread spectrum embedding for data hiding. IEEE Transactions on Information Forensics and Security, 2011. 6(2): pp. 267-282.
- [8] Y. Chen et al. A cross spread spectrum audio watermarking robust to acoustic transmission. in Control Conference (CCC), 2017 36th Chinese. 2017. IEEE.
- [9] Y. Xiang, et al., Spread spectrum-based high embedding capacity watermarking method for audio signals. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 2015. 23(12): pp. 2228-2237.
- [10] Y. Xiang, et al., Spread Spectrum Audio Watermarking Using Multiple Orthogonal PN Sequences and Variable Embedding Strengths and Polarities. IEEE/ACM Transactions on Audio, Speech and Language Processing (TASLP), 2018. 26(3): p. 529-539.
- [11] 徐詠翔, "利用互補碼方式之展頻音訊浮水印演算法," 台灣, 國立成功大學電機工程學系研究所, 2018.
- [12] Lin, J.-X., Complementary Code Based CDMA Architecture, in Department of Electrical Engineering. 2003, National Sun Yat-sen University.
- [13] N. Suehiro, and M. Hatori, N-shift cross-orthogonal sequences. IEEE Transactions on Information theory, 1988. 34(1): pp. 143-146.

- [14] 林金賢, “互補碼分碼多工系統之研究,” 台灣, 國立中山大學電機工程學系研究所, 2003.
- [15] R. Mayuzumi and T. Kojima, “A blind digital steganography scheme based on complete complementary codes,” Proc. Of IIHMSP 2011, Dalian, China, pp.45-48, Oct. 2011.
- [16] Kojima T, Oizumi A, Okayasu K, Parampalli U. An audio data hiding based on complete complementary codes and its application to an evacuation guiding system. *Signal Design and Its Applications in Communications, The Sixth International Workshop*. pp.118-121. 2013.



誌謝

猶記得六年前的我還在抉擇是否走上電機這條路，轉眼間大學四年過去了，如今正完成碩士論文，並且來到碩士生涯的末端，這一路上要感謝很多人的支持與陪伴，當中最感謝的是我的媽媽在我求學時候，總是不厭其煩的鼓勵我、資助我，感謝姊姊、弟弟、爺爺、奶奶以及我所有的家人們，陪伴、督促並且支持我完成我的碩士學位。

謝謝我的指導教授雷曉方教授，在兩年內對我的教導，並且在我寫碩士論文與研究上都給了我寶貴的建議，讓我能在論文的表現上呈現最好的一面，除此之外，也要感謝百忙之中撥空參與口試的口試委員，邱瀝毅教授、郭致宏教授以及姚書農教授，對我的論文提出建議，改進論文當中不足的地方，使我的論文更加完整。

同時還要感謝詠翔學長、新明學長、瑞傑學長、偉軒學長以及秉鴻學長，在我剛進實驗室時的耐心指導與提點。感謝這兩年陪我成長的夥伴尹鑾與俊修，我們一起經歷的碩一積體電路修課與碩二潛心研究的時光至今難忘，也培養了許多我們之間共同的興趣，希望接下來的日子你們都能有更好的未來。還有這兩年間我人生中的朋友們，念軒、至翔、冠甫、祐銅、許增、睿騰、宜緯，感謝你們一直以來的陪伴，讓枯燥的碩士生涯增添不少樂趣。

最後要感謝我的女朋友怡安，在這兩年間陪我經歷了碩士生活的點點滴滴，妳的陪伴與鼓勵我會銘記在心，恭喜在我碩士生涯的最後階段，妳也如願的考到正式老師。謝謝我人生中重要的家人、朋友以及女朋友，你(妳)們的包容、體諒以及鼓勵，成就了今天的我，希望能將這份喜悅分享給你(妳)們，也祝大家在未來的各個方面都能達到更好的成就。

游凱名 謹誌

於 民國一零八年七月

目錄

內容

中文摘要.....	I
EXTENDED ABSTRACT	II
誌謝.....	XIII
目錄.....	XIV
表目錄.....	XVI
圖目錄.....	XVII
第一章 緒論.....	1
1.1. 研究背景.....	1
1.2. 浮水印簡介.....	1
1.2.1. 最低有效位元調整.....	2
1.2.2. 相位編碼.....	3
1.2.3. 展頻法.....	4
1.2.4. 回聲編碼.....	4
1.3. 研究動機.....	5
1.4. 論文章節組織.....	6
第二章 相關文獻回顧與分析.....	7
2.1. 浮水印文獻回顧與相關介紹.....	7
2.2. 訊號干擾問題.....	10
2.2.1. 傳統型浮水印嵌入法.....	10
2.2.2. 加成型展頻浮水印嵌入法.....	12
2.2.3. 交互型展頻浮水印法.....	13
2.2.4. 加成交互型浮水印嵌入法.....	15
2.3. 容量問題.....	17

2.3.1.	循環碼法	18
2.3.2.	Gram-Schmidt 正交法	23
2.3.3.	互補碼法	29
第三章		37
3.1.	浮水印架構及嵌入流程	37
3.2.	互補碼	38
3.2.1.	Hadamard 矩陣	38
3.2.2.	超級互補碼產生的例子	39
3.2.3.	超級互補碼的正交證明	40
3.3.	浮水印嵌入流程	48
3.3.1.	離散餘弦轉換與嵌入頻段選擇	48
3.3.2.	浮水印訊號與偽隨機碼序列之排列	49
3.3.3.	比例因數的選擇	54
3.3.4.	最終演算法比較之調整	55
3.4.	浮水印萃取流程	56
3.4.1.	離散餘弦轉換與萃取頻段選擇	56
3.4.2.	浮水印萃取演算法	57
第四章	多層互補碼演算法結果分析及比較	60
4.1.	感知音訊品質評估	60
4.2.	浮水印強健性測試	61
4.2.1.	強健性測試環境	61
4.2.2.	實驗結果比較與討論	71
4.3.	演算法時間與演算法時間複雜度討論	98
第五章	結論與未來展望	101
參考文獻		102

表目錄

表.3.3.3-1 比例因數設定演算法.....	54
表.4.1-1 客觀數據評估分數(ODG)對照表	61
表.4.2.1-1 音訊處理攻擊方式一覽表.....	66
表.4.2-2 強健性測試環境數據表.....	69
表.4.2-3 強健性測試演算法差異表	70
表.4.2.2-1 樣本一測試實驗數據.....	71
表.4.2.2-2 樣本一測試結果.....	72
表.4.2.2-3 樣本二測試實驗數據.....	73
表.4.2.2-4 樣本二測試結果.....	74
表.4.2.2-5 樣本三測試實驗數據.....	75
表.4.2.2-6 樣本三測試結果.....	76
表.4.2.2-7 樣本四測試實驗數據.....	77
表.4.2.2-8 樣本四測試結果.....	78
表.4.2.2-9 樣本五測試實驗數據.....	79
表.4.2.2-10 樣本五測試結果.....	80
表.4.2.2-11 樣本六測試實驗數據.....	81
表.4.2.2-12 樣本六測試結果.....	82
表.4.2.2-13 樣本七測試實驗數據.....	83
表.4.2.2-14 樣本七測試結果.....	84
表.4.2.2-15 樣本八測試實驗數據.....	85
表.4.2.2-16 樣本八測試結果.....	86
表.4.2.2-17 樣本九測試實驗數據.....	87
表.4.2.2-18 樣本九測試結果.....	88
表.4.2.2-19 樣本十測試實驗數據.....	89
表.4.2.2-20 樣本十測試結果.....	90
表.4.3-1 三種偽隨機碼序列產生演算法之時間複雜度比較表.....	98
表.4.3-4 偽隨機碼序列實際運算時機比較表.....	100
表.4.3-5 比例因數選擇實際運算時機比較表.....	100

圖目錄

圖 1.2.1-1 最低有效位元調整前後音訊與差異比較圖	3
圖 1.2.2-1 相位編碼前後音訊與差異比較圖	4
圖 1.2.4-1 回聲編碼嵌入方式示意圖	5
圖 2.1-1 浮水印架構圖	7
圖 2.1-2 展頻浮水印嵌入式意圖	8
圖 2.2.1-1 傳統型浮水印嵌入法架構圖	11
圖 2.2.3-1 交互型展頻浮水印嵌入法架構圖	13
圖 2.2.3-2 交互型展頻浮水印嵌入法示意圖	14
圖 2.3-1 高容量嵌入演算法示意圖	18
圖 2.3.1-1 高容量循環碼展頻浮水印演算法	18
圖 2.3.1-2 高容量循環碼展頻浮水印演算法浮水印嵌入示意圖	20
圖 2.3.1-3 循環碼嵌入法在餘弦頻域之嵌入前後比較圖	21
圖 2.3.1-4 循環碼嵌入法在時域之嵌入前後比較圖	21
圖 2.3.2-1 高容量 GRAM-SCHMIDT 展頻浮水印演算法架構圖	23
圖 2.3.2-2 高容量 GRAM-SCHMIDT 展頻浮水印演算法嵌入示意圖	25
圖 2.3.2-3 高容量 GRAM-SCHMIDT 展頻浮水印演算法萃取示意圖	26
圖 2.3.2-4 GRAM-SCHMIDT 展頻浮水印演算法在餘弦頻域之嵌入前後比較圖	27
圖 2.3.2-5 GRAM-SCHMIDT 展頻浮水印演算法在時域之嵌入前後比較圖	27
圖 2.3.2-6 GRAM-SCHMIDT 正交法所產生之序列一	28
圖 2.3.2-7 GRAM-SCHMIDT 正交法所產生之序列二	28
圖 2.3.3-1 高容量互補碼展頻浮水印演算法	29
圖 2.3.3-2 高容量互補碼展頻浮水印演算法嵌入示意圖	33
圖 2.3.3-3 高容量互補碼展頻浮水印演算法萃取示意圖	34
圖 2.3.3-4 互補碼展頻浮水印演算法在餘弦頻域之嵌入前後比較圖	35
圖 2.3.3-5 互補碼展頻浮水印演算法在時域之嵌入前後比較圖	35
圖 2.3.3-6 互補碼序列圖	36
圖 3.1-1 互補碼展頻浮水印嵌入架構圖	37
圖 3.3-1 本論文演算法浮水印嵌入架構圖	48
圖 3.3.2-1 互補碼基礎序列產生示意圖	50
圖 3.3.2-2 單層捲積序列產生示意圖	51
圖 3.3.2-3 多層捲積序列產生示意圖	52
圖 3.3.2-4 浮水印嵌入前後與時域差異值比較圖	52
圖 3.3.2-5 浮水印嵌入前後與頻域差異值比較圖	53

圖 3.4-1 浮水印萃取架構圖	56
圖 3.4.2 浮水印萃取示意圖	57
圖 3.4.3 正相關互補碼萃取範例	58
圖 3.4.4 互相關互補碼萃取範例	58
圖 4.1-1 感知音訊品質評估概念示意圖	60
圖 4.2.1-1 強健性測試樣本一	61
圖 4.2.1-2 強健性測試樣本二	62
圖 4.2.1-3 強健性測試樣本三	62
圖 4.2.1-4 強健性測試樣本四	63
圖 4.2.1-5 強健性測試樣本五	63
圖 4.2.1-6 強健性測試樣本六	64
圖 4.2.1-7 強健性測試樣本七	64
圖 4.2.1-8 強健性測試樣本八	65
圖 4.2.1-9 強健性測試樣本九	65
圖 4.2.1-10 強健性測試樣本十	66
圖 4.2.1-11 六階 BUTTERWORTH 高通濾波器($f_c = 100Hz$)	68
圖 4.2.1-12 六階 BUTTERWORTH 低通濾波器($f_c = 8000Hz$)	68
圖 4.2.2-1 樣本一嵌入浮水印頻率前後差值比較圖	91
圖 4.2.2-2 樣本二嵌入浮水印頻率前後差值比較圖	91
圖 4.2.2-3 樣本三嵌入浮水印頻率前後差值比較圖	92
圖 4.2.2-4 樣本四嵌入浮水印頻率前後差值比較圖	92
圖 4.2.2-5 樣本五嵌入浮水印頻率前後差值比較圖	93
圖 4.2.2-6 樣本六嵌入浮水印頻率前後差值比較圖	93
圖 4.2.2-7 樣本七嵌入浮水印頻率前後差值比較圖	94
圖 4.2.2-8 樣本八嵌入浮水印頻率前後差值比較圖	94
圖 4.2.2-9 樣本九嵌入浮水印頻率前後差值比較圖	95
圖 4.2.2-10 樣本十嵌入浮水印頻率前後差值比較圖	95
圖 4.2.2-11 將離散餘弦轉換後係數取絕對值圖	96
圖 4.2.2-13 本演算法之四層比例因數選擇圖	97
圖 4.3-1 MATLAB BENCHMARK 數據圖	99
圖 4.3-2 MATLAB BENCHMARK 相對執行速度直方圖	99

第一章 緒論

本章節在章節 1.1 時會探討研究背景其中包括浮水印的用途，接著在章節 1.2 分別介紹浮水印的種類、浮水印演算法須具備的特性與不同浮水印的方法簡介，章節 1.3 為研究動機以及目的，最後章節 1.4 為論文章節組織。

1.1. 研究背景

數位音訊浮水印，顧名思義即是將浮水印的技術運用於數位音訊之中，而此技術是將浮水印訊號(如圖片、訊息…)嵌入於一段須受保護的音檔之中。近年來科技的蓬勃發展、網路的普及使我們在生活中可以取得更多的資訊，然而也因此網路中，經常散佈著非經過著作者同意的智慧財產(如圖片、音樂、影像…)，這已嚴重違反智慧財產權，並侵犯著作者權益。有鑑於此，為了使著作者的智慧財產受到應有的保護，學者紛紛提出可辨識所有權的浮水印方法，用以將浮水印嵌入須被保護的智慧財產當中，若產生版權爭議時，運用相對的演算法將所嵌入的浮水印萃取出來，便能證明財產的所有人，以化解版權糾紛，這樣即可以達到保護智慧財產權的目的。

1.2. 浮水印簡介

浮水印技術根據其不同特性，可以分為許多同種類，而每個種類可以組合成不同特性的浮水印：

1. 圖片、影像、文字、音訊

現今許的不同的數位媒體都可以使用浮水印技術，這些數位媒體例如：圖片、影像、文字、音訊…等[1]。而圖片的浮水印技術在浮水印開始至今，已經廣為發展。影像的語音浮水印與圖片浮水印的關係甚近，大多現行的技術，將影像分成圖片，而使用圖片浮水印的技術，完成影像浮水印。文字浮水印在任何有版權的地方，皆有被使用。然而近年語音浮水印也愈來愈被關注，並快速的發展中，語音浮水印是本研究所用的方法，詳細情形會在後續章節中介紹。

2. 不可感知型(Imperceptible)、可感知型(perceptible)

對於圖片與影像，可感知的浮水印例如：標誌，將其合併到圖像的一隅，而其可以察覺且容易實現，但數位浮水印並不著重於此，而數位浮水印是意圖將浮水印在不

被察覺之情況下放入媒體當中。

3. 強健型(Robust)、半脆弱型(Semi-fragile)、脆弱型(Fragile)

強健型的浮水印在原圖未被破壞之下是難以去除的，通常用於版權保護、所有權之驗證或其他安全性上之應用。半脆弱型的浮水印則是相較之下較容易被修改的，主要用於數據之認證。脆弱型浮水印則是對於攻擊非常敏感的一種形式。

4. 盲浮水印(Blind)、非盲浮水印(Non-blind)

盲浮水印在浮水印萃取時，不需要原訊號即可解出，反之需要藉由原訊號方可萃取浮水印之技術，稱之為非盲訊號浮水印。

然而對於研究中語音訊號浮水印的演算法，需要有下列特性[2]:

➤ 不可感知型(Imperceptible)

為了維持良好的通透性，浮水印資料必須是肉眼難以察覺的，並且要將對原訊號之影響降到最低，就此使人不易察覺，以達到隱藏資訊的目的

➤ 強健性(Robustness)

一個可靠的浮水印演算法，在基於保護版權的情況下，浮水印必須再經由不同音訊處理以後，依然能將隱藏的浮水印資訊正常的還原，以確保能維護版權。而一般的音訊訊號處理包括：雜訊(Noise)、高通濾波器(High-pass filter)、低通濾波器(Low-pass filter)、調幅(Amplitude)、音訊壓縮(例如 MP3、AAC)…等。

➤ 安全性(Security)

語音浮水印由於是使用特定的演算法嵌入資訊，這也代表不肖攻擊者能藉由逆向工程取得浮水印資訊，所以浮水印的演算法必須具備特定的密鑰，以防止此類種情況發生，倒置無法保護版權。

➤ 容量(Data Payload)

容量也就是一個需受保護之語音檔案，能夠嵌入多少浮水印資訊，而通常定義的單位為每秒多少位元(Bits per second, bps)。而我們希望浮水印資訊嵌入越多越好，但相反的是容量越大則相對影響強健性以及語音品質，所以我們必須從中做出取捨。

➤ 運算複雜度(Computational Complexity)

運算複雜度也就是說一個演算法的運算快慢，如果演算法的計算時間能達到較為快速，也就代表效能相對的優秀，對於一個語音演算法是一個重要的指標。

1.2.1. 最低有效位元調整

較早且較直接的浮水印嵌入演算法[3]，其將浮水印嵌入時域[4]中，最低有效位

元演算法將浮水印轉為二進位元之字串，以取代每個取樣音訊的最低有效位元(Least Significant Bit)。然而此方法雖然具有浮水印嵌入的大容量，但其面對音訊處理是非常不可抗拒(irresistible)的。

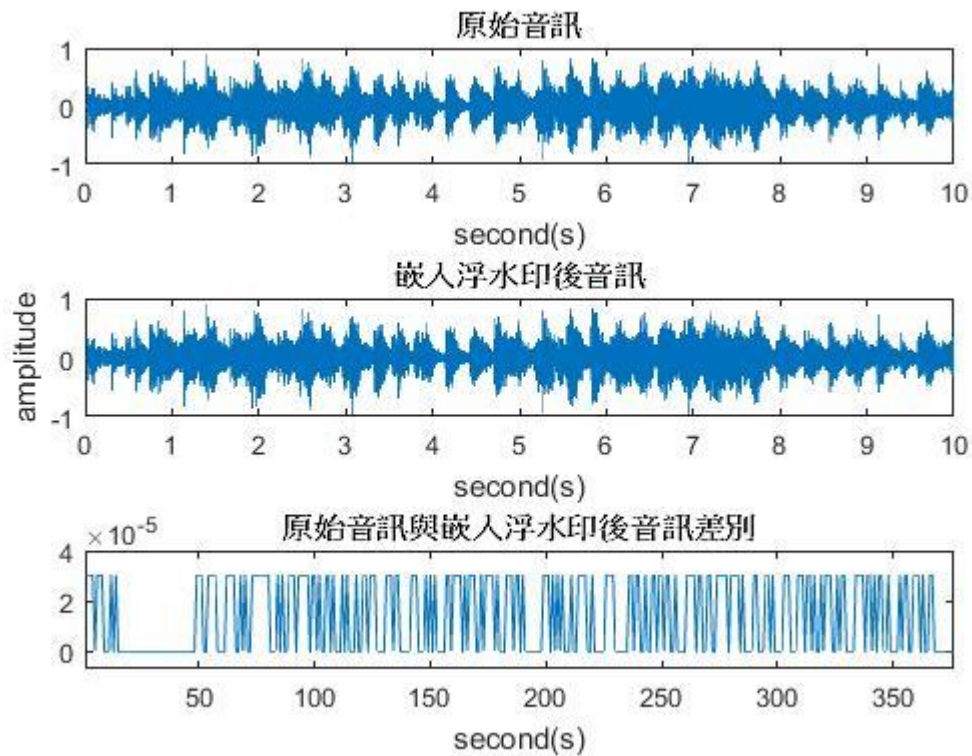


圖 1.2.1-1 最低有效位元調整前後音訊與差異比較圖

1.2.2. 相位編碼

基於人類感知聽覺系統(Human auditory system, HAS)人耳無法分辨絕對的相位變化，而此方法保持參考相位不變的情況之下，將浮水印嵌入原音訊之相位中，而此方法之缺點為解碼複雜[5]。

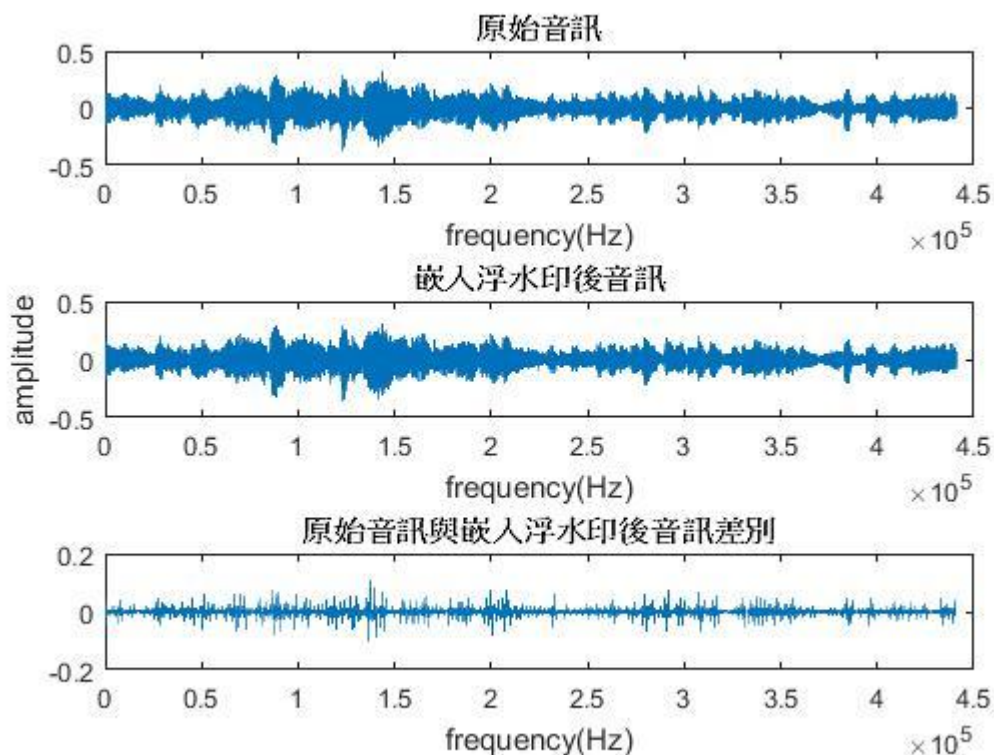


圖 1.2.2-1 相位編碼前後音訊與差異比較圖

1.2.3. 展頻法

展頻法被為現行較為流行的數位浮水印方法，此方法類似通訊展頻，其將浮水印擴展至原訊號的頻域[6]當中，也使得其信號能量非常微小並且難以察覺，因為如此展頻法有著安全且穩健的性質，但反之，浮水印的嵌入容量也有所限制。而展頻法又分為直序列展頻(Direct-Sequence Spread Spectrum)[7]與跳頻(Frequency-Hopping Spread Spectrum)[8]，直接序列展頻法是本研究所用之方法，詳細的展頻法會在後續章節再做介紹。

1.2.4. 回聲編碼

回聲編碼[9]藉由不同的回聲將浮水印嵌入原訊號當中，其原理是將原訊號分成不同的區段，而根據區段所對應到的浮水印位元訊號(1 or 0)，如圖 1.2.4-1 所示，以不同的延遲時間(如圖所示的 d_0 與 d_1 ，而 α 表示衰減常數)，嵌入原訊號當中。然而必須設計出延遲得參數以及衰減常數，以避免聆聽者能聽出音訊的差別。此方法的缺點在於攻擊者可以使用自相關(auto-correlation)方法解出浮水印的資訊，此點在安

全上有所疑慮。

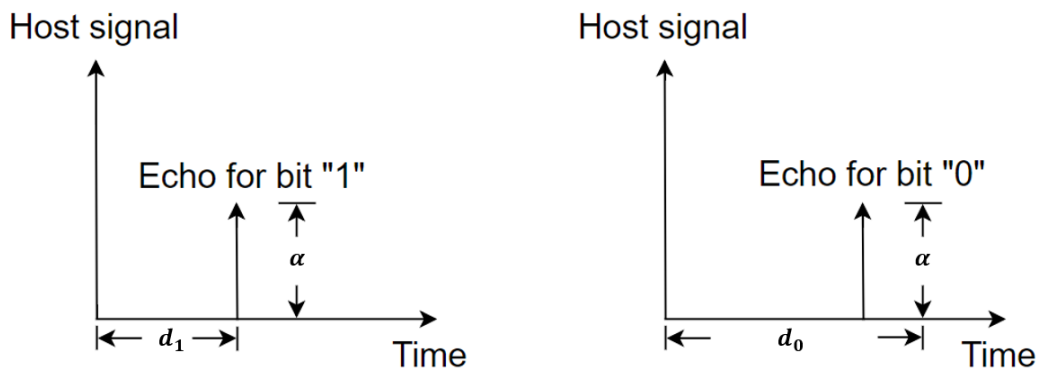


圖 1.2.4-1 回聲編碼嵌入方式示意圖

1.3. 研究動機

現行的展頻浮水印演算法不管是對於雜訊干擾亦或者是容量的問題都有提出改善，雜訊干擾解決方面有加成型浮水印[10,11]、交互展頻浮水印[12]以及改進交互展頻浮水印[12]，容量的問題解決上有 Yong Xiang et al. 提出的演算法[13,14]，雖然具有相當好的還原率與嵌入容量，可是其面臨而來的問題是運算複雜度高並且安全性低，而在此之後徐詠翔[15]的論文提出以完全互補碼(Complete complementary Codes)解決這些問題，雖然解決了安全性的問題但還原率的方面還是有改進的空間，並且在密鑰的方面因為只存在著一組密鑰，所以要保護之檔案可能只能為一人所有，因此本研究希望發展出一個還原率高並且可以使多人分別持有密鑰的演算法。

1.4. 論文章節組織

本論文章節安排如下：

第一章為**緒論**，介紹本論文相關研究背景與動機。

第二章為**相關文獻回顧與分析**，介紹與本論文相關既有學術研究。

第三章為**本論文所提出之演算法**。

第四章為本論文所提出演算法之**模擬與探討**，將提出演算法與既有演算法比較。

第五章為**結論與未來展望與發展**，將對本論文內容提出總結與未來發展目標。



第二章 相關文獻回顧與分析

本章節主要討論與論文相關之文獻以及相關知識背景，其中包含章節 2.1 之浮水印相關介紹與傳統展頻浮水印的架構，並且將探討展頻浮水印訊號干擾以及容量問題，章節 2.2 針對展頻浮水印訊號干擾問題探討，並且提及目前學術上相關應對演算法，章節 2.3 會討論嵌入容量問題，並且提及目前學術上相關應對演算法。

2.1. 浮水印文獻回顧與相關介紹

直序列展頻(Direct-Spread Spectrum)[7]的傳統展頻架構如圖(2.1-1)所示，而直序列傳統展頻分為嵌入(Embedding)與萃取(Extraction)兩部分，這兩部分為傳統展頻的主要架構(虛線部分為不一定需要之步驟)，而原訊號(Host signal)的虛線可以將浮水印架構分為非盲浮水印與盲浮水印，處理(Process)則可將浮水印架構分為時域或頻域。

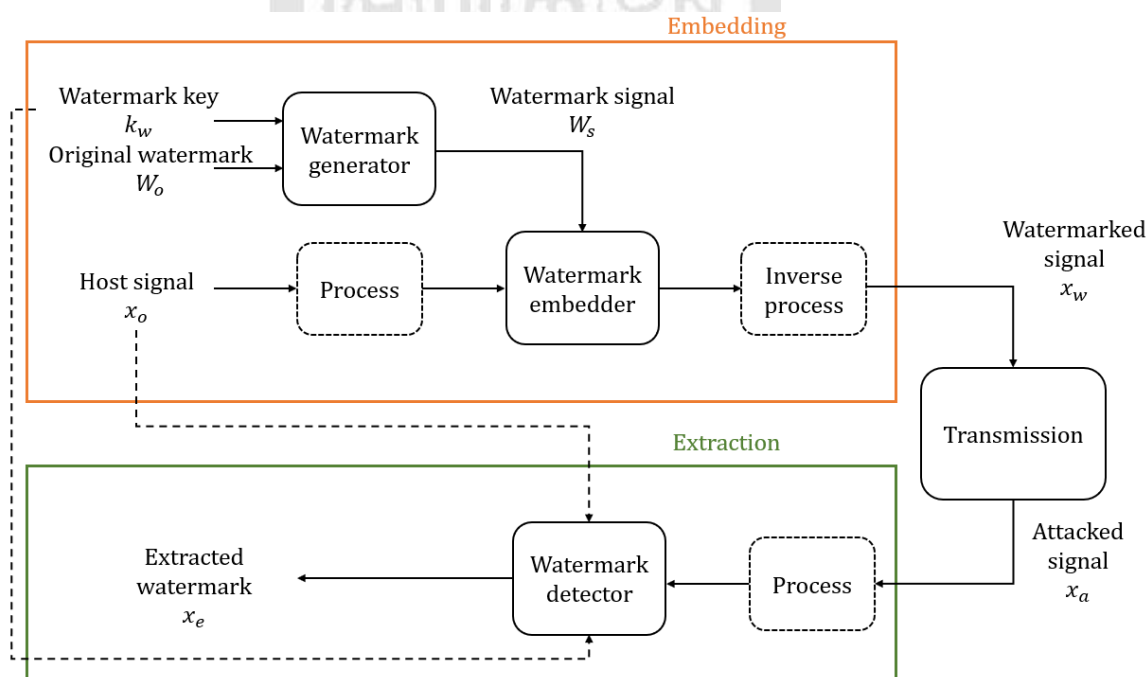


圖.2.1-1 浮水印架構圖

在傳統展頻的嵌入部分如圖中橘色框線所示，首先我們將原訊號(Host signal, x_0) 依 L_s 點做音框(frame)之分割成 N 個音框，而利用向量 $x_0 = [x_0, x_1, \dots, x_{N-1}]$ 可以表示為 N 個音框之向量，產生完 N 個音框之後我們需要將浮水印 W_0 嵌入其中，所以先將 W_0 轉成 N 個二進位的字串，再以只有產權所有者知道的密鑰

k_w ，也就是偽隨機碼序列(Pseudo-Noise Sequence)產生出浮水印訊號 $W_s = [w_0, w_1, \dots, w_{N-1}]$ ，最後我們將浮水印訊號乘上一個與音品質的比例因數值(Scaling Factor) α 與密鑰 k_w ，其中 α 值介於0到1之間。將產生出的浮水印訊號乘上 α 再加上 x_0 ，可以產生出式(2.1-1)的加入浮水印後訊號 x_w (Watermarked Signal)，嵌入示意圖如圖 2.1-2 所示。

$$x_w = x_0 + \alpha W_s = x_0 + \alpha W_o k_w \quad (2.1-1)$$

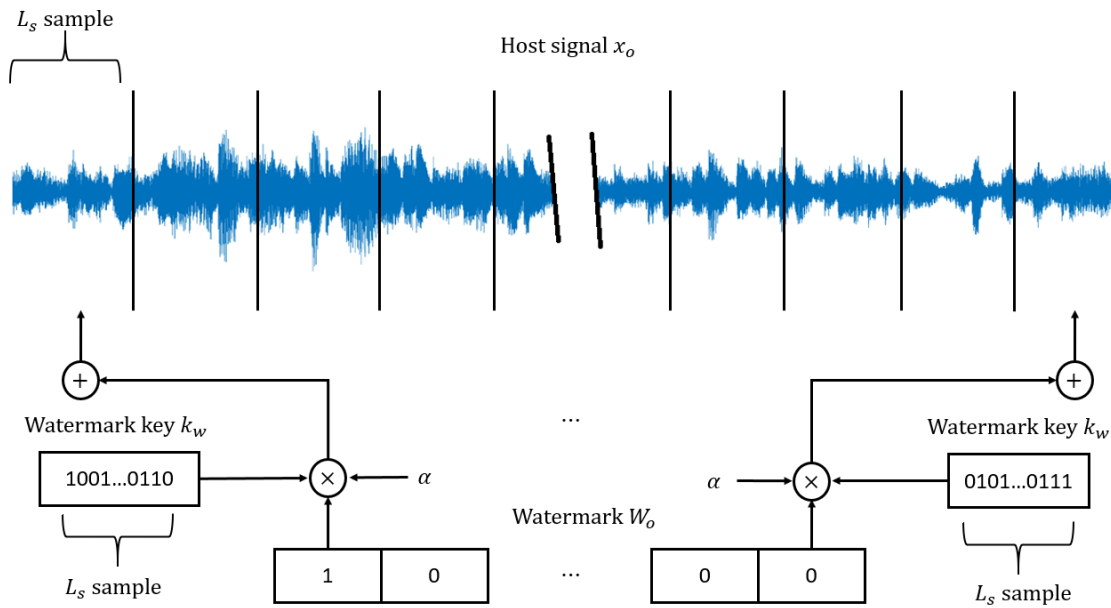


圖 2.1-2 展頻浮水印嵌入示意圖

另外是萃取部分，我們將經過傳輸或者音訊處理(如：壓縮、雜訊、濾波器、調幅...等)過的訊號 x_a 分割成 N 個 L_s 點的音框，以便後續萃取的處理，而這些分割好的音框，我們用 $x_a = [x'_0, x'_1, \dots, x'_{N-1}]$ 表示，最後再計算浮水印時使用密鑰 k_w 與 x_a 以式(2.1-2)的方式計算，其中 k_w^T 表示 k_w 的轉置(Transpose)， $\|k_w\| = k_w k_w^T$ 。

$$r = \frac{\langle x_a, k_w \rangle}{\langle k_w, k_w \rangle} = \frac{x_a k_w^T}{\|k_w\|} \quad (2.1-2)$$

而若考慮沒有音訊傳輸或處理的情況，則我們可以視 $x_a = x_w$ ，因此我們可以将式(2.1-1)代入式(2.1-2)並且整理成式(2.1-3)，其中 $\hat{x} = \frac{x_o k_w^T}{\|k_w\|}$ ， r 為統計決定值(decision statistic)。

$$r = \frac{\langle (x_o + \alpha W_o k_w), k_w \rangle}{\|k_w\|} = \frac{x_o k_w^T}{\|k_w\|} + \alpha W_o = \hat{x} + \alpha W_o \quad (2.1-3)$$

接著我們討論若密鑰 k_w 與原訊號 x_o 彼此之間為獨立(Independent)關係，則 $\hat{x} = 0$ ，也就是我們可以將式(2.1-3)改寫為 $r = \alpha W_o$ ，最後我們可以利用式(2.1-5)將我們的嵌入的浮水印資訊 W_o 還原，而其中的 $\text{sign}()$ 為符號函數(Sign Function)。

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases} \quad (2.1-4)$$

$$x_e = \text{sign}(r) = \left(\frac{x_o k_w^T}{\|k_w\|} + \alpha W_o \right) = \text{sign}(\alpha W_o) \quad (2.1-5)$$

在介紹完沒有音訊傳輸或處理的情況之後，接下來我們要探討的是訊號干擾的部分(Interference Effect of the Host Signal)問題，也就是加入雜訊 n 後的影響，我們以式(2.1-6)表示之。我們接著也將統計決定值的式(2.1-3)改寫成式(2.1-7)，其中 $\hat{n} = \frac{n k_w^T}{\|k_w\|}$ 。

$$x_a = x_w + n = x_o + \alpha W_o k_w + n = x_o + \alpha W_o k_w + n \quad (2.1-6)$$

$$r = \frac{\langle (x_o + \alpha W_o k_w + n), k_w \rangle}{\|k_w\|} = \frac{x_o k_w^T}{\|k_w\|} + \alpha W_o + \frac{n k_w^T}{\|k_w\|} = \hat{x} + \alpha W_o + \hat{n} \quad (2.1-7)$$

在此我們若假設原訊號 x_o 與雜訊 n 的關係為不相關的高斯隨機過程(Uncorrelated Gaussian Random Processes)，也就是 $x_o \sim N(0, \sigma_x^2)$ 與 $n \sim N(0, \sigma_n^2)$ ，其中 σ_x^2 、 σ_n^2 分別為原訊號與雜訊的變異數(Variance)，就此我們可以將統計決定值 r 表示為式(2.1-8)， σ_u^2 為偽隨機碼序列的變異數。

$$r \sim N(m_r, \sigma_r^2), \quad m_r = E[r] = \alpha W_o, \quad \sigma_r^2 = \frac{\sigma_x^2 + \sigma_n^2}{L_S \sigma_k^2} \quad (2.1-8)$$

最後加入訊號干擾後的萃取資訊的錯誤率(Error Probability)如式(2.1-9)所示，其中的 $\text{erfc}()$ 為補誤差函數(Complementary Error Function)。

$$p = \frac{1}{2} \operatorname{erfc} \left(\frac{m_r}{\sigma_r \sqrt{2}} \right) = \frac{1}{2} \operatorname{erfc} \left(\sqrt{\frac{L_s \sigma_k^2}{2(\sigma_x^2 + \sigma_n^2)}} \right) \quad (2.1-9)$$

在討論完傳統浮水印的基本概念之後，我們可以簡略的得到一些相關資訊，根據式(2.1-9)可以看出浮水印的錯誤率與偽隨機碼序列的變異數成反比，而錯誤率與雜訊的變異量呈現正比的情形。另外還有一點所可以觀察出的是由圖(2.1-2)可以看出，浮水印的嵌入量會因為音框長度 L_s 而有所限制，並且在一個音框之中只嵌入了一位元的浮水印資訊，這顯然有點浪費，而另一方面也突顯出浮水印嵌入的容量問題，然而這些問題就是我們後續要探討的方向，在後面的章節中，會依序的代出訊號干擾以及容量上面的問題。

2.2. 訊號干擾問題

在章節 2.1 中我們提及了展頻浮水印存在雜訊影響問題，就目前學術上之演算法有加成型浮水印嵌入法(Additive Spread Spectrum)、交互展頻浮水印嵌入法(Cross Spread Spectrum)、改進交互型浮水印嵌入法(Improved Cross Spread Spectrum)，然而在介紹這些演算法之前我們會先提到傳統型浮水印嵌入法(Traditional Spread Spectrum)。

2.2.1. 傳統型浮水印嵌入法

傳統型浮水印嵌入法[7]架構圖如圖(2.2.1-1)所示，傳統浮水印演算法如式(2.2.1-1)所示，此種方法為較早期的浮水印演算法。為了比較接下來要討論的三種方法，我們再次回顧 2.1 章節所提及的統計決定值、變異數與錯誤率。

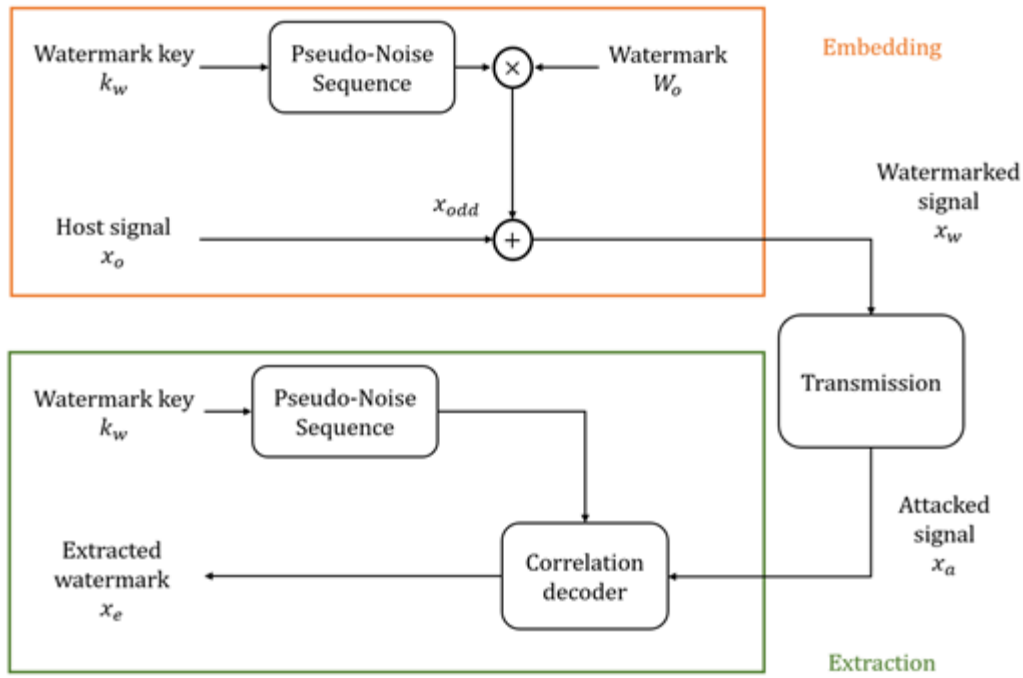


圖 2.2.1-1 傳統型浮水印嵌入法架構圖

$$x_a = x_w + n = x_o + \alpha W_s + n = x_o + \alpha W_o k_w + n \quad (2.2.1-1)$$

在考慮雜訊 n 的影響後，為了得到萃取之錯誤率我們將(2.2.1-1)帶入(2.1-2)得到

$$r = \frac{\langle (x_o + \alpha W_o k_w + n), k_w \rangle}{\|k_w\|} = \frac{x_o k_w^T}{\|k_w\|} + \alpha W_o + \frac{n k_w^T}{\|k_w\|} = \hat{x} + \alpha W_o + \hat{n} \quad (2.2.1-2)$$

$$r \sim N(m_r, \sigma_r^2), \quad m_r = E[r] = \alpha W_o, \quad \sigma_r^2 = \frac{\sigma_x^2 + \sigma_n^2}{L_S \sigma_k^2} \quad (2.2.1-3)$$

最後由(2.2.1-3)所得出的平均數以及變異數代入萃取錯誤率的公式，得到(2.2.1-4)的運算結果。

$$p = \frac{1}{2} \operatorname{erfc} \left(\frac{m_r}{\sigma_r \sqrt{2}} \right) = \frac{1}{2} \operatorname{erfc} \left(\sqrt{\frac{L_S \sigma_k^2}{2(\sigma_x^2 + \sigma_n^2)}} \right) \quad (2.2.1-4)$$

回顧完傳統型浮水印嵌入法後我們將在下面章節提及另外的演算法，並且討論訊

號干擾的問題。

2.2.2. 加成型展頻浮水印嵌入法

在前面的討論我們已經了解雜訊會影響萃取資訊時的錯誤率，然而在加成型展頻浮水印(Additive Spread Spectrum)[10, 11]的演算法中，提出了以修改嵌入演算法的方程式(2.2.2-1)，用以降低雜訊在萃取時的影響，演算法中的 α 、 λ 是用來調整語音品值的參數，若要考慮與原本的傳統展頻品質相同，則 α 可以推導出為式(2.2.2-2)。

$$x_w = x_o + (\alpha W_o - \lambda x_o)p \quad (2.2.2-1)$$

$$\alpha = \sqrt{1 - \frac{\lambda^2 \sigma_x^2}{N \sigma_k^2}} \quad (2.2.2-2)$$

將語音品質調整完後，我們接著討論雜訊 n 的影響，然而步驟與前一小節相似，將式(2.2.2-1)代入(2.1-2)可以得到：

$$r = (1 - \lambda) \frac{x_o k_w^T}{\|k_w\|} + \alpha W_o + \frac{n k_w^T}{\|k_w\|} \quad (2.2.2-3)$$

$$r \sim N(m_r, \sigma_r^2), \quad m_r = E[r] = \alpha W_o, \quad \sigma_r^2 = \frac{\sigma_n^2 + (1 + \lambda)^2 \sigma_x^2}{L_S \sigma_k^2} \quad (2.2.2-4)$$

最後由(2.2.2-4)所得出的平均數以及變異數代入萃取錯誤率的公式，得到(2.2.2-5)的運算結果。

$$p = \frac{1}{2} \operatorname{erfc} \left(\frac{m_r}{\sigma_r \sqrt{2}} \right) = \frac{1}{2} \operatorname{erfc} \left(\sqrt{\frac{L_S \sigma_k^2 - \lambda^2 \sigma_x^2}{2(\sigma_x^2 + (1 - \lambda)^2 \sigma_n^2)}} \right) \quad (2.2.2-5)$$

在得出式(2.2.2-5)後與式(2.2.1-4)比較後可以發現，此演算法可以在偽隨機碼序列與原始音訊訊號不變的情況下，降低訊號干擾的問題。

2.2.3. 交互型展頻浮水印法

若觀察式(2.2.1-4)的萃取錯誤率方程式，我們可以發現影響錯誤率的原因有：雜訊、原始訊號和偽隨機碼序列，再觀察加成型展頻浮水印嵌入法，其利用改變浮水印嵌入的方程式，將雜訊的影響成分降低，然而在交互型展頻浮水印嵌入法是藉由改變嵌入浮水印的方式，達到將原始訊號的影響降低，以降低萃取錯誤率，演算法架構如圖(2.2.3-1)所示。

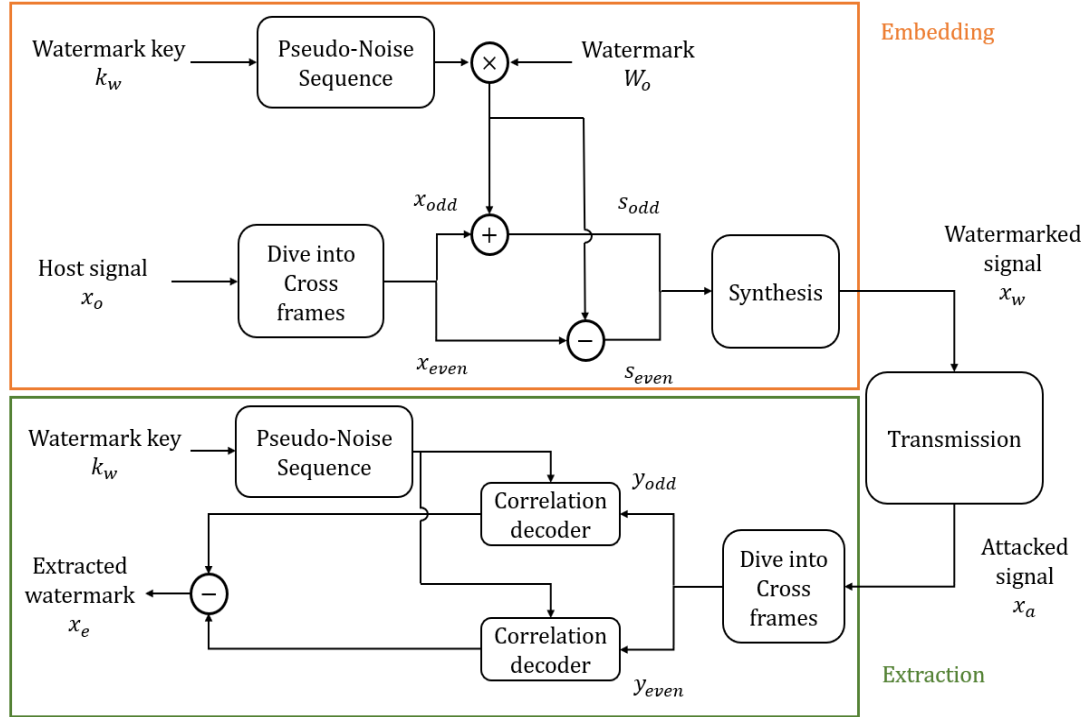


圖 2.2.3-1 交互型展頻浮水印嵌入法架構圖

交互型展頻浮水印[12]的演算法概念是為了消除原訊號對於浮水印解出所造成的影響，於是以相鄰兩個音框具有相似度較高的特性，將相同浮水印訊號嵌入兩個相鄰的音框以達成目的。嵌入的方法大致與原本的展頻浮水印相同，差異在於浮水印必須是兩兩相對的，也就是說訊號被經由 L_s 點切分為 $2N$ 個音框，再將2個音框視為同一組嵌入浮水印，而兩個一組的音框又被分為奇數(odd)與偶數(even)音框，如式(2.2.3-1)，然而式中的下標 o 代表奇數(odd)，下標 e 代表偶數(even)，式(2.2.3-2)是將訊號分成 X_{odd} 、 X_{even} 方便後續的演算法討論。

$$\mathbf{x} = [x_{o1}, x_{e1}, \dots, x_{oN}, x_{eN}] \quad (2.2.3-1)$$

$$\begin{cases} X_{odd} = [x_{o1}, x_{o2}, \dots, x_{oN}] \\ X_{even} = [x_{e1}, x_{e2}, \dots, x_{eN}] \end{cases} \quad (2.2.3-2)$$

在音框分配完成後，即將嵌入浮水印訊號，就如同前述所提的，我們將同一個浮水印訊號，嵌入一組由奇數、偶數組成的音框，如(2.2.3-3)所示，在嵌入完成之後我們會再將訊號合成為一個嵌入完畢的訊號 X_w ，整個演算法示意圖如圖(2.2.3-2)所示。

$$\begin{aligned} s_{odd} &= x_{odd} + bk_w \\ s_{even} &= x_{even} - bk_w \end{aligned} \quad (2.2.3-3)$$

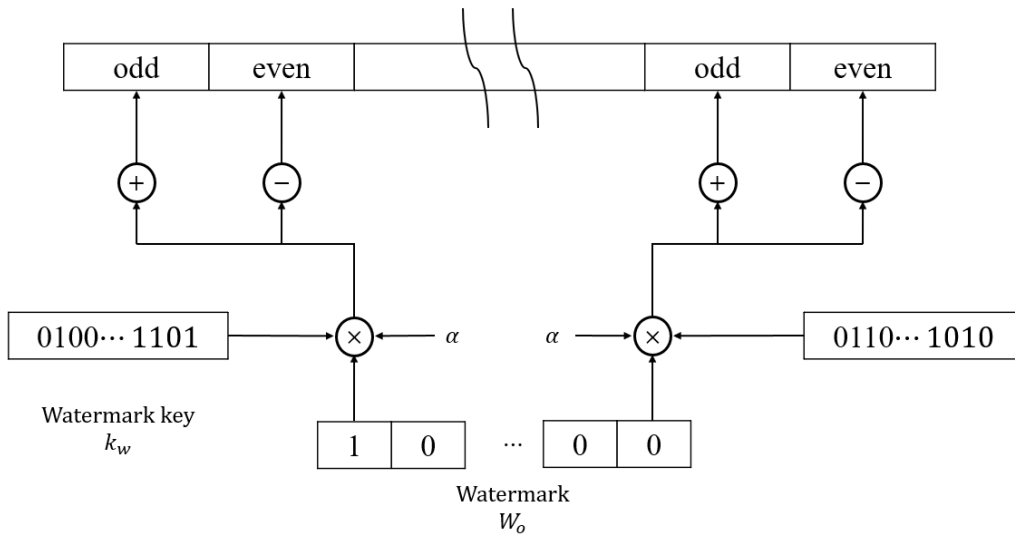


圖 2.2.3-2 交互型展頻浮水印嵌入法示意圖

在介紹完嵌入浮水印的部分，接下來則要討論浮水印萃取部分，最開始我們會接收到音訊訊號 X_w ，我們將以 L_s 點將其分割為 $2N$ 個音框，再將兩個音框視為同一組，並分為奇數音框部分 Y_{odd} 與偶數音框部分 Y_{even} ，如式(2.2.3-4)所示。如果考慮雜訊 n 的影響，我們則將接收到的音訊訊號表示為式(2.2.3-5)。

$$\begin{aligned} X_w &= [x'_{o1}, x'_{e1}, \dots, x'_{oN}, x'_{eN}] \\ \begin{cases} Y_{odd} &= [x'_{o1}, x'_{o2}, \dots, x'_{oN}] \\ Y_{even} &= [x'_{e1}, x'_{e2}, \dots, x'_{eN}] \end{cases} \end{aligned} \quad (2.2.3-4)$$

$$\begin{cases} Y_{odd} = X'_{odd} + n_{odd} \\ Y_{even} = X'_{even} + n_{even} \end{cases} \quad (2.2.3-5)$$

最後我們將式(2.2.3-5)代入式(2.2.1-2)可以分別得到奇數部分與偶數部分的統計決定值式(2.2.3-6)，然而最後整體的統計決定值為式(2.2.3-7)。

$$\begin{aligned} r_{odd} &= \frac{\langle Y_{odd}, k_w \rangle}{\langle k_w, k_w \rangle} = \frac{X_{odd} k_w^T}{\|k_w\|} + W_o + \frac{n_{odd} k_w^T}{\|k_w\|} \\ r_{even} &= \frac{\langle Y_{even}, k_w \rangle}{\langle k_w, k_w \rangle} = \frac{X_{even} k_w^T}{\|k_w\|} - W_o + \frac{n_{even} k_w^T}{\|k_w\|} \end{aligned} \quad (2.2.3-6)$$

$$r = r_{odd} - r_{even} = 2W_o + \frac{(X_{odd} - X_{even})k_w^T}{\|k_w\|} + \frac{(n_{odd} - n_{even})k_w^T}{\|k_w\|} \quad (2.2.3-7)$$

由式(2.2.3-7)可以分別推出平均值與統計決定值的變異數，如式(2.2.3-8)，最後將式(2.2.3-8)代入式(2.2.1-4)可以導出式(2.2.3-9)，其中 ρ 為兩個音框的相關度，而我們可以觀察出如果兩個相連的音框，如果關聯度越高，則錯誤率將會越低。

$$\begin{aligned} E[r] &= m_r = 2W_o \\ \sigma_r^2 &= \frac{2\sigma_n^2 + 2\sigma_x^2(1-\rho)}{L_s \sigma_k^2} \end{aligned} \quad (2.2.3-8)$$

$$p = \frac{1}{2} \operatorname{erfc}\left(\frac{m_r}{\sigma_r \sqrt{2}}\right) = \frac{1}{2} \operatorname{erfc}\left(\sqrt{\frac{L_s \sigma_k^2}{\sigma_x^2(1-\rho) + \sigma_n^2}}\right) \quad (2.2.3-9)$$

2.2.4. 改進交互型浮水印嵌入法

在相繼介紹完加成型浮水印嵌入法與交互型浮水印嵌入法之後，我們要討論的是將兩種浮水印嵌入法皆使用到的方法[12]。首先回顧加成型浮水印嵌入法，我們可以得知，其利用修改嵌入演算法的方法，可以達到降低錯誤的效果。觀察交互型嵌入演算法，我們則可得知其利用兩個相連音框的關聯度，能有效降低原訊號對於錯誤率的影響，所以這章節我們將加入此兩項演算法的優點，產生出更佳的浮水印嵌入法。

首先如交互型展頻浮水印嵌入法一樣，我們先將訊號經由 L_s 點切分為 $2N$ 個音框，

再將兩個音框視為同一組，將其嵌入浮水印，如式(2.2.4-1)所示，再者我們將訊號分成 X_{odd} 、 X_{even} 方便後續的演算法討論，如式(2.2.4-2)。

$$\mathbf{x} = [x_{o1}, x_{e1}, \dots, x_{oN}, x_{eN}] \quad (2.2.4-1)$$

$$\begin{cases} X_{odd} = [x_{o1}, x_{o2}, \dots, x_{oN}] \\ X_{even} = [x_{e1}, x_{e2}, \dots, x_{eN}] \end{cases} \quad (2.2.4-2)$$

運用式(2.2.2-1)改變浮水印嵌入得方法，我們可以得到加成交互型浮水印嵌入法的基本方程式(2.2.4-3)

$$\begin{aligned} s_{odd} &= X_{odd} + (\alpha W_o - \lambda x_o) k_w \\ s_{even} &= X_{even} + (-\alpha W_o - \lambda x_o) k_w \end{aligned} \quad (2.2.4-3)$$

在嵌入完浮水印後，則是要進入浮水印萃取的部分，我們一樣將以 L_s 點將其分割為 $2N$ 個音框，再將兩個音框視為同一組，並分為奇數音框部分 Y_{odd} 與偶數音框部分 Y_{even} ，如式(2.2.4-4)所示。如果考慮雜訊 n 影響，我們接收到的音訊訊號表示為式(2.2.4-5)。

$$\begin{aligned} X_w &= [x'_{o1}, x'_{e1}, \dots, x'_{oN}, x'_{eN}] \\ \begin{cases} Y_{odd} = [x'_{o1}, x'_{o2}, \dots, x'_{oN}] \\ Y_{even} = [x'_{e1}, x'_{e2}, \dots, x'_{eN}] \end{cases} \end{aligned} \quad (2.2.4-4)$$

$$\begin{cases} Y_{odd} = X'_{odd} + n_{odd} \\ Y_{even} = X'_{even} + n_{even} \end{cases} \quad (2.2.4-5)$$

最後我們將式(2.2.4-5)代入式(2.2.1-2)可以分別得到奇數部分與偶數部分的統計決定值式(2.2.4-6)，然而最後整體的統計決定值為式(2.2.4-7)。

$$\begin{aligned} r_{odd} &= \frac{\langle Y_{odd}, k_w \rangle}{\langle k_w, k_w \rangle} = \frac{(1-\lambda)X_{odd} k_w^T}{\|k_w\|} + \alpha W_o + \frac{n_{odd} k_w^T}{\|k_w\|} \\ r_{even} &= \frac{\langle Y_{even}, k_w \rangle}{\langle k_w, k_w \rangle} = \frac{(1-\lambda)X_{even} k_w^T}{\|k_w\|} - \alpha W_o + \frac{n_{even} k_w^T}{\|k_w\|} \end{aligned} \quad (2.2.4-6)$$

$$r = r_{odd} - r_{even} = 2\alpha W_o + \frac{(1-\lambda)(X_{odd} - X_{even})k_w^T}{\|k_w\|} + \frac{(n_{odd} - n_{even})k_w^T}{\|k_w\|} \quad (2.2.4-7)$$

由式(2.2.4-7)可以分別推出平均值與統計決定值的變異數，如式(2.2.4-8)，最後將式(2.2.4-8)代入式(2.2.1-4)可以導出式(2.2.4-9)，其中 ρ 為兩個音框的相關度，觀察式(2.2.3-9)與式(2.2.4-9)我們可以明顯的發現錯誤率降低了。

$$E[r] = m_r = 2\alpha W_o$$

$$\sigma_r^2 = \frac{2\sigma_n^2 + 2\sigma_x^2(1-\lambda)^2(1-\rho)}{L_s\sigma_k^2} \quad (2.2.4-8)$$

$$p = \frac{1}{2}\text{erfc}\left(\frac{m_r}{\sigma_r\sqrt{2}}\right) = \frac{1}{2}\text{erfc}\left(\sqrt{\frac{L_s\sigma_k^2 - \lambda^2\sigma_x^2}{\sigma_x^2(1-\lambda)^2(1-\rho) + \sigma_n^2}}\right) \quad (2.2.4-9)$$

2.3. 容量問題

在 2.2 章節提及了訊號干擾問題，並且討論了不同的相關演算法以降低萃取時的錯誤率，雖然解決了部分問題，但衍生出了容量問題，章節 2.2 中所提的演算法多半是將一個或兩個的音框嵌入一個浮水印位元，這導致浮水印位元嵌入容量受到限制。在 2015 年時，Yong Xiang et al. 提出將一個音框嵌入多個浮水印位元的方法(循環碼法[13])，又在 2018 年時提出 Gram-Schmidt 演算法[14]，解決部分問題，再者徐詠翔提出了互補碼演算法[15]，其對於安全性上得到改善。這些演算法的共通點是建立彼此正交的偽隨機碼序列來對應不相同的浮水印資訊，如圖(2.3-1)所示，本章節將會介紹此些方法。

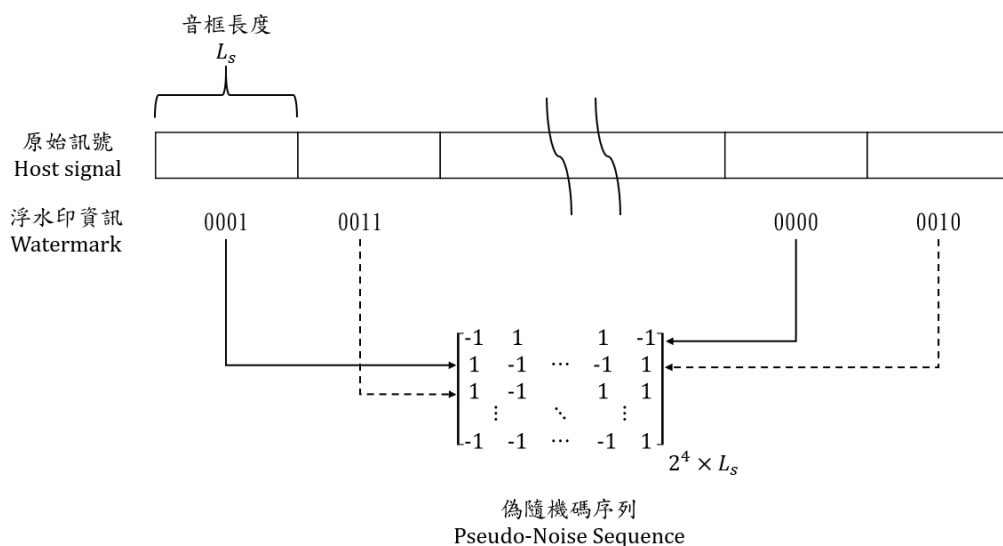


圖 2.3-1 高容量嵌入演算法示意圖

2.3.1. 循環碼法

Yong Xiang et al. 在 2015 年提出在一個音框中嵌入多個浮水印訊號的想法，而為了在萃取浮水印階段，能夠較精確的解出浮水印資訊，在偽隨機碼序列產生階段，必須產生出近似正交的偽隨機碼序列，於是 Yong Xiang et al. 提出了循環碼的浮水印嵌入[13]想法，演算法架構圖如圖(2.3.1-1)所示，率先產生一組長度為 L_s 且其中數字由-1 與 1 平均分佈的偽隨機碼序列 p_1 ，接著再利用如式(2.3.1-1)的循環碼產生出 N_p 個偽隨機碼序列，也就是因為使用循環碼的產生方式，當 L_s 長度越長偽隨機碼序列將越趨於正交。

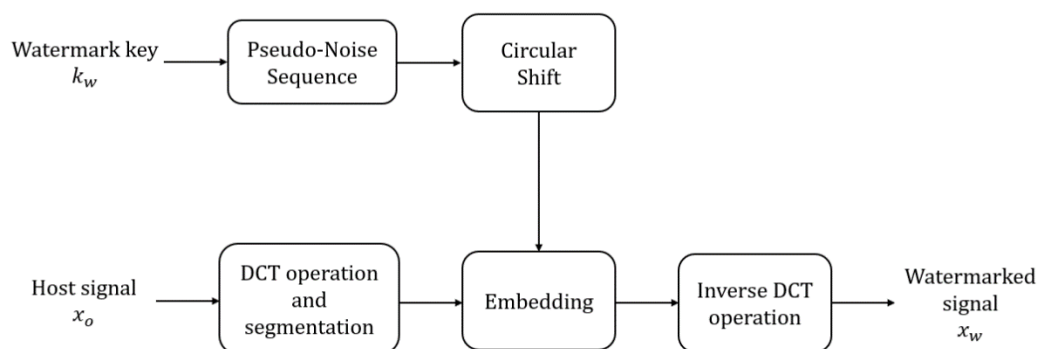


圖 2.3.1-1 高容量循環碼展頻浮水印演算法

循環碼展頻浮水印增加容量的方法，是在一個音框中嵌入多個浮水印訊號，再以每個音框嵌入的浮水印序號對應到需嵌入的偽隨機碼序列，若一段浮水印的位元數為 N_p ，也代表了為了表達每一種嵌入浮水印的情況，必須產生出 2^{N_p} 個偽隨機碼序列，才能完全涵蓋所有可能，舉例如果嵌入的浮水印位元數為 2，則嵌入的浮水印位元有 $\{00\}$ 、 $\{01\}$ 、 $\{10\}$ 、 $\{11\}$ ，我們必須產生出 $2^2 = 4$ 個偽隨機碼序列，其分別為 p_1 、 p_2 、 p_3 、 p_4 ，最後視嵌入浮水印位元選擇相對印的偽隨機碼序列。

$$P = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{N_p} \end{bmatrix} = \begin{bmatrix} \overline{p_1} & \overline{p_2} & \cdots & \overline{p_{L_s-1}} & \overline{p_{L_s}} \\ \overline{p_{L_s}} & \overline{p_1} & & \overline{p_{L_s-2}} & \overline{p_{L_s-1}} \\ & \vdots & \ddots & & \vdots \\ \overline{p_{L_s-N_p+2}} & \overline{p_{L_s-N_p+3}} & \cdots & \overline{p_{L_s-N_p}} & \overline{p_{L_s-N_p+1}} \end{bmatrix} \quad (2.3.1-1)$$

在介紹完偽隨機碼序列的選擇後，接著是浮水印嵌入的部分，首先，我們將原訊號 x_o 經過離散餘弦轉換(Discrete Cosine Transform)，再以 $2L_s$ 將訊號分割成 N 個音框，並且將分割好的離散餘弦轉換係數 $X_i(k)$ 分成奇數部分與偶數部分，如式(2.3.1-2)所示，其中下標 o 表示奇數(odd)，下標 e 表示偶數(even)， i 則代表第幾個音框。

$$\begin{aligned} X_i(k) &= [X_i(0), X_i(1), \dots, X_i(2L_s - 1)] \\ \begin{cases} x_{i,e} = [X_i(0), X_i(2), \dots, X_i(2L_s - 2)] \\ x_{i,o} = [X_i(1), X_i(3), \dots, X_i(2L_s - 1)] \end{cases} \end{aligned} \quad (2.3.1-2)$$

若以嵌入第 i 個音框作為全部音框的表示，首先是將每個音框所需的 k 個浮水印位元轉換成對應到的隨機碼序列 p_t ，緊接著運用式(2.3.1-3)的方式將浮水印資訊嵌入於音框之中，式中的「 $\circ$ 」為阿達瑪乘法(Hadamard Product)。然而在嵌入完浮水印訊號後，我們須將奇數序列($\tilde{x}_{i,e}$)與偶數序列($\tilde{x}_{i,o}$)結合成嵌入後的浮水印序列($\tilde{X}_i$)，如式(2.3.1-4)所式，而最後可以完成整體的浮水印資訊嵌入，此嵌入法的式意圖如圖(2.3.1-2)所式。

$$\begin{aligned} \tilde{x}_{i,e} &= (1 + \beta p_t) \circ x_{i,e} = [\tilde{X}_{i,e}(0), \tilde{X}_{i,e}(1), \dots, \tilde{X}_{i,e}(L_s - 1)] \\ \tilde{x}_{i,o} &= (1 + \beta p_t) \circ x_{i,o} = [\tilde{X}_{i,o}(0), \tilde{X}_{i,o}(1), \dots, \tilde{X}_{i,o}(L_s - 1)] \end{aligned} \quad (2.3.1-3)$$

$$\tilde{X}_i = [\tilde{X}_{i,e}(0), \tilde{X}_{i,o}(0), \tilde{X}_{i,e}(1), \tilde{X}_{i,o}(1), \dots, \tilde{X}_{i,e}(L_s - 1), \tilde{X}_{i,o}(L_s - 1)] \quad (2.3.1-4)$$

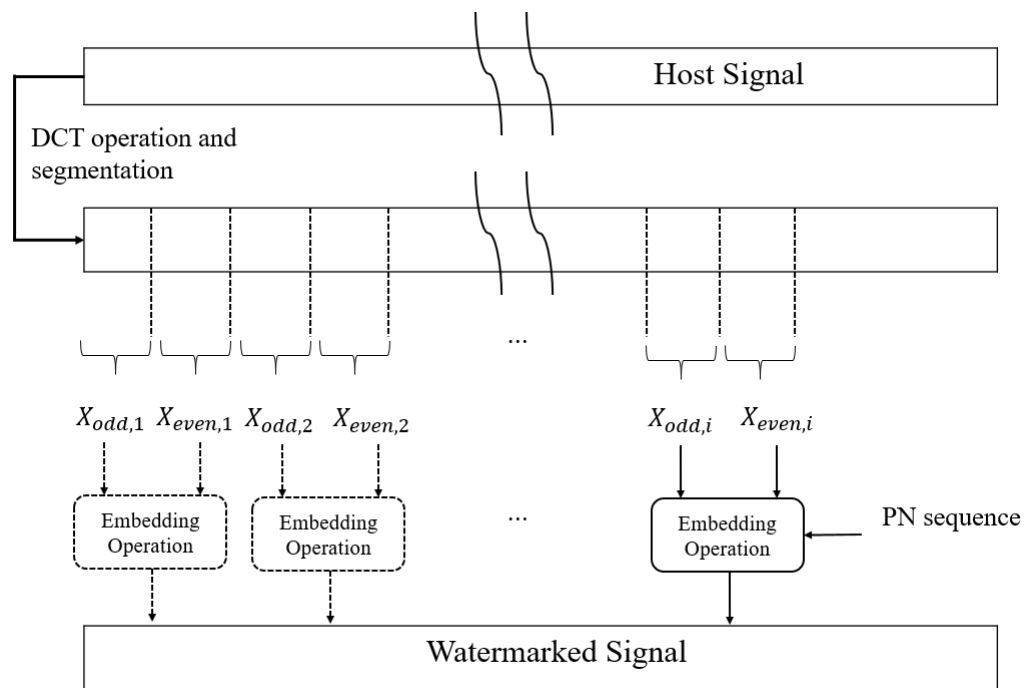


圖 2.3.1-2 高容量循環碼展頻浮水印演算法浮水印嵌入示意圖



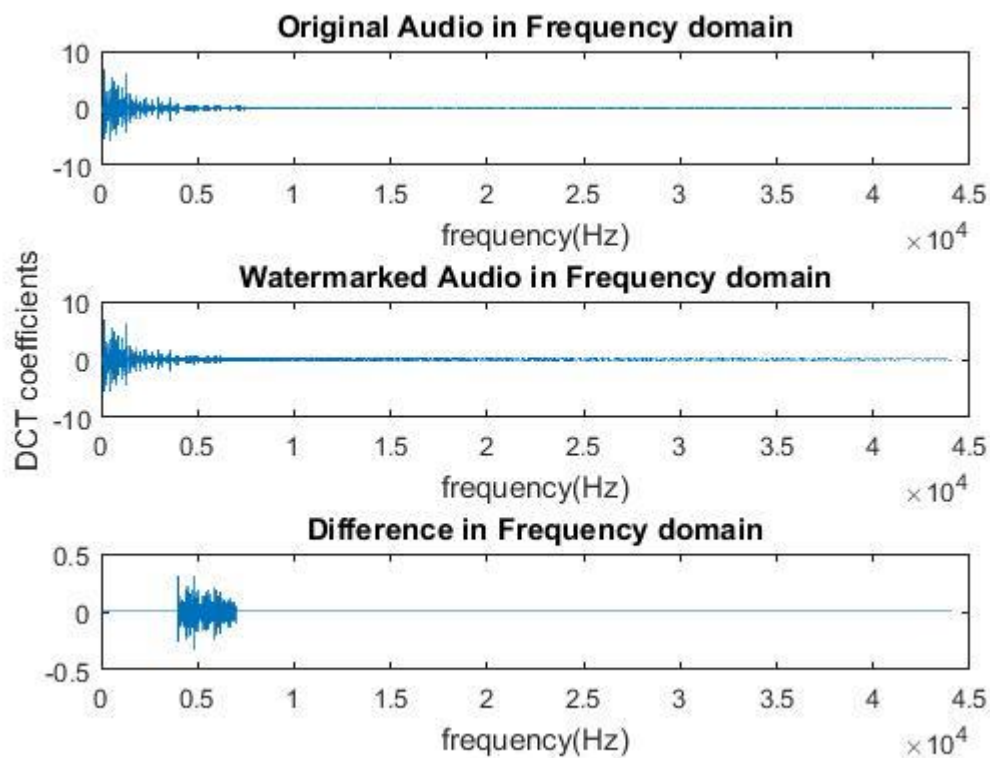


圖 2.3.1-3 循環碼嵌入法在餘弦頻域之嵌入前後比較圖

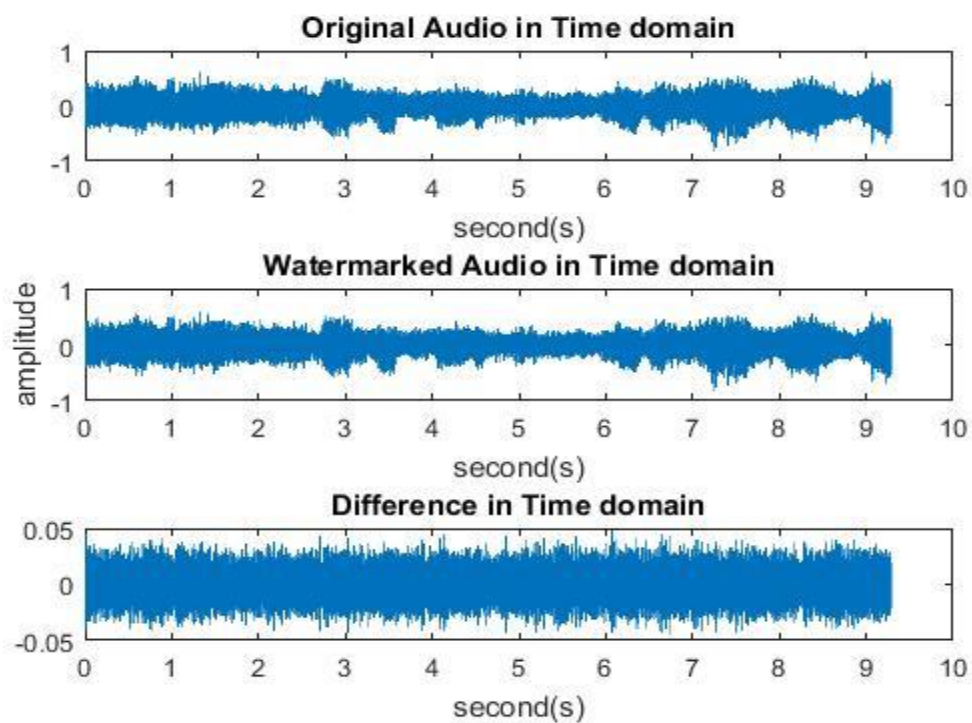


圖 2.3.1-4 循環碼嵌入法在時域之嵌入前後比較圖

在做完浮水印嵌入之後，接下來是萃取的部分，首先我們將嵌入完浮水印的音檔經由離散餘弦轉換(discrete cosine transform)將其轉為離散餘弦係數，並且以 $2L_s$ 點將訊號切割為 N 個音框，最後將分割好的離散餘弦轉換係數分成奇數部分 $y_{i,o}$ 與偶數部分 $y_{i,e}$ (與嵌入時的方法相同)，其中如式(2.3.1-5)所示，可以觀察出由於不考慮雜訊的影響，所以方程式與嵌入方程式一致。

$$\begin{aligned} y_{i,e} &= \tilde{x}_{i,e} = (1 + \beta p_t) \circ x_{i,e} \\ y_{i,o} &= \tilde{x}_{i,o} = (1 - \beta p_t) \circ x_{i,o} \end{aligned} \quad (2.3.1-5)$$

由於在嵌入浮水印資訊時，是採取對應的方式，故在解出時使用所有的偽隨機碼序列一一與兩訊號的差值作乘積，最後能對應出每個乘積值，找尋其中乘積值最大的序列，也就代表與偽隨機碼序列愈相關，而使用此序列對應的索引數字 m ，可以換算出浮水印嵌入之數值，其中兩訊號的差值如式(2.3.1-6)所示，差值與偽隨機序列的乘積如式(2.3.1-7)所示。

$$y_{i,d} = |y_{i,e}| - |y_{i,o}| = |x_{i,e} - x_{i,o}| + |\beta p_t \circ (x_{i,e} + x_{i,o})| \quad (2.3.1-6)$$

$$m = \arg \max_{j \in \{1,2,\dots,N_p\}} y_{i,d} p_j^T \quad (2.3.1-7)$$

循環碼嵌入演算法使用近似正交的偽隨機序列嵌入浮水印資訊，這點產生出了問題，當訊號雜訊比(Signal to Noise Ratio)值降低，也就是雜訊相對升高時，會產生萃取的正確率下降的情形，根據此演算法的數據指出，當 SNR 值降至 5dB 時，會使浮水印資訊解出的正確性大幅下降。然而雖然此演算法雖然對於容量的嵌入的部分大幅上升，但在浮水印萃取率上差強人意，而在下一節中將會介紹另一種偽隨機碼序列更為正交的演算法改善此問題。

2.3.2. Gram-Schmidt 正交法

前一節所提到的循環碼嵌入法，由於使用接近正交的偽隨機序列嵌入浮水印資訊，故導致在雜訊比較低(5dB)時，對浮水印萃取產生了負面影響，導致浮水印萃取率大幅下降，也因如此 Yong Xiang et al. 在 2018 年提出了一個改進的演算法[13]，此演算法使用 Gram-Schmidt 正交法[16]產生出正交的偽隨機碼序列，對於雜訊相對高的語音訊號，可以在萃取率上得到提升，演算法的架構如圖 2.3.2-1 所示。

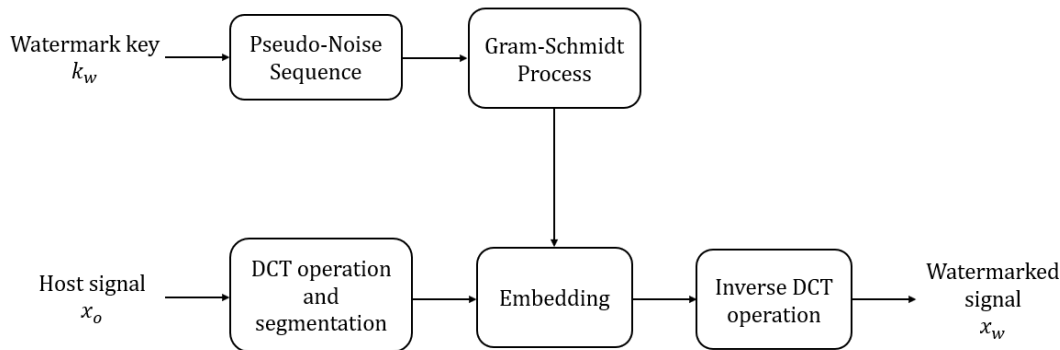


圖 2.3.2-1 高容量 Gram-Schmidt 展頻浮水印演算法架構圖

在演算法嵌入之前必須選擇適當的嵌入區域，以及嵌入的音框大小(L_s)，然而此音框的大小也就是偽隨機碼序列之長度。Gram-Schmidt 展頻浮水印演算法嵌入浮水印的方式與 2.3.1 節的循環碼浮水印演算法類似，在決定完一個音框中的浮水印資訊嵌入位元(k)以後，會以位元對應到相對的偽隨機碼序列，再將選擇完後的偽隨機碼與音框作浮水印的嵌入，舉例來說若浮水印的嵌入位元數為 $k = 2$ ，而嵌入的資訊分別有 $\{0, 0\}$ 、 $\{0, 1\}$ 、 $\{1, 0\}$ 、 $\{1, 1\}$ 種可能，其中分別對應到 p_1 、 p_2 、 p_3 、 p_4 ，最後視嵌入浮水印位元選擇相對應的偽隨機碼序列，而這個例子也間接證明若嵌入的浮水印資訊位元數為 k ，就必須產生 $N_p = 2^k$ 個偽隨機碼序列。

在了解完嵌入的過程後，我們將要討論偽隨機序列的產生，在前文提及了選擇音框大小的長度，也代表選擇了偽隨機碼的長度，於是我們先產生出一個長度為 L_s 的偽隨機碼(p_0)，其中碼的數字由均勻分布的+1 與-1 組成，如式(2.3.2-1)所示，接著產生出一個 L_s 階的單位矩陣 P_0 ，並且使用偽隨機碼 p_0 取代 P_0 的第一欄(Column)，如式(2.3.2-2)所示。

$$p_0 \in \{+1, -1\}^{L_s} \quad (2.3.2-1)$$

$$P_0(:, 1) = p_0^T \quad (2.3.2-2)$$

再取代完 P_0 的第一欄後，使用 Gram-Schmidt 正交化法[16]依序產生 N_p 個偽隨機碼序列 $p_1, p_2, \dots, p_{N_p}$ ，產生的方法如式(2.3.2-3)所示，式中 $\langle x, y \rangle$ 為 x 與 y 的內積 (Inner Product)， $\|x\|$ 為 x 的範數(norm)。

$$\begin{aligned} p'_1 &= P_0(1,:), p_1 = \frac{p'_1}{\|p'_1\|} \\ p'_2 &= P_0(2,:) - \langle P_0(2,:), p_1 \rangle p_1, p_2 = \frac{p'_2}{\|p'_2\|} \\ &\vdots \\ p'_{N_p} &= P_0(N_p,:) - \langle P_0(N_p,:), p_{N_p-1} \rangle p_{N_p-1} \\ &\quad - \langle P_0(N_p,:), p_{N_p-2} \rangle p_{N_p-2} - \dots - \langle P_0(N_p,:), p_1 \rangle p_1, p_{N_p} = \frac{p'_{N_p}}{\|p'_{N_p}\|} \end{aligned} \quad (2.3.2-3)$$

嵌入浮水印資訊時，先將原訊號 x_o 經由離散餘弦轉換(Discrete Cosine Transform)取得離散餘弦轉換係數 $x(k)$ ，接著就如前面所說的將離散餘弦轉換係數 $x(k)$ ，以長度 L_s 切割成 N 個音框，在嵌入第 i 個音框時，會以要嵌入音框的 k 個浮水位元對應到相對的偽隨機碼序列 p_t ，接著再以式(2.3.2-4)嵌入浮水印資訊。然而嵌入完的離散餘弦係數 X_i ，必須再經由反離散餘弦轉換(Inverse Discrete Cosine Transform)將其轉回時域之嵌入浮水印資訊訊號 x_w ，嵌入流程如圖(2.3.2-2)所示。

$$X_i = x_i + \alpha_i \left(\frac{x_i p_t^T}{|x_i p_t^T|} \right) p_t \quad (2.3.2-4)$$

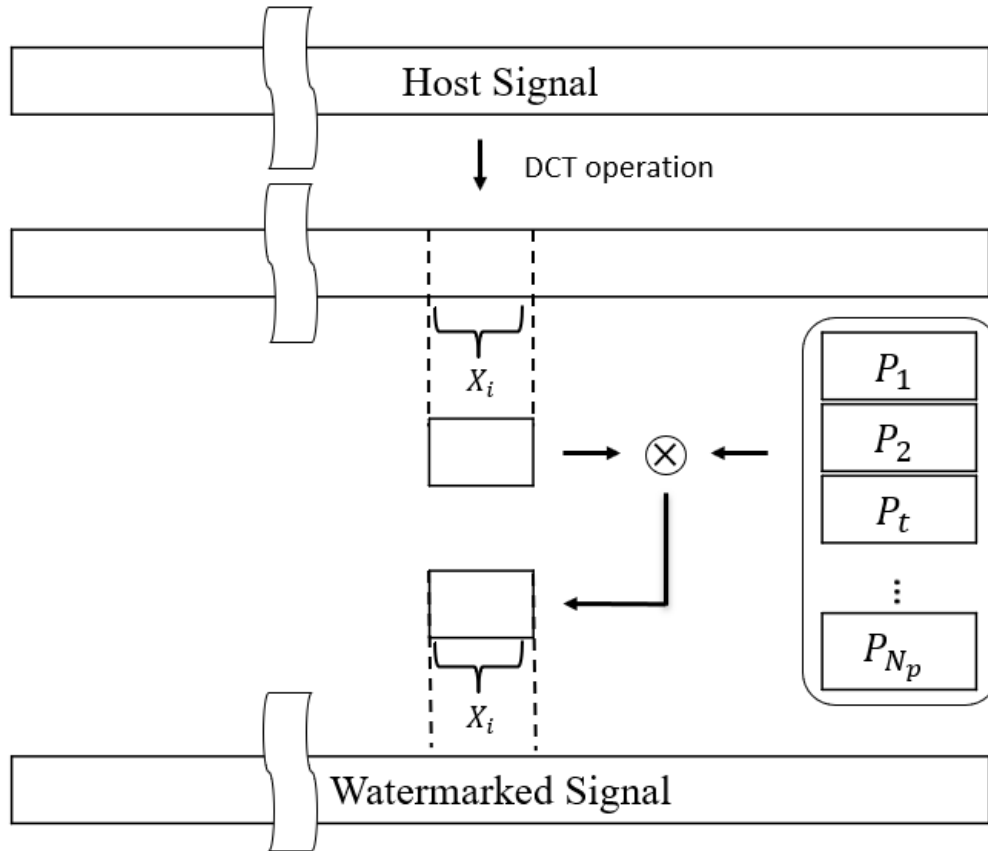


圖 2.3.2-2 高容量 Gram-Schmidt 展頻浮水印演算法嵌入示意圖

在做完浮水印嵌入之後，接下來是萃取的部分，我們將嵌入完浮水印的音檔經由離散餘弦轉換(discrete cosine transform)將其轉為離散餘弦係數，並且以 L_s 點將訊號切割為 N 個音框，接著所有的偽隨機碼序列一一與切割完之訊號($D_{i,j}$)作乘積，如式(2.3.2-5)，找尋其中乘積值最大的序列，也就代表與偽隨機碼序列愈相關，而使用此序列對應的索引數字 m ，可以換算出浮水印嵌入之數值，浮水印資訊萃取示意圖如圖(2.3.2-3)所示。

$$D_{i,j} = |y_i p_j^T|$$

$$t = \arg \max_{j \in \{1,2,\dots,N_p\}} D_{i,j} = \arg \max_{j \in \{1,2,\dots,N_p\}} |y_i' p_j^T|$$

(2.3.2-5)

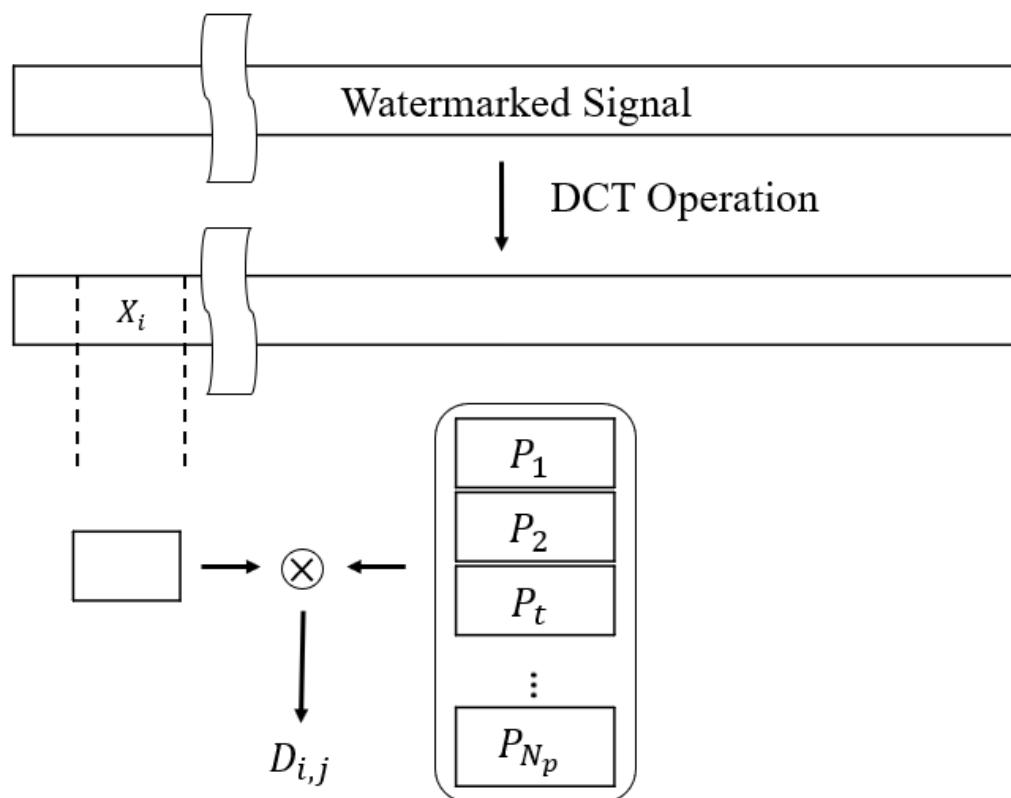


圖 2.3.2-3 高容量 Gram-Schmidt 展頻浮水印演算法萃取示意圖

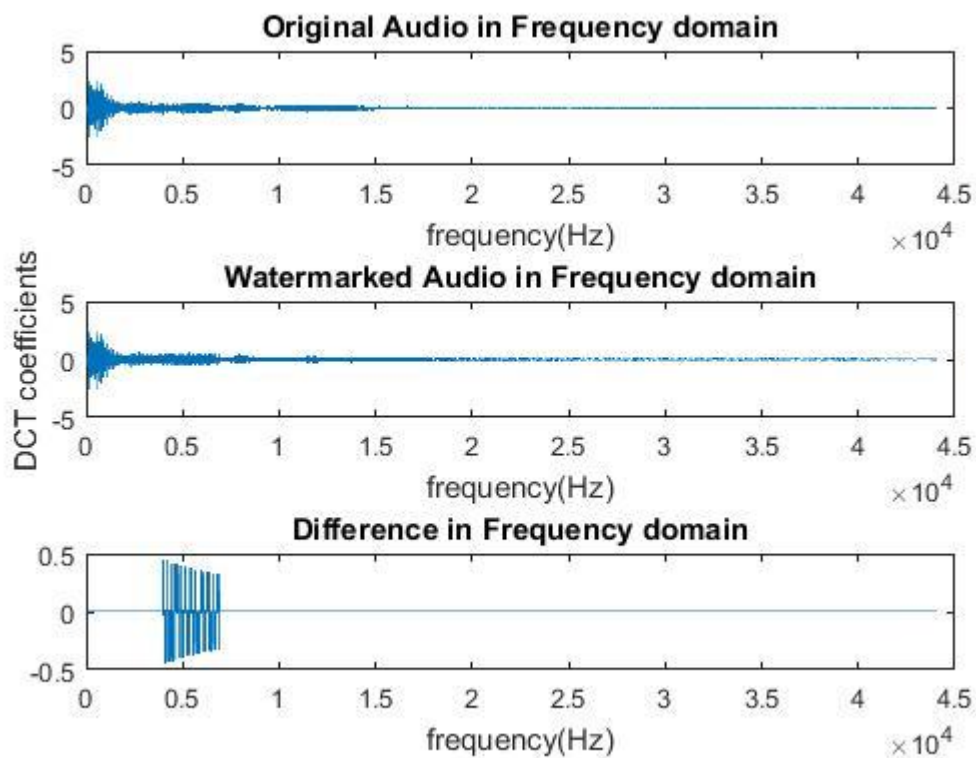


圖 2.3.2-4 Gram-Schmidt 展頻浮水印演算法在餘弦頻域之嵌入前後比較圖

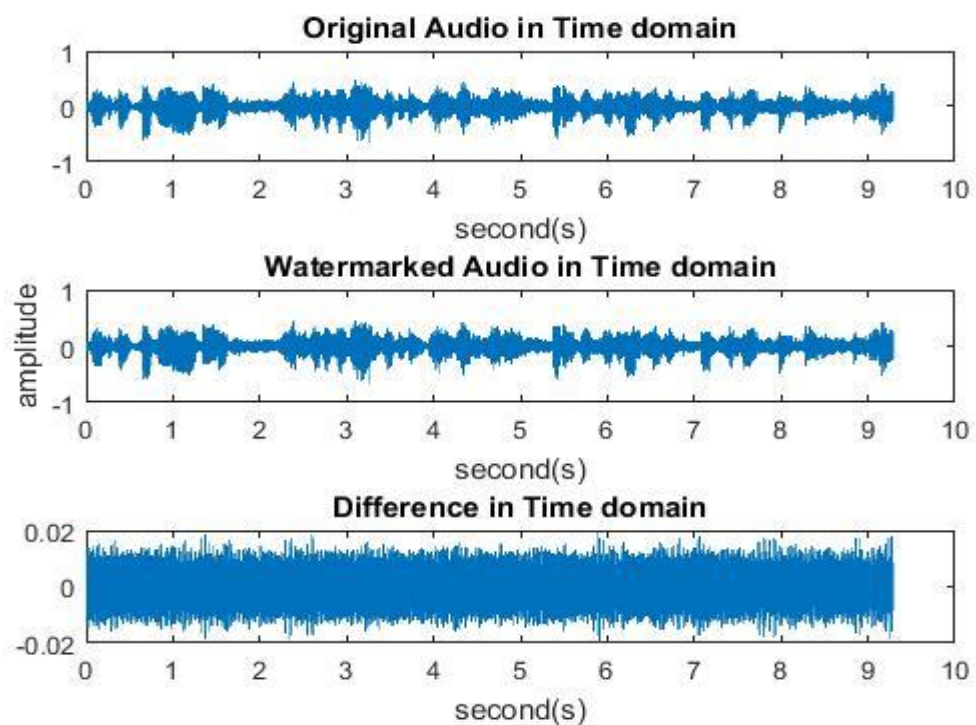


圖 2.3.2-5 Gram-Schmidt 展頻浮水印演算法在時域之嵌入前後比較圖

Gram-Schmidt 正交展頻浮水印演算法看似解決了浮水印在高雜訊的環境下解出率下降的問題，但伴隨而來的問題則是在於安全性上的問題，在創造偽隨機序列時，因為要產生出正交的偽隨機序列，故使用了 Gram-Schmidt 產生之，我們可以由下圖(2.3.2-6)與圖(2.3.2-7)觀察到，兩組不同的偽隨機序列，由不同的 p_0 產生出不同的 $p_1, p_2, \dots, p_{N_p}$ ，雖然看似是不同的序列數值，但他們有相同的趨勢，也就是說這兩者的形式皆類似單位矩陣，這樣就存在安全性的問題，故緊接著下一節要介紹的方法即是使用完全互補碼來取代 Gram-Schmidt 正交法所產生之序列，以提高安全性，並且在浮水印資訊萃取上保留原有的萃取率。

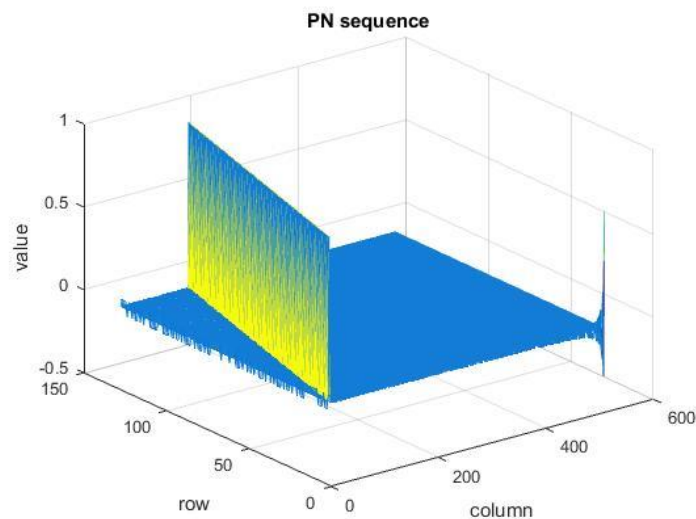


圖 2.3.2-6 Gram-Schmidt 正交法所產生之序列一

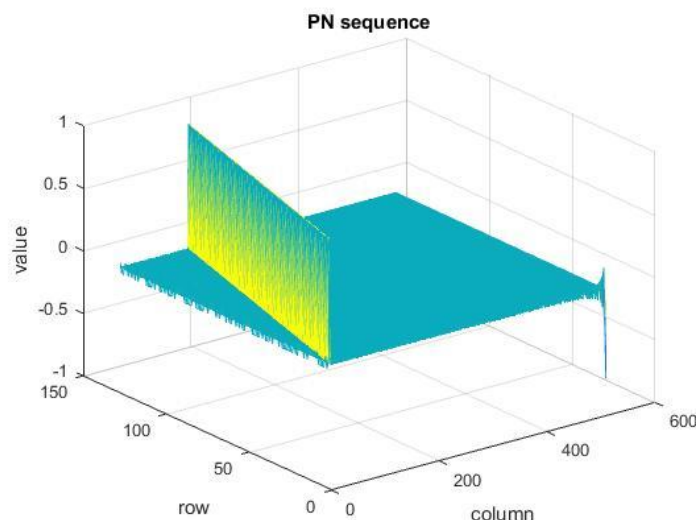


圖 2.3.2-7 Gram-Schmidt 正交法所產生之序列二

2.3.3. 互補碼法

在前一小節提及的 Gram-Schmidt 正交浮水印嵌入法，存在了安全性上的問題，因為偽隨機序列的相似性，導致在解出方面存在安全性的疑慮，故徐詠翔在[17]提出了一種使用互補碼水印的方法，這種演算法使用互補碼取代 Gram-Schmidt 法產生正交序列，互補碼序列的優點是其擁有完美的正交性值，並且產生出的多個序列並不相似單位矩陣，這也就大大的提升了嵌入法的安全性，並且其完美的正交性值又可以保留原本的萃取率，演算法的架構如圖 2.3.3-1 所示。

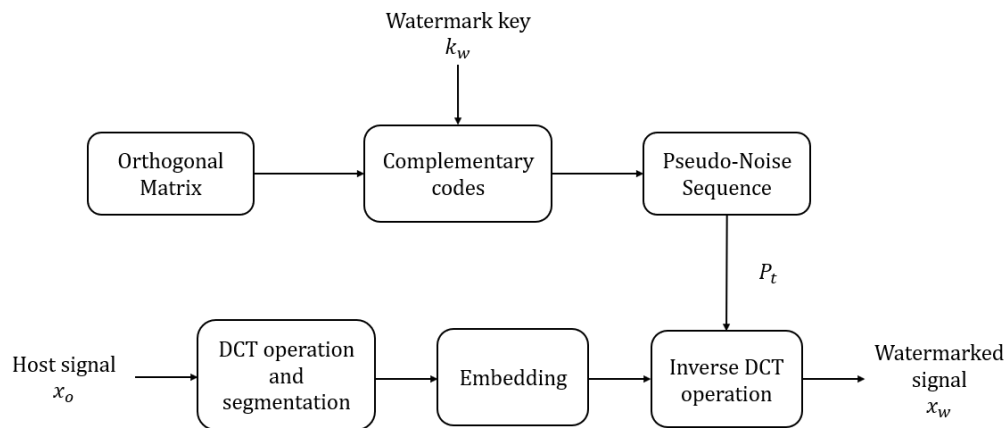


圖 2.3.3-1 高容量互補碼展頻浮水印演算法

此演算法的嵌入方式類似章節 2.3.1 與章節 2.3.2，簡單說也就是先決定在一個音框之中嵌入位元數 k ，再使用這 k 個位元對應到相對應的偽隨機序列，也就是說必須產生出 $N_p = 2^k$ 個偽隨機碼序列，方能使所有浮水印位元組合皆嵌入於音檔之中，舉例來說如果嵌入的浮水印位元數為 2，則嵌入的浮水印位元有 {00}、{01}、{10}、{11}，我們必須產生出 $2^2 = 4$ 個偽隨機碼序列，其分別為 p_1 、 p_2 、 p_3 、 p_4 ，最後視嵌入浮水印位元選擇相對印的偽隨機碼序列。

接下來是互補碼[18][19]產生的部分，假設一個維度為 $N \times N$ 的正交矩陣 A ，並且矩陣 A 中的每一個矩陣元素為複數且範數(norm)為 1。

$$A = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_N \end{bmatrix}, \quad |a_{ij}| = 1, \text{ for } i, j = 1, 2, \dots, N \quad (2.3.3-1)$$

再假設有另一個同樣維度為 $N \times N$ 的正交矩陣 B ，並且矩陣 B 中的每一個矩陣元素

為複數且範數(norm)為 1。

$$B = \begin{bmatrix} B_1 \\ B_2 \\ \vdots \\ B_N \end{bmatrix}, \quad |b_{ij}| = 1, \text{ for } i, j = 1, 2, \dots, N \quad (2.3.3-2)$$

則我們可以利用矩陣A與以及矩陣B，產生出一組N個長度為 N^2 的序列 $C_1, C_2, \dots, C_N$ ，如式(2.3.3-3)所示

$$\begin{aligned} C_1 &= (b_{11}A_1, b_{12}A_2, \dots, b_{1N}A_N) = (c_{11}, c_{12}, \dots, c_{1N^2}) \\ C_2 &= (b_{21}A_1, b_{22}A_2, \dots, b_{2N}A_N) = (c_{21}, c_{22}, \dots, c_{2N^2}) \\ &\vdots \\ C_N &= (b_{N1}A_1, b_{N2}A_2, \dots, b_{NN}A_N) = (c_{N1}, c_{N2}, \dots, c_{NN^2}) \end{aligned} \quad (2.3.3-3)$$

在產生完序列後，我們再假設矩陣D為一個維度為 $N \times N$ 的正交矩陣，並且矩陣D中的每一個矩陣元素為複數且範數(norm)為 1。

$$D = \begin{bmatrix} D_1 \\ D_2 \\ \vdots \\ D_N \end{bmatrix}, \quad |d_{ij}| = 1, \text{ for } i, j = 1, 2, \dots, N \quad (2.3.3-4)$$

於是我們可以利用序列 $C_1, C_2, \dots, C_N$ 與正交矩陣D產生出 N^2 個長度為 N^2 的序列，其中 $i, j = 1, 2, \dots, N$ 。

$$\begin{aligned} &E_{ij} \\ &= [c_{i1}d_{j1}, c_{i2}d_{j2}, \dots, c_{iN}d_{jN}, c_{i(N+1)}d_{j1}, \dots, c_{i(2N)}d_{jN}, \dots, c_{i(N^2-N+1)}d_{j1}, \dots, c_{iN^2}d_{jN}] \\ &= [e_{ij1}, e_{ij2}, \dots, e_{ijN^2}] \text{ for } i, j = 1, 2, \dots, N \end{aligned} \quad (2.3.3-5)$$

我們可以利用(2.3.3-5)所產生的 N^2 的序列組成N個互補碼，如式(2.3.3-6)所示，其中 $E_1, E_2, \dots, E_N$ 各為一組互補碼，每個互補碼中有N個序列(或稱為子碼)，其中N的大小是由產生互補碼的矩陣維度決定的。由於每個子碼的長度為 N^2 ，並且一組互補碼有N個子碼，所以完全互補碼的長度為 $N \times N^2 = N^3$ 。

$$\begin{aligned} E_1 &= [E_{11}, E_{12}, \dots, E_{1N}] \\ E_2 &= [E_{21}, E_{22}, \dots, E_{2N}] \end{aligned}$$

$$\vdots$$

$$E_N = [E_{N1}, E_{N2}, \dots, E_{NN}]$$

$$(2.3.3-6)$$

由於此演算法所使用的偽隨機序列為超級互補碼[20]，在產生完完全互補碼後將會先介紹擴展完全互補碼[19]，最後才會以擴展完全互補碼得到超級互補碼。由前述的討論，我們得到並且發現完全互補碼的長度與完全互補碼的子碼個數存在一個三次方的關係，而由於產生出的互補碼長度對於浮水印的資訊嵌入仍不足夠，因此將使用擴展完全互補碼改變此種三次方關係，讓完全互補碼與子碼的關係可以是三次方、四次方…等等得以一直無限的延伸下去。這個方法能使每次擴展後子碼長度變為原本的 N 倍，卻不改變子碼的個數，因為擴展完全互補碼是完全互補碼利用一種擴展方式得出的，也就是如此，若我們重複使用這種方法 n 次，我們可以將原本子碼長度擴展為原來的 N^n 倍，則我們可以得到完全互補碼與擴展完全互補碼存在 $(3+n)$ 的次方關係。擴展的方次為假設原本子碼長度為 N^{r-1} ，一個完全互補碼中有 N 個子碼，則利用相互插入與排列的方法產生出一個序列，如式(2.3.3-7)所示。

$$F_i = [e_{i11}, e_{i21}, \dots, e_{iN1}, e_{i12}, e_{i22}, \dots, e_{iN2}, \dots, e_{i1N^{r-1}}, e_{i2N^{r-1}}, \dots, e_{iNN^{r-1}}]$$

$$= [f_{i1}, f_{i2}, \dots, f_{iN^r}] \text{ for } i = 1, 2, \dots, N$$

$$(2.3.3-7)$$

接著可以將產生出的 N 個序列 $F_1, F_2, \dots, F_N$ 取代原先(2.3.3-3)式所產生出的序列 $C_1, C_2, \dots, C_N$ ，接著再利用序列 $C_1, C_2, \dots, C_N$ 與正交矩陣 D 重做一次(2.3.3-5)的運算，就可以產生出擴展 N 倍的擴展完全互補碼，若重複使用此方法便可一直將子碼擴展下去。

介紹完擴展完全互補碼之後，由於演算法的偽隨機碼數還是不足，故接下來會介紹產生超級互補碼的演算法，提升完全互補碼的個數。

超級完全互補碼的演算法在產生互補碼的方面，首先我們要產生出與前面所提及的同維度正交矩陣 A 、 B 、 D ，接著再利用這三個矩陣經過(2.3.3-3)、(2.3.3-5)產生出 E_{ij} ，接著將 E_{ij} 做為超級互補碼的基本碼，如(2.3.3-8)所示。最後我們可以利用式(2.3.3-8)所產生的互補碼經由式(2.3.3-9)產生超級互補碼。其中 T 為 T 的互補， L 則為原先擴展前碼的數目。產生完之超級互補碼，將會使用於此演算法中做為嵌入浮水印資訊的偽隨機碼序列。

$$\begin{aligned}
E_{11} &= T_1 = [T_{11}, T_{12}, \dots, T_{1N^2}] \\
E_{12} &= T_2 = [T_{21}, T_{22}, \dots, T_{2N^2}] \\
E_{13} &= T_3 = [T_{31}, T_{32}, \dots, T_{3N^2}] \\
E_{14} &= T_4 = [T_{41}, T_{42}, \dots, T_{4N^2}] \\
&\vdots \\
E_{1N} &= T_N = [T_{N1}, T_{N2}, \dots, T_{NN^2}] \\
&\vdots \\
E_{NN} &= T_{N^2} = [T_{N^2 1}, T_{N^2 2}, \dots, T_{N^2 N^2}]
\end{aligned}$$

(2.3.3-8)

$$\begin{aligned}
S_1 &= [T_{11}, T_{21}, T_{12}, T_{22}, \dots, T_{1l}, T_{2l}] = [S_{11}, S_{12}, \dots, S_{1(2l)}] \\
S_2 &= [T_{11}, \overline{T_{21}}, T_{12}, \overline{T_{22}}, \dots, T_{1l}, \overline{T_{2l}}] = [S_{21}, S_{22}, \dots, S_{2(2l)}] \\
S_3 &= [T_{21}, T_{11}, T_{22}, T_{12}, \dots, T_{2l}, T_{1l}] = [S_{31}, S_{32}, \dots, S_{3(2l)}] \\
S_4 &= [T_{21}, \overline{T_{11}}, T_{22}, \overline{T_{12}}, \dots, T_{2l}, \overline{T_{1l}}] = [S_{41}, S_{42}, \dots, S_{4(2l)}] \\
&\vdots
\end{aligned}$$

$$\begin{aligned}
S_{4j+1} &= [T_{(2j+1)1}, T_{(2j+2)1}, T_{(2j+1)2}, T_{(2j+2)2}, \dots, T_{(2j+1)l}, T_{(2j+2)l}] \\
&= [S_{(4j+1)1}, S_{(4j+1)2}, \dots, S_{(4j+1)(2l)}] \\
S_{4j+2} &= [T_{(2j+1)1}, \overline{T_{(2j+1)1}}, T_{(2j+1)2}, \overline{T_{(2j+1)2}}, \dots, T_{(2j+1)l}, \overline{T_{(2j+1)l}}] \\
&= [S_{(4j+2)1}, S_{(4j+2)2}, \dots, S_{(4j+2)(2l)}] \\
S_{4j+3} &= [T_{(2j+2)1}, T_{(2j+1)1}, T_{(2j+2)2}, T_{(2j+1)2}, \dots, T_{(2j+2)l}, T_{(2j+1)l}] \\
&= [S_{(4j+3)1}, S_{(4j+3)2}, \dots, S_{(4j+3)(2l)}] \\
S_{4j+4} &= [T_{(2j+2)1}, \overline{T_{(2j+1)1}}, T_{(2j+2)2}, \overline{T_{(2j+1)2}}, \dots, T_{(2j+2)l}, \overline{T_{(2j+1)l}}] \\
&= [S_{(4j+4)1}, S_{(4j+4)2}, \dots, S_{(4j+4)(2l)}] \\
&\vdots
\end{aligned}$$

$$\begin{aligned}
S_{2l-3} &= [T_{(l-1)1}, T_{l1}, T_{(l-1)2}, T_{l2}, \dots, T_{(l-1)l}, T_{ll}] = [S_{(2l-3)1}, S_{(2l-3)2}, \dots, S_{(2l-3)(2l)}] \\
S_{2l-2} &= [T_{(l-1)1}, \overline{T_{l1}}, T_{(l-1)2}, \overline{T_{l2}}, \dots, T_{(l-1)l}, \overline{T_{ll}}] = [S_{(2l-2)1}, S_{(2l-2)2}, \dots, S_{(2l-2)(2l)}] \\
S_{2l-1} &= [T_{l1}, T_{(l-1)1}, T_{l2}, T_{(l-1)2}, \dots, T_{ll}, T_{(l-1)l}] = [S_{(2l-1)1}, S_{(2l-1)2}, \dots, S_{(2l-1)(2l)}] \\
S_{2l} &= [T_{l1}, \overline{T_{(l-1)1}}, T_{l2}, \overline{T_{(l-1)2}}, \dots, T_{ll}, \overline{T_{(l-1)l}}] = [S_{(2l)1}, S_{(2l)2}, \dots, S_{(2l)(2l)}]
\end{aligned}$$

(2.3.3-9)

在嵌入浮水印資訊時，我們必須先將原訊號 x_o 經由離散餘弦轉換(Discrete Cosine Transform)取得離散餘弦轉換係數 $x(k)$ ，接著將離散餘弦轉換係數 $x(k)$ ，以長度 L_s 切割成 N 個音框，假設在嵌入第 i 個音框時，先以嵌入音框的 k 個浮水印位元，對應相對的偽隨機碼序列 p_t ，接著再以(2.3.3-10)的方程式嵌入浮水印資訊。然而嵌入完的離散餘弦係數 X_i ，必須再經由反離散餘弦轉換(Inverse Discrete Cosine Transform)將其轉回時域之嵌入浮水印資訊訊號 x_w ，嵌入流程如圖(2.3.3-2)所示，其嵌入流程與上一章節的 Gram-Schmidt 正交展頻浮水印演算法相同。

$$X_i = x_i + \alpha_i \left(\frac{x_i p_t^T}{|x_i p_t^T|} \right) p_t \quad (2.3.3-10)$$

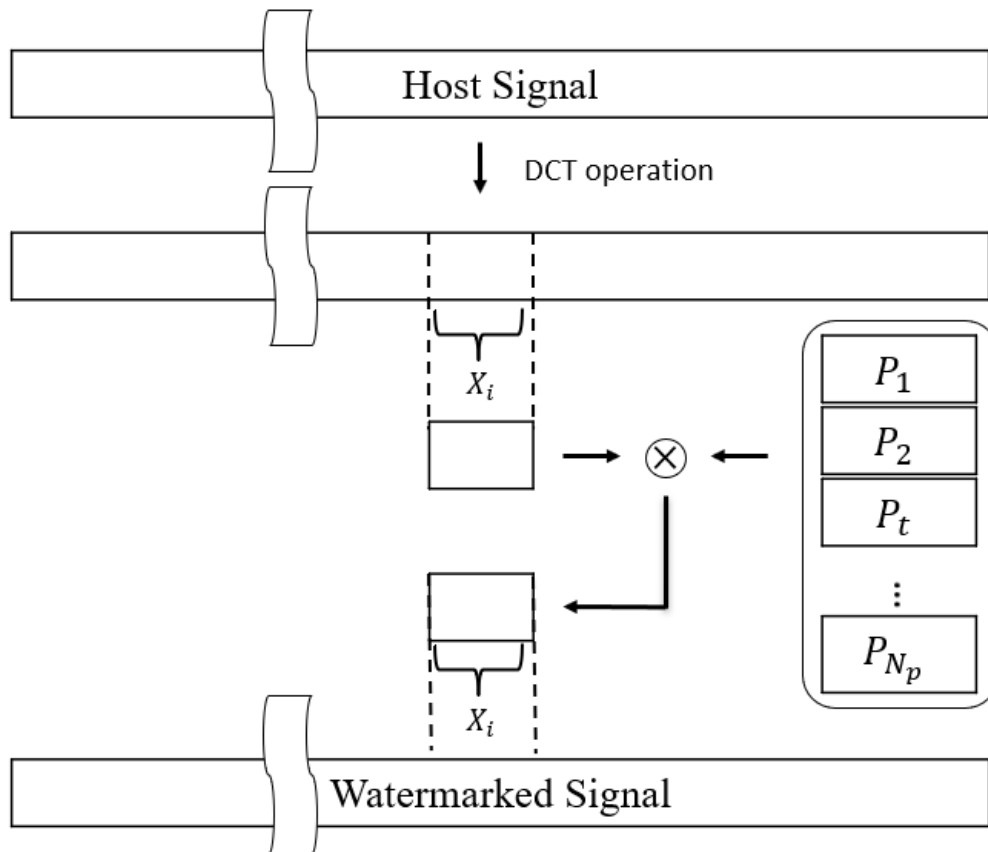


圖 2.3.3-2 高容量互補碼展頻浮水印演算法嵌入示意圖

演算法在浮水印嵌入之後，緊接著是萃取的部分。首先將嵌入完浮水印的音檔經由離散餘弦轉換(discrete cosine transform)轉換為離散餘弦係數，並且以 L_s 點將訊號切割為 N 個音框，再將所有的偽隨機碼序列一一與切割完之訊號($D_{i,j}$)作乘積，如式(2.3.3-11)，逐一比較並且找到乘積值最大的一個序列，此序列將會與相乘積的

偽隨機碼序列非常相關，我們便可以使用此序列對應的索引數字 m ，換算出浮水印嵌入之數值，浮水印資訊萃取式意圖如圖(2.3.3-3)所示。

$$D_{i,j} = |y_i p_j^T|$$

$$t = \arg \max_{j \in \{1,2,\dots,N_p\}} D_{i,j} = \arg \max_{j \in \{1,2,\dots,N_p\}} |y_i' p_j^T|$$

(2.3.3-11)

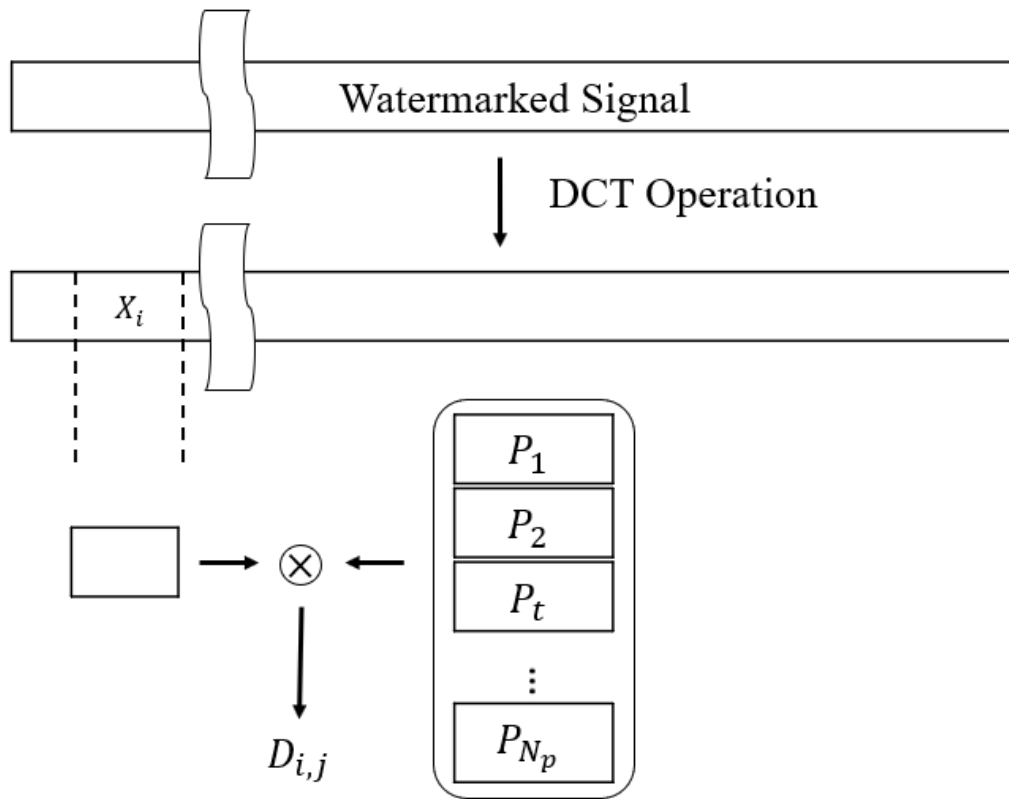


圖 2.3.3-3 高容量互補碼展頻浮水印演算法萃取示意圖

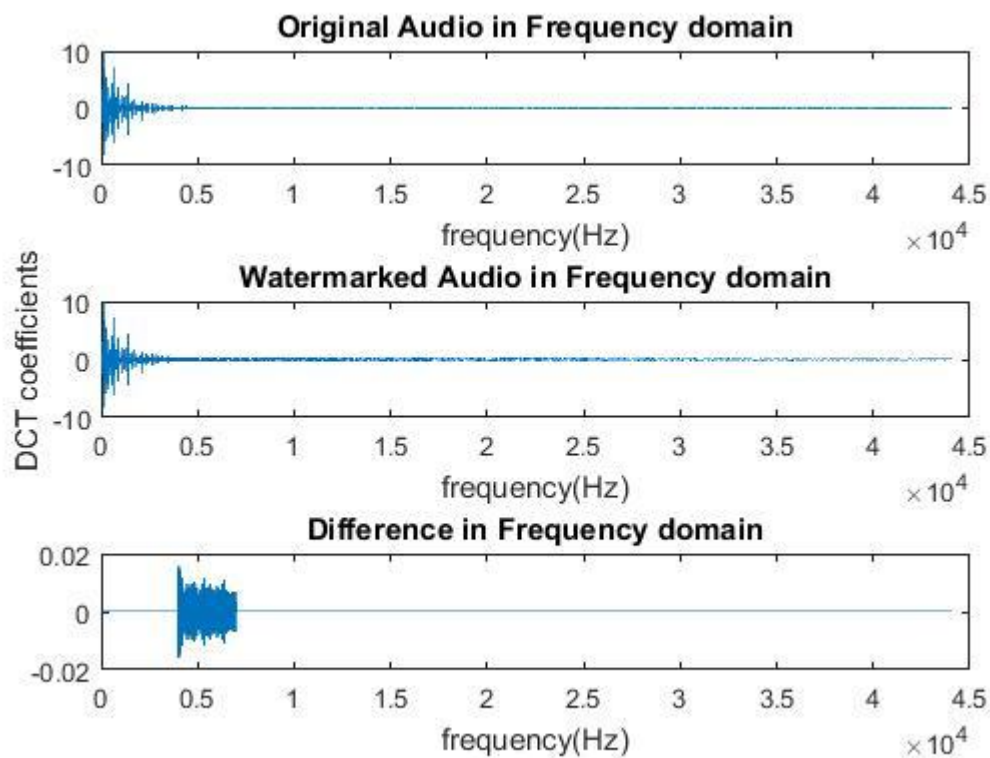


圖 2.3.3-4 互補碼展頻浮水印演算法在餘弦頻域之嵌入前後比較圖

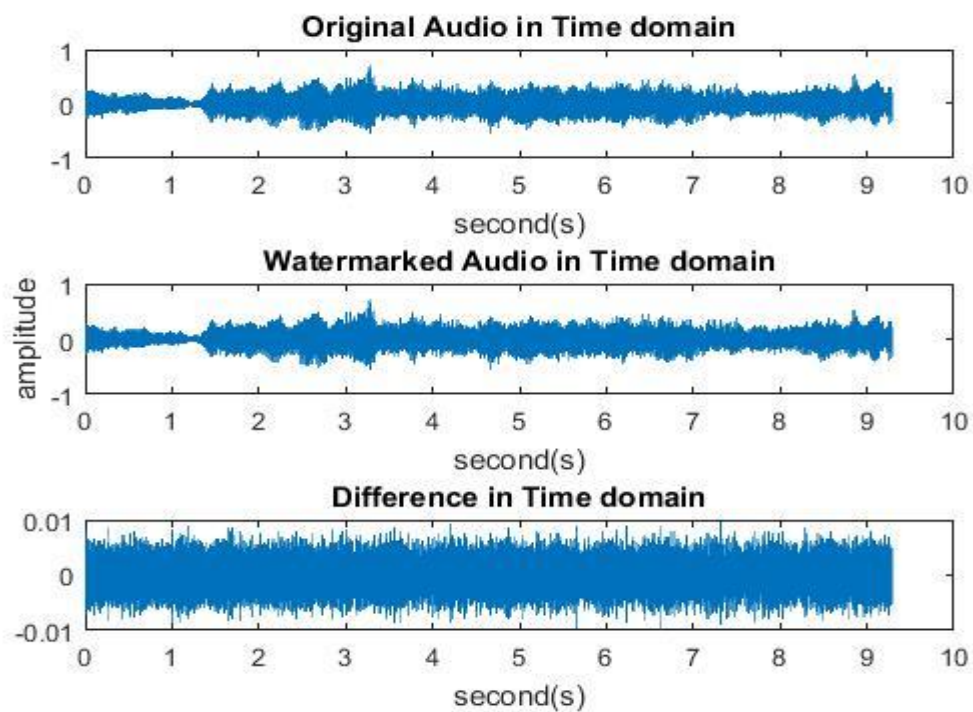


圖 2.3.3-5 互補碼展頻浮水印演算法在時域之嵌入前後比較圖

在上一章節所提到 Gram-Schmidt 展頻浮水印演算法產生出的偽隨機序列，其安全性上存在著問題，由於當中的序列藉由相異的兩個起始序列，卻可以產生出其他相似單位矩陣的形式，這方面確實產生出了疑慮，而在本章節的互補碼展頻演算法中，提出了超級互補碼序列，藉由此演算法所產生的序列，可以由圖(2.3.3-6)看出，其中的序列差異性相較於 Gram-Schmidt 正交法產生出的序列差異性相對較高，這也就提升了演算法的安全性，並且此方法也得以保留 Gram-Schmidt 正交法的萃取率。

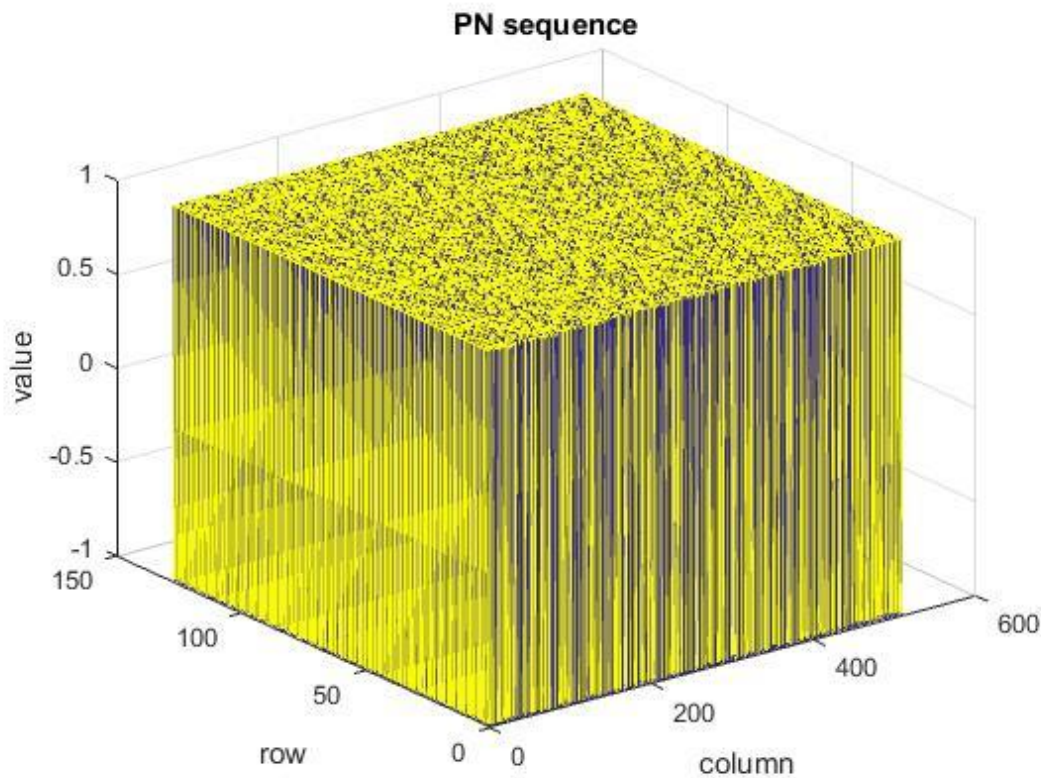


圖 2.3.3-6 互補碼序列圖

第三章

本章節主要探討本論文的演算法核心概念、演算法的架構以及相關方法進行探討、分析與研究。章節 3.1 為浮水印的嵌入架構以及流程介紹，章節 3.2 將會先以 Hadmard 矩陣開始介紹並舉例互補碼的產生，章節 3.3 會介紹互補碼的正交證明，章節 3.4 介紹如何萃取嵌入於語音訊號中的浮水印資訊。

3.1. 浮水印架構及嵌入流程.

在前一章節中我們提及了幾個較近期的展頻浮水印嵌入演算法，然而章節 2.3.3 的互補碼嵌入演算法，雖然解決了 2.3.2 的 Gram-Schmidt 正交化法安全性上的問題，但在演算法的複雜度上相對也提高了不少。因此本研究將以複雜度較低的演算法為主要方向，提出利用互補碼(Complementary Code)的方式建立出一組彼此互為正交且由數字 1 及 -1 所構成的偽隨機碼序列，使用序列的正交性質，達成能夠多層嵌入的演算法。本研究的嵌入流程架構如圖 3.1-1 所示，綜觀整體的嵌入流程，可以分為產生偽隨機碼序列及將浮水印嵌入音訊訊號的部分，然而在產生偽隨機序列的方面，將會先隨機的產生正交矩陣(Orthogonal matrix)，並且利用這幾組正交矩陣來產生一組偽隨機碼序列，也就是用於嵌入的互補碼序列，這部分將會在下面章節提及，接著是將浮水印嵌入至音訊訊號的部分，首先我們會把音訊訊號經由離散餘弦轉換，再將各層所需嵌入的浮水印資訊以及偽隨機碼序列作排列的處理，最後再將已經嵌入浮水印的訊號與轉換完的音訊訊號相加完成嵌入，並且將嵌入浮水印的頻域音訊訊號再經過返離散餘弦轉換回時域的音訊訊號，這部分在之後章節也會提及。

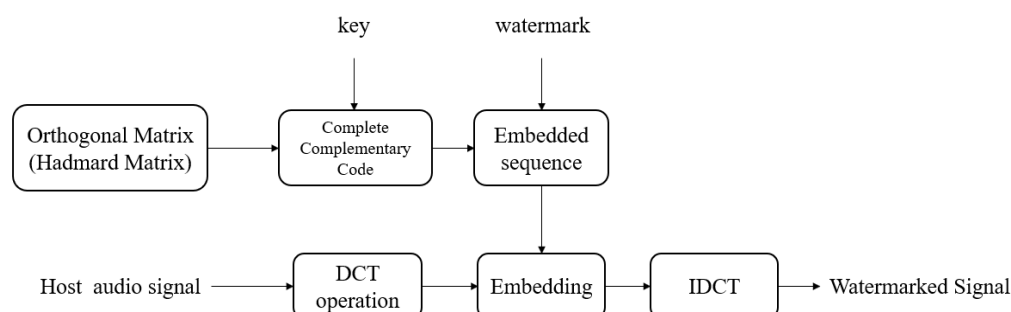


圖 3.1-1 互補碼展頻浮水印嵌入架構圖

3.2. 互補碼

互補碼(Complementary codes)由 Golay 與 Turun 等人在 1960 年代提出[17][18]，基本的互補碼定義為兩個長度相同的序列而這兩個序列中存在數目相同的相同元素以及不同的元素。以簡單的例子作為舉例， $R = - - - + - - + -$ 及 $Q = - - - + + + - +$ 就是一組互補碼(其中+代表+1，-代表-1)，兩組序列的長度皆為 8，並且這兩個序列的前面 4 個元素相同，後面 4 個元素不相同。互補碼有完美的正交性質，在本研究中使用互補碼的正交特性做為偽隨機碼序列以嵌入浮水印資訊。由於在章節 2.3.3 中提及了超級互補碼的產生方式[20]，在此章節將會舉例如何產生超級互補碼，並且將會著重於超級互補碼的正交性質證明[20]。

3.2.1. Hadmard 矩陣

由於本論文所使用到的超級互補碼將會運用正交矩陣，我們將在本章節介紹 hadmard 矩陣[21]。當考慮一個 N 階的 hadmard 矩陣，則此矩陣將滿足下列式子(3.2.1-1)，其中 I_N 為 $N \times N$ 的單位矩陣。

$$HH^T = NI_N \quad (3.2.1-1)$$

Hadmard 矩陣最初構造的例子是由 James Joseph Sylvester 所提出的。其假設 H 為一個 N 階的 Hadmard 矩陣，則下式(3.2.1-2)矩陣為 Hadmard 矩陣。

$$\begin{bmatrix} H & H \\ H & -H \end{bmatrix} \quad (3.2.1-2)$$

我們可以進一步歸納出 Hadmard 矩陣的標準式(3.2.1-3)[22]，接下來我們就可以產生出一個 N 階的 Hadmard 矩陣，其中 k 為非負整數。

$$H_{2^k} = \begin{bmatrix} H_{2^{k-1}} & H_{2^{k-1}} \\ H_{2^{k-1}} & -H_{2^{k-1}} \end{bmatrix} \quad (3.2.1-3)$$

觀察式(3.2.1-3)我們可以看出 Hadmard 矩陣有特殊的對稱性，連續運用這個構造，我們可以產生出一系列的 Hadmard 矩陣，如式(3.2.1-4)所式，矩陣中扣除第一列與第一行，其餘的各行各列矩陣元素+1 和-1 個數皆相同。

$$\begin{aligned}
H_1 &= [1] \\
H_2 &= \begin{bmatrix} H_1 & H_1 \\ H_1 & -H_1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \\
H_3 &= \begin{bmatrix} H_2 & H_2 \\ H_2 & -H_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}
\end{aligned}$$

(3.2.1-4)

3.2.2. 超級互補碼產生的例子

在產生超級互補碼[20]前，我們先利用 Hadmard 矩陣產生維度為 2 並且滿足式(2.3.3-1)、式(2.3.3-2)與式(2.3.3-4)的矩陣 A、B 以及 D。

$$A = \begin{bmatrix} + & + \\ + & - \end{bmatrix}, B = \begin{bmatrix} + & + \\ + & - \end{bmatrix}, D = \begin{bmatrix} + & + \\ + & - \end{bmatrix}$$

(3.2.2-1)

接著使用矩陣 A 與矩陣 B 以式(2.3.3-3)產生 C 序列

$$\begin{aligned}
C_1 &= (+ + + -) \\
C_2 &= (+ + - +)
\end{aligned}$$

(3.2.2-2)

再使用序列 C 與矩陣 D 以式(2.3.3-5)產生 E 序列

$$\begin{aligned}
E_1 &= [E_{11}, E_{12}] \\
E_2 &= [E_{21}, E_{22}]
\end{aligned}$$

(3.2.2-3)

$$\begin{aligned}
E_{11} &= (+ + + -) \\
E_{12} &= (+ - + +) \\
E_{21} &= (+ + - +) \\
E_{22} &= (+ - - -)
\end{aligned}$$

(3.2.2-4)

我們可以使用式(2.3.3-6)得到已擴展子碼長度的序列 F

$$\begin{aligned}
F_1 &= (+ + + - + + - +) \\
F_2 &= (+ + + - - - + -)
\end{aligned}$$

(3.2.2-5)

接著我們將式(3.2.2-5)利用式(2.3.3-5)重新產生出新的一組 E，再將產生出的 E 帶入式(2.3.3-8)即可以產生出一組彼此正交的超級互補碼。

$$\begin{aligned}
E_{11} &= T_1 = (+ + + - + + - +) \\
E_{12} &= T_2 = (+ - + + + - - -) \\
E_{21} &= T_3 = (+ + + - - - + -) \\
E_{22} &= T_4 = (+ - + + - + + +)
\end{aligned}
\tag{3.2.2-6}$$

$$\begin{aligned}
V_1 &= (+ + + -, + + - +, + + + -, - - + -) \\
V_2 &= (+ - + +, + - - -, + - + +, - + + +) \\
V_3 &= (+ + - +, + + + -, + + - +, - - - +) \\
V_4 &= (+ - - -, + - + +, + - - -, - + - -) \\
V_5 &= (+ + + -, + + - +, - - - +, + + - +) \\
V_6 &= (+ - + +, + - - -, - + - -, + - - -) \\
V_7 &= (+ + - +, + + + -, - - + -, + + + -) \\
V_8 &= (+ - - -, + - + +, - + + +, + - + +)
\end{aligned}
\tag{3.2.2-7}$$

3.2.3. 超級互補碼的正交證明

在前面的小節中，我們介紹如何產生出彼此正交的超級互補碼，在這小節中，我們將進一步證明[20]其正交性質，由於證明過程繁雜，於是分成幾種不同的情況，以下是我們分析證明的步驟：

步驟 1. 我們先將之後會用到的證明條件列出。若已知 $Z = X \times Y$ ，則我們可得 $Z^* = X^* \times Y^*$ ，其中“*”為共軛符號，其證明如下：

$$\begin{aligned}
Z &= X \times Y, \quad X = a + bi, \quad Y = c + di \\
Z &= X \times Y = (a + bi)(c + di) = (ac - bd) + i(ad + bc) \\
Z^* &= (ac - bd) - i(ad + bc) \\
X^* \times Y^* &= (a - bi)(c - di) = (ac - bd) - i(ad + bc) = Z^*
\end{aligned}
\tag{3.2.3-1}$$

而若我們已知 $Z = X \times Y$ 且 X 和 Y 的範數(norm)為1，則 Z 的範數(norm)也會為1，證明如下：

$$\begin{aligned}
Z &= X \times Y, \quad |X| = 1, \quad |Y| = 1 \\
|Z|^2 &= Z \times Z^* = X \times Y \times X^* \times Y^* = (X \times X^*)(Y \times Y^*) = |X|^2 \times |Y|^2 = 1 \\
\therefore |Z| &= 1
\end{aligned}
\tag{3.2.3-2}$$

接著我們要證明完全互補碼與擴展完全互補碼中每一個子碼在同步的情況下是正交的，於是我們將為四部份證明，第一部分為完全互補碼中同一個碼裡的兩個子碼彼此的正交關係，第二部分為完全互補碼中不同碼裡的子碼彼此的正交關係，第三部分為擴展完全互補碼中同一個碼裡的兩個子碼彼此的正交關係，第四部份為擴展完全互補碼中不同碼裡彼此的子碼的正交關係。

第一部分證明為完全互補碼中同一個碼裡的兩個子碼彼此的正交關係：

$$\begin{aligned}
 & \rho(E_{ni}, E_{nj}; 0) \\
 &= \frac{1}{N^2} (e_{ni1} e_{nj1}^* + e_{ni2} e_{nj2}^* + \cdots + e_{niN^2} e_{njN^2}^*) \\
 &= \frac{1}{N^2} (c_{n1} d_{i1} c_{n1}^* d_{j1}^* + c_{n2} d_{i2} c_{n2}^* d_{j2}^* + \cdots + c_{nN^2} d_{iN} c_{nN^2}^* d_{jN}^*) \\
 &= \frac{N}{N^2} (d_{i1} d_{j1}^* + d_{i2} d_{j2}^* + \cdots + d_{iN} d_{jN}^*) = 0 \\
 & \text{for } n = 1, 2, \dots, N; i, j = 1, 2, \dots, N \text{ and } i \neq j
 \end{aligned} \tag{3.2.3-3}$$

第二部分為證明完全互補碼中不同互補碼裡的子碼彼此的正交關係：

$$\begin{aligned}
 & \rho(E_{ni}, E_{mj}; 0) = \frac{1}{N^2} (e_{ni1} e_{mj1}^* + e_{ni2} e_{mj2}^* + \cdots + e_{niN^2} e_{mjN^2}^*) \\
 &= \frac{1}{N^2} (c_{n1} d_{i1} c_{m1}^* d_{j1}^* + c_{n2} d_{i2} c_{m2}^* d_{j2}^* + \cdots + c_{nN^2} d_{iN} c_{mN^2}^* d_{jN}^*) \\
 &= \frac{1}{N^2} (b_{n1} a_{11} d_{i1} b_{m1}^* a_{11}^* d_{j1}^* + b_{n1} a_{12} d_{i2} b_{m1}^* a_{12}^* d_{j2}^* + \cdots + b_{nN} a_{NN} d_{iN} b_{mN}^* a_{NN}^* d_{jN}^*) \\
 &= \frac{1}{N^2} (b_{n1} d_{i1} b_{m1}^* d_{j1}^* + b_{n1} d_{i2} b_{m1}^* d_{j2}^* + \cdots + b_{n1} d_{iN} b_{m1}^* d_{jN}^*) \\
 &+ \frac{1}{N^2} (b_{n2} d_{i1} b_{m2}^* d_{j1}^* + b_{n2} d_{i2} b_{m2}^* d_{j2}^* + \cdots + b_{n2} d_{iN} b_{m2}^* d_{jN}^*) \\
 &\quad \vdots \\
 &+ \frac{1}{N^2} (b_{nN} d_{i1} b_{mN}^* d_{j1}^* + b_{nN} d_{i2} b_{mN}^* d_{j2}^* + \cdots + b_{nN} d_{iN} b_{mN}^* d_{jN}^*) \\
 &= \frac{1}{N^2} [(d_{i1} d_{j1}^* (b_{n1} b_{m1}^* + b_{n2} b_{m2}^* + \cdots b_{nN} b_{mN}^*))] \\
 &+ \frac{1}{N^2} [(d_{i2} d_{j2}^* (b_{n1} b_{m1}^* + b_{n2} b_{m2}^* + \cdots b_{nN} b_{mN}^*))] \\
 &\quad \vdots \\
 &+ \frac{1}{N^2} [(d_{iN} d_{jN}^* (b_{n1} b_{m1}^* + b_{n2} b_{m2}^* + \cdots b_{nN} b_{mN}^*))]
 \end{aligned}$$

$$= 0 \text{ for } n, m = 1, 2, \dots, N \text{ and } n \neq m; i, j = 1, 2, \dots, N$$

(3.2.3-4)

接著第三部分我們要證明擴展完全互補碼中同一個碼裡的兩個子碼彼此的正交關係，而在下面的證明，假設擴展完全互補碼的子碼長度已經過展到 N^r ，而且我們用 e 來表示現在的擴展完全互補碼裡面的元素，而 e' 是表示前一次擴展完全互補碼裡面的元素：

$$\begin{aligned} \rho(E_{ni}, E_{nj}; 0) &= \frac{1}{N^r} (e_{ni1} e_{nj1}^* + e_{ni2} e_{nj2}^* + \dots + e_{niN^2} e_{njN^2}^*) \\ &= \frac{1}{N^r} (f_{n1} d_{i1} f_{n1}^* d_{j1}^* + f_{n2} d_{i2} f_{n2}^* d_{j2}^* + \dots + f_{nN^r} d_{iN} f_{nN^r}^* d_{jN}^*) \\ &= \frac{1}{N^r} (e'_{n11} d_{i1} e_{n11}^* d_{j1}^* + e'_{n21} d_{i2} e_{n21}^* d_{j2}^* + \dots + e'_{nNN^{r-1}} d_{iN} e_{nNN^{r-1}}^* d_{jN}^*) \\ &= \frac{N^{r-1}}{N^2} (d_{i1} d_{j1}^* + d_{i2} d_{j2}^* + \dots + d_{iN} d_{jN}^*) \\ &= 0 \text{ for } n = 1, 2, \dots, N; i, j = 1, 2, \dots, N \text{ and } i \neq j; r \geq 3 \end{aligned}$$

(3.2.3-5)

第四部份我們要證明擴展完全互補碼中不同碼裡的子碼彼此的正交關係：

$$\begin{aligned} \rho(E_{ni}, E_{mj}; 0) &= \frac{1}{N^2} (e_{ni1} e_{mj1}^* + e_{ni2} e_{mj2}^* + \dots + e_{niN^2} e_{mjN^2}^*) \\ &= \frac{1}{N^r} (e_{ni1} e_{mj1}^* + e_{ni2} e_{mj2}^* + \dots + e_{niN^r} e_{mjN^r}^*) \\ &= \frac{1}{N^r} (f_{n1} d_{i1} f_{m1}^* d_{j1}^* + f_{n2} d_{i2} f_{m2}^* d_{j2}^* + \dots + f_{nN^r} d_{iN} f_{mN^r}^* d_{jN}^*) \\ &= \frac{1}{N^r} (e'_{n11} d_{i1} e_{m11}^* d_{j1}^* + e'_{n21} d_{i2} e_{m21}^* d_{j2}^* + \dots + e'_{nN1} d_{iN} e_{mN1}^* d_{jN}^*) \\ &\quad + \frac{1}{N^r} (e'_{n12} d_{i1} e_{m12}^* d_{j1}^* + e'_{n22} d_{i2} e_{m22}^* d_{j2}^* + \dots + e'_{nN2} d_{iN} e_{mN2}^* d_{jN}^*) \\ &\quad \vdots \\ &\quad + \frac{1}{N^r} (e'_{n1N^{r-1}} d_{i1} e_{m1N^{r-1}}^* d_{j1}^* + e'_{n2N^{r-1}} d_{i2} e_{m2N^{r-1}}^* d_{j2}^* + \dots + e'_{nNN^{r-1}} d_{iN} e_{mNN^{r-1}}^* d_{jN}^*) \\ &= \frac{1}{N^r} \left[(d_{i1} d_{j1}^* (e'_{n11} e_{m11}^* + e'_{n12} e_{m12}^* + \dots + e'_{n1N^{r-1}} e_{m1N^{r-1}}^*)) \right] \\ &\quad + \frac{1}{N^r} \left[(d_{i2} d_{j2}^* (e'_{n21} e_{m21}^* + e'_{n22} e_{m22}^* + \dots + e'_{n2N^{r-1}} e_{m2N^{r-1}}^*)) \right] \\ &\quad \vdots \end{aligned}$$

$$\begin{aligned}
& + \frac{1}{N^r} \left[\left(d_{iN} d_{jN}^* (e'_{nN1} e_{mN1}^* + e'_{nN2} e_{mN2}^* + \cdots e'_{nNN^{r-1}} e_{mNN^{r-1}}^*) \right) \right] \\
& = 0 \text{ for } n, m = 1, 2, \dots, N \text{ and } n \neq m; i, j = 1, 2, \dots, N; r \geq 3
\end{aligned}
\tag{3.2.3-6}$$

步驟 2.

接著要證明的是超級互補碼基本碼的相關函數正交互補性質，而此基本碼包括不同子碼長度的基本碼，於是我們將討論各種長度的可能性，證明在下文分為三部分，第一部分為位移是 0 時的互相關函數正交互補性，第二部分為位移為奇數的自相關函數以及互相關函數的正交互補性，第三部分為偶數的自相關函數以及互相關函數的正交互補性。

第一部分證明為位移是 0 時的互相關函數正交互補性，而可以發現超級互補碼的基本碼在位移 0 時的互相關函數其實會等於完全互補碼或擴展完全互補碼的每個子碼在位移 0 時的互相關函數，由(3.2.3-3)、(3.2.3-4)、(3.2.3-5)、(3.2.3-6)四式中得到完全互補碼和擴展完全互補碼的每個子碼在位移 0 的互相關函數皆為 0，由此可知超級互補碼的基本碼在位移 0 的相關函數是正交互補的，而下式即證明在不同基本碼碼長之情況下皆成立：

$$\begin{aligned}
\sum_{i=1}^4 \rho(T_{ni}, T_{mi}; 0) &= \rho(E_{pq}, E_{uv}; 0) = 0 \text{ for } n, m = 1, 2, 3, 4 \text{ and } n \neq m \\
n &= 2(p-1) + q \text{ and } m = 2(u-1) + v \\
p, q, u, v &= 1, 2
\end{aligned}
\tag{3.2.3-7}$$

第二部分與第三部分分別是，位移為奇數時的自相關係數與互相關係數、位移為偶數時的自相關係數與互相關係數，然而在證明前假設已知在基本碼長度為 2^r 情況下，相關函數的正交互補特性是成立的：

$$\sum_{i=1}^4 \rho(T'_{ni}, T'_{mi}; k) = 0 \text{ for } k = 1, 2, \dots, 2^r - 1 \text{ and } n = 1, 2, 3, 4
\tag{3.2.3-8}$$

$$\sum_{i=1}^4 \rho(T'_{ni}, T'_{mi}; k) = 0 \text{ for } k = 1, 2, \dots, 2^r - 1 \text{ and } n = 1, 2, 3, 4 \text{ and } n \neq m
\tag{3.2.3-9}$$

有了此假設，我們必須證明在子碼長度為 2^{r+1} 情況下，相關函數的正交互補性

也是成立的。首先為位移是奇數的情況， T 代表超級互補碼基本碼子碼長度為 2^{r+1} 的子碼而 T' 代表超級互補碼基本碼子碼長度為 2^r 的子碼：

$$\begin{aligned}
& \sum_{i=1}^4 \rho(T_{ni}, T_{mi}; 2k-1) \\
&= \sum_{i=1}^4 \frac{1}{2^{r+1}} [(t_{ni(1+2k-1)} t_{mi1}^* + t_{ni(2+2k-1)} t_{mi2}^* + \cdots + t_{ni2^{r+1}} t_{mi(2^{r+1}-2k+1)}^*)] \\
&= \sum_{i=1}^4 \frac{1}{2^{r+1}} [(t'_{(2p)i(1+k-1)} d_{q2} t_{(2u-1)i1}^* d_{v1}^* + t'_{(2p-1)i(1+k)} d_{q1} t_{(2u)}^* d_{v2}^* + \cdots \\
&\quad + t'_{(2p)i2^r} d_{q2} t_{(2u-1)i(2^r-k+1)}^* d_{v1}^*)] \\
&= \sum_{i=1}^4 \frac{1}{2^{r+1}} d_{q2} d_{v1}^* [(t'_{(2p)i(1+k-1)} t_{(2u-1)i1}^* + t'_{(2p)i(2+k-1)} t_{(2u-1)i2}^* + \cdots \\
&\quad + t'_{(2p)i2^r} t_{(2u-1)i(2^r-k+1)}^*)] \\
&\quad + \sum_{i=1}^4 \frac{1}{2^{r+1}} d_{q1} d_{v2}^* [(t'_{(2p-1)i(1+k)} t_{(2u)i1}^* + t'_{(2p-1)i(2+k)} t_{(2u)i2}^* + \cdots \\
&\quad + t'_{(2p-1)i2^r} t_{(2u)i(2^r-k)}^*)] \\
&= \frac{1}{2} d_{q2} d_{v1}^* \sum_{i=1}^4 \rho(T'_{(2p)i}, T'_{(2u-1)i}; k-1) + \frac{1}{2} d_{q1} d_{v2}^* \sum_{i=1}^4 \rho(T'_{(2p-1)i}, T'_{(2u)i}; k) \\
&\quad = \frac{1}{2} d_{q2} d_{v1}^* * 0 + \frac{1}{2} d_{q1} d_{v2}^* * 0 \\
&= 0 \text{ for } k = 1, 2, \cdots, 2^r \text{ and } n = 2(p-1) + q; m = 2(u-1) + v \\
&\quad n, m = 1, 2, 3, 4 \text{ and } p, q, u, v = 1, 2
\end{aligned} \tag{3.2.3-10}$$

最後我們要證明位移是偶數的情況， T 代表超級互補碼基本碼子碼長度為 2^{r+1} 的子碼而 T' 代表超級互補碼基本碼子碼長度為 2^r 的子碼：

$$\begin{aligned}
& \sum_{i=1}^4 \rho(T_{ni}, T_{mi}; 2k) \\
&= \sum_{i=1}^4 \frac{1}{2^{r+1}} [(t_{ni(1+2k)} t_{mi1}^* + t_{ni(2+2k)} t_{mi2}^* + \cdots + t_{ni2^{r+1}} t_{mi(2^{r+1}-2k)}^*)]
\end{aligned}$$

$$\begin{aligned}
&= \sum_{i=1}^4 \frac{1}{2^{r+1}} [(t'_{(2p-1)i(1+k)} d_{q1} t'^*_{(2u-1)i1} d_{v1}^* + t'_{(2p)i(1+k)} d_{q2} t'^*_{(2u)i1} d_{v2}^* + \cdots \\
&\quad + t'_{(2p)i2^r} d_{q2} t'^*_{(2u)i(2^r-k)} d_{v2}^*)] \\
&= \sum_{i=1}^4 \frac{1}{2^{r+1}} d_{q1} d_{v1}^* [(t'_{(2p-1)i(1+k)} t'^*_{(2u-1)i1} + t'_{(2p-1)i(2+k)} t'^*_{(2u-1)i2} + \cdots \\
&\quad + t'_{(2p-1)i2^r} t'^*_{(2u-1)i(2^r-k)})] \\
&+ \sum_{i=1}^4 \frac{1}{2^{r+1}} d_{q2} d_{v2}^* [(t'_{(2p)i(1+k)} t'^*_{(2u)i1} + t'_{(2p)i(2+k)} t'^*_{(2u)i2} + \cdots + t'_{(2p)i2^r} t'^*_{(2u)i(2^r-k)})] \\
&= \frac{1}{2} d_{q1} d_{v1}^* \sum_{i=1}^4 \rho(T'_{(2p-1)i}, T'_{(2u-1)i}; k-1) + \frac{1}{2} d_{q2} d_{v2}^* \sum_{i=1}^4 \rho(T'_{(2p)i}, T'_{(2u)i}; k) \\
&= \frac{1}{2} d_{q1} d_{v1}^* * 0 + \frac{1}{2} d_{q2} d_{v2}^* * 0 \\
&= 0 \text{ for } k = 1, 2, \dots, 2^r - 1 \text{ and } n = 2(p-1) + q; m = 2(u-1) + v \\
&\quad n, m = 1, 2, 3, 4 \text{ and } p, q, u, v = 1, 2
\end{aligned} \tag{3.2.3-11}$$

將上面三部分結合在一起，我們就可證明出超級互補碼基本碼是有完美正交性的。

步驟 3.

接下來我們要證明擴展後會保有原先基本碼的完美正交性，我們在步驟 2 得知：

$$\begin{aligned}
&\sum_{i=1}^N \rho(T_{ji}, T_{ji}; \tau) = 0 \text{ for } \tau \neq 0; j = 1, 2, \dots, N \text{ and } j \neq k \\
&\sum_{i=1}^N \rho(T_{ji}, T_{ki}; \tau) = 0 \text{ for any } \tau; j = 1, 2, \dots, N \text{ and } j \neq k
\end{aligned} \tag{3.2.3-12}$$

於此同時我們便能得到基本碼的一些特性：

$$\begin{aligned}
&\sum_{i=1}^N \rho(T_{ji}, T_{ji}^-; \tau) = 0 \text{ for } \tau \neq 0; j = 1, 2, \dots, N \\
&\sum_{i=1}^N \rho(T_{ji}^-, T_{ji}^-; \tau) = 0 \text{ for } \tau \neq 0; j = 1, 2, \dots, N
\end{aligned}$$

$$\sum_{i=1}^N \rho(T_{ji}, \overline{T_{ki}}; \tau) = 0 \text{ for any } \tau; j, k = 1, 2, \dots, N \text{ and } j \neq k$$

$$\sum_{i=1}^N \rho(\overline{T_{ji}}, \overline{T_{ki}}; \tau) = 0 \text{ for any } \tau; j, k = 1, 2, \dots, N \text{ and } j \neq k$$

(3.2.3-13)

接下來我們分成兩部分分析擴展完全互補碼仍保有基本碼的完美正交性

$$\begin{aligned} \sum_{m=1}^{2N} \rho(S_{(4j+1)m}, S_{(4j+1)m}; \tau) &= \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2j+1)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2j+2)n}; \tau) \\ &= 0 \\ \sum_{m=1}^{2N} \rho(S_{(4j+2)m}, S_{(4j+2)m}; \tau) &= \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2j+1)n}; \tau) + \sum_{n=1}^N \rho(\overline{T_{(2j+2)n}}, \overline{T_{(2j+2)n}}; \tau) \\ &= 0 \\ \sum_{m=1}^{2N} \rho(S_{(4j+3)m}, S_{(4j+3)m}; \tau) &= \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2j+2)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2j+1)n}; \tau) \\ &= 0 \\ \sum_{m=1}^{2N} \rho(S_{(4j+4)m}, S_{(4j+4)m}; \tau) &= \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2j+2)n}; \tau) + \sum_{n=1}^N \rho(\overline{T_{(2j+1)n}}, \overline{T_{(2j+1)n}}; \tau) \\ &= 0 \end{aligned}$$

for $\tau \neq 0; j = 0, 1, \dots, \frac{N}{2} - 1$

(3.2.3-14)

由上式我們可以證明擴展後的超級互補碼仍保有基本碼的自相關函數正交互補性質，整理後可得：

$$\sum_{i=1}^N \rho(S_{ji}, S_{ji}; \tau) = 0 \text{ for } \tau \neq 0; j = 1, 2, \dots, 2N$$

我們接著證明(2.3.3-8)也會保有基本碼的互相關函數正交互補性質

$$\begin{aligned} \sum_{m=1}^{2N} \rho(S_{(4j+1)m}, S_{(4k+1)m}; \tau) &= \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+1)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+1)n}; \tau) \\ &= 0 \end{aligned}$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+2)m}, S_{(4k+2)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+1)n}; \tau) + \sum_{n=1}^N \rho(\overline{T_{(2j+2)n}}, \overline{T_{(2k+2)n}}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+3)m}, S_{(4k+3)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+1)n}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+4)m}, S_{(4k+4)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(\overline{T_{(2j+1)n}}, \overline{T_{(2k+1)n}}; \tau)$$

$$= 0$$

for any $\tau; j, k = 0, 1, \dots, \frac{N}{2} - 1$ and $j \neq k$

(3.2.3-15)

$$\sum_{m=1}^{2N} \rho(S_{(4j+1)m}, S_{(4k+2)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+1)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+2)n}, \overline{T_{(2k+2)n}}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+1)m}, S_{(4k+3)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2k+1)n}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+1)m}, S_{(4k+4)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+2)n}, \overline{T_{(2k+1)n}}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+2)m}, S_{(4k+3)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+2)n}, \overline{T_{(2k+1)n}}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+2)m}, S_{(4k+4)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+1)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(\overline{T_{(2j+2)n}}, \overline{T_{(2k+1)n}}; \tau)$$

$$= 0$$

$$\sum_{m=1}^{2N} \rho(S_{(4j+2)m}, S_{(4k+4)m}; \tau) = \sum_{n=1}^N \rho(T_{(2j+2)n}, T_{(2k+2)n}; \tau) + \sum_{n=1}^N \rho(T_{(2j+1)n}, \overline{T_{(2k+1)n}}; \tau)$$

$$= 0$$

$$\text{for any } \tau; j, k = 0, 1, \dots, \frac{N}{2} - 1$$

(3.2.3-16)

上面兩式含所有可能的互相關函數情形，以此可以看出基本碼的正交互補性質被保留，整理後可得：

$$\sum_{i=1}^N \rho(S_{ji}, S_{ki}; \tau) = 0 \text{ for any } \tau; j, k = 1, 2, \dots, 2N \text{ and } j \neq k$$

在此小節我們經由步驟 1. 2. 3. 可以證明出超級完全互補碼的完美正交性。

3.3. 浮水印嵌入流程

本章節我們將針對每一個浮水印嵌入的步驟進行介紹，浮水印嵌入演算法的架構如圖 3.3-1 所示，嵌入浮水印的架構分為：將語音訊號進行離散餘弦轉換、將偽隨機碼序列排列[23]、演算法浮水印資訊的嵌入以及最後將嵌入完之訊號以反離散餘弦轉換為時域訊號。

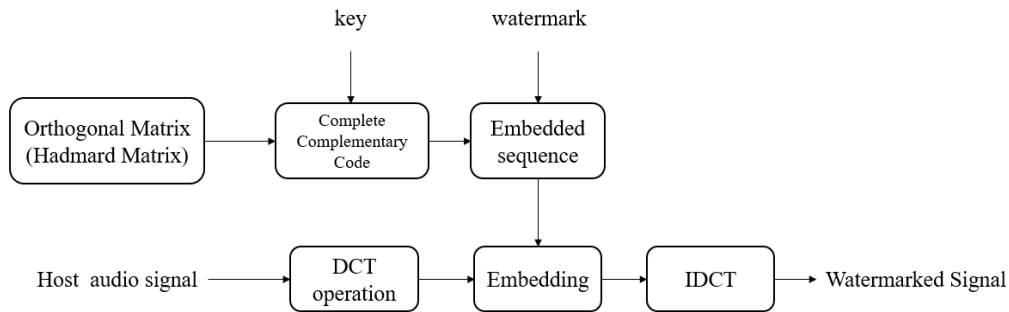


圖 3.3-1 本論文演算法浮水印嵌入架構圖

3.3.1. 離散餘弦轉換與嵌入頻段選擇

對於一段音訊訊號而言，在訊號低頻以及高頻的部分容易受到濾波器、音訊壓縮修改及影響，因此我們必須選擇適合的頻段對浮水印資訊做嵌入，首先我們以取樣頻率 F_s 及音訊長度為 L 個取樣點的原始音訊訊號 $x(n)$ ，利用離散餘弦轉換將音訊訊號轉

換為頻域，轉換的公式如式 3.3.1-1 所示。

$$Y(k) = E(m) \sum_{n=0}^{L-1} x(n) \cos\left(\frac{m\pi(2n+1)}{2L}\right), \quad m = 0, 1, \dots, L-1 \quad (3.3.1-1)$$

$$E(m) = \begin{cases} \frac{1}{\sqrt{L}}, & \text{if } m = 0 \\ \sqrt{\frac{2}{L}}, & \text{if } 1 \leq m < L \end{cases} \quad (3.3.1-2)$$

在轉換完後，根據希望的起始頻段 f_{low} 利用式(3.3.1-3)求出對應之離散餘弦係數位置 M_{low} 。

$$f_{step} = \frac{f_s}{L}$$

$$[M_{low}] = [\text{ceil}(\frac{f_{low}}{f_{step}})] \quad (3.3.1-3)$$

本論文的起始頻段 f_{low} 的選擇為頻率 1000Hz 至 2000Hz 之間，藉由 1000Hz 開始以每 100Hz 分別計算客觀數據評估(Objective difference grade, ODG)以取得最佳客觀數據評估的起始頻率。為了方便後續萃取介紹，我們將選擇完頻段的離散餘弦轉換係數以 Y 表示。

3.3.2. 浮水印訊號與偽隨機碼序列之排列

本演算法在嵌入浮水印資訊時使用互補碼序列排列過後的序列進行浮水印資訊 $d_t^{(i)} \triangleq (d_0^{(i)}, d_1^{(i)}, \dots, d_{B^{(i)}-1}^{(i)}) \in \{+1, -1\}^{B^{(i)}}$ 的嵌入，其中 $B^{(i)}$ 為浮水印位元個數 i 則表示屬於第幾層之浮水印位元，首先我們利用章節 3.2 所提到之互補法產生方法產生出一組互補碼序列，如式(3.3.2-1)所示，其中 m 為互補碼序列數， n 為互補碼子碼組數，子碼長度則為 l 。首先我們將互補碼序列的子碼位移 T ，以式 3.3.2-2 產生基礎序列(Basic sequence) $S[23]$ ，產生示意圖如圖 3.3.2-1 所示，基礎序列被用於浮水印資訊嵌入，而單層嵌入的浮水印位元數為位移大小 T 與子碼長度 l 的差值 $T-l$ ，最後基礎序列長度為 $(n-1)T+l$ 。由於本演算法為多層浮水印嵌入故總嵌入浮水印資訊位元數為差值與層數 m 的乘機，嵌入位元總數為 $(T-l) \times m$ 。浮水印資訊的嵌入藉由基礎序列

的位移，依序將 $T-l$ 個浮水印位元以式(3.3.2-3)嵌入單一層基礎序列當中，如圖 3.3.2-2 所示， m 層不同的浮水印資訊以相同步驟嵌入至各層基礎序列當中產生捲積序列 (Convolved sequence) W_N [23]，如式(3.3.2-4)所示，而嵌入方式如圖 3.3.2-3 所示，最後將捲積序列與經由離散餘弦轉換完的音訊頻段之離散餘弦係數相加，完成浮水印資訊的嵌入。

$$\begin{aligned}
 V^{(0)} &= (C_0^{(0)}, C_1^{(0)}, \dots, C_{n-1}^{(0)}) \\
 V^{(1)} &= (C_0^{(1)}, C_1^{(1)}, \dots, C_{n-1}^{(1)}) \\
 &\vdots \\
 V^{(m-1)} &= (C_0^{(m-1)}, C_1^{(m-1)}, \dots, C_{n-1}^{(m-1)})
 \end{aligned}
 \tag{3.3.2-1}$$

$$S^{(i)} \stackrel{\text{def}}{=} \sum_{j=0}^{n-1} C_j^{(i)} Z^{-jT}
 \tag{3.3.2-2}$$

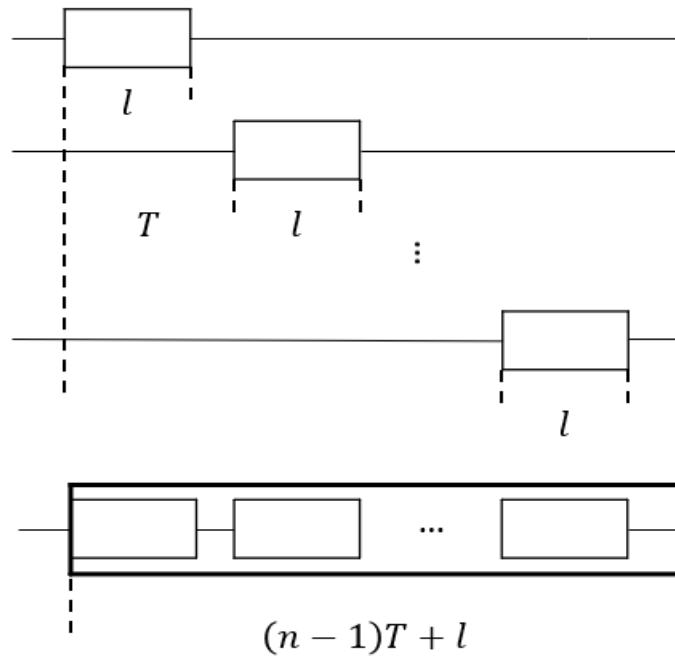


圖 3.3.2-1 互補碼基礎序列產生示意圖

$$W^{(i)} \stackrel{\text{def}}{=} \sum_{t=0}^{B^{(i)}-1} d_t^{(i)} S^{(i)} Z^{-t}
 \tag{3.3.2-3}$$

$$W_N = \sum_{i=0}^{m-1} W^{(i)}$$

(3.3.2-4)

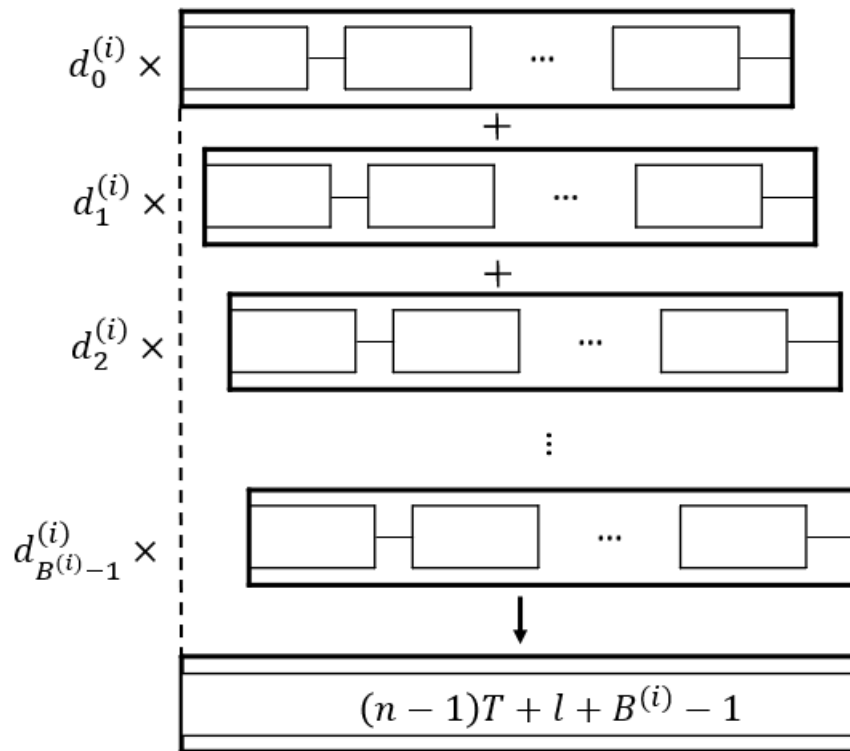


圖 3.3.2-2 單層捲積序列產生示意圖

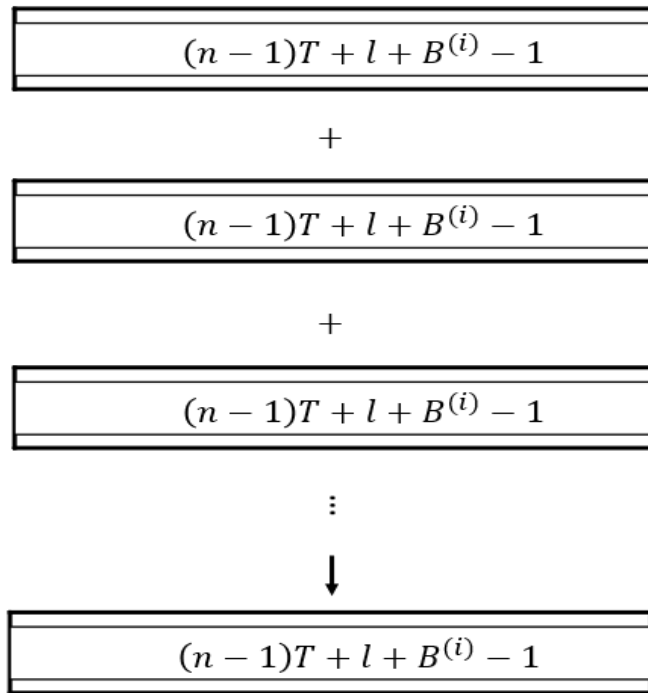


圖 3.3.2-3 多層捲積序列產生示意圖

最後嵌入完的訊號與嵌入前訊號時域差值比較圖如圖 3.3.2-4 所示，頻域差值圖比較圖如圖 3.3.2-5 所示。

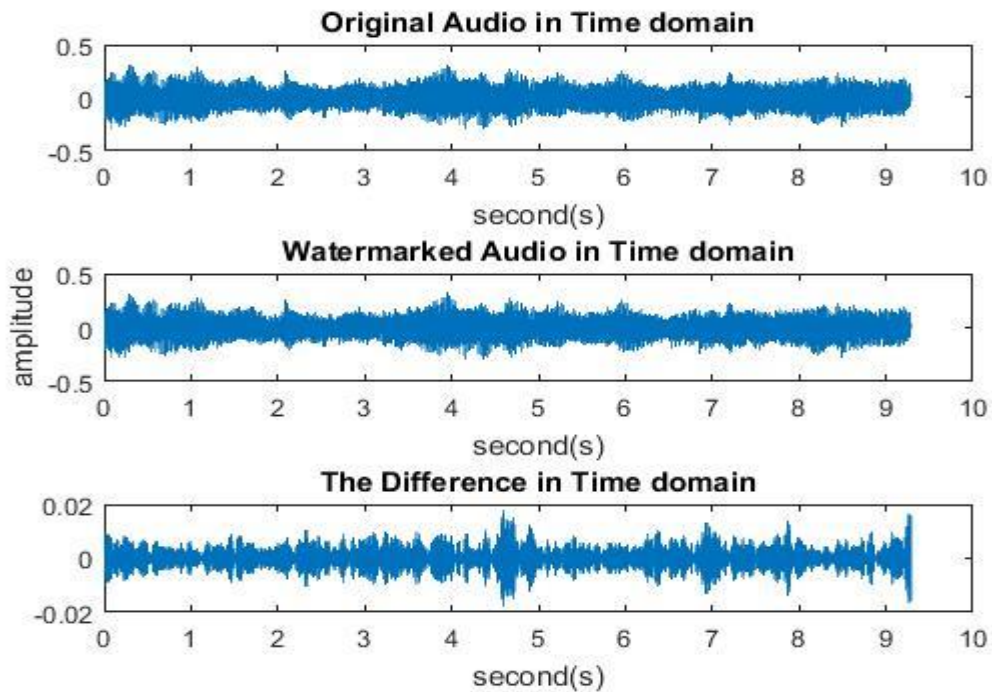


圖 3.3.2-4 浮水印嵌入前後與時域差異值比較圖

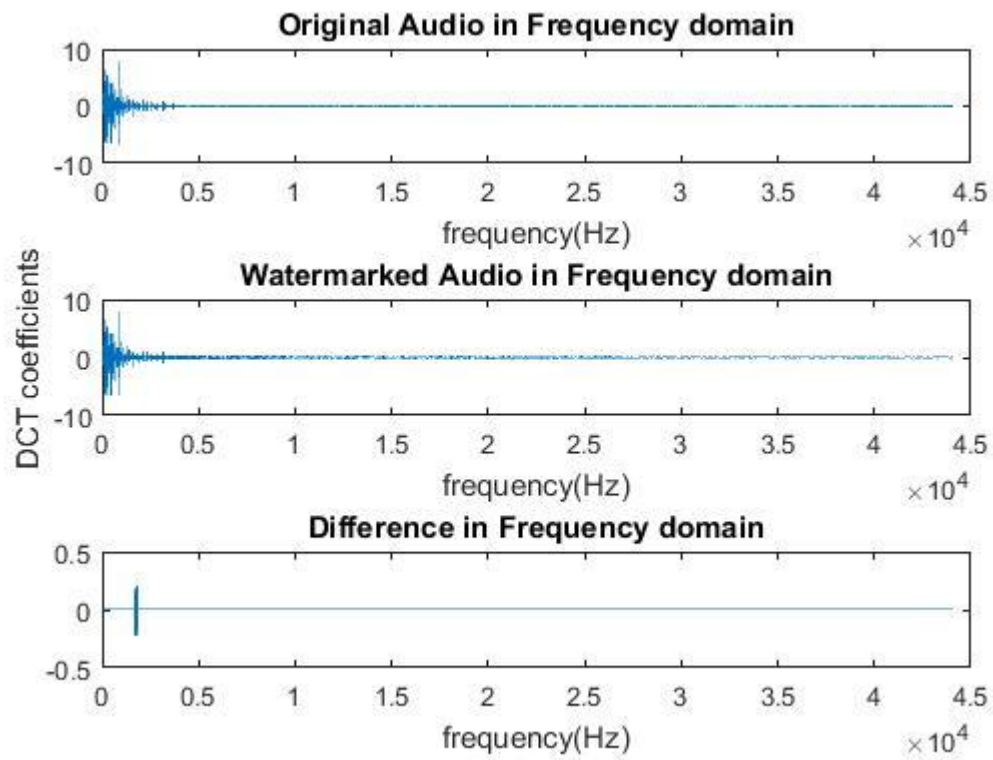


圖 3.3.2-5 浮水印嵌入前後與頻域差異值比較圖

3.3.3. 比例因數的選擇

由於浮水印的解出正確率與語音品質上必須做出取捨，而本論文演算法運用原訊號與基礎序列的關聯性(Correlation)以及浮水印資訊的比較，對於比例因數上做出選擇。為了滿足語音品質與萃取正確率，我們必須先以式 3.3.3-1 計算出原訊號(X)與基礎訊號(S)的關聯性(R)，其中(Z)為位移數，試想若原訊號與基礎序列的關聯性與嵌入的浮水印資訊位元為同號數，表示我們只需要調整微量的嵌入值即可，反之，關聯性與浮水印資訊為異號數，則嵌入浮水印的能量則必須大於原訊號與基礎序列的關聯性，最後藉由逐個序列計算與逐個序列比較完成各層的比例因數設定。此比例因數選擇演算法，技術複雜度較低，可以降低運算成本，提高嵌入演算法效能。關於整個比例因數的演算法步驟可以參考表 3.3.3-1。

$$R_{(i)} = X_i S_i Z^{-i} \quad (3.3.3-1)$$

表.3.3.3-1 比例因數設定演算法

Selection of α
<i>Input: Embedded frequency range of original signal X_i , Basic sequence , watermark bit d</i> <i>Output: α_i</i> <i>Step:</i> 1. <i>Compute</i> $R = X_i S_i Z^{-i}$ 2. <i>If $R < 0, R = R - 1$</i> 3. $\alpha = \text{ceil}(R) / \text{CCC number } (n) * \text{CCC bits } (l) * d_{(i)}$ 4. <i>If $\alpha > 0, \alpha = \text{abs}(\min(\alpha)) / 10$</i> 5. $A = \text{abs}(\alpha)$

3.3.4. 最終演算法比較之調整

在選擇完各層、各序列的比例因數後，將各序列的比例因數乘上後，便可以得到加入比例因數後的新多層捲積序列 W_N' ，如式(3.3.4-2)所示，而單層的家入比例因數捲積序列，如式(3.3.4-1)所示。接著我們將進行最後的調整，以達到與其他演算法相同之語音品質，這個調整利於往後對各演算法效能之比較，最後調整之方程式如式(3.3.4-3)所示，其中 Y 為原訊號經由離散餘弦轉換後之係數、 $\tilde{X}$ 為嵌入完浮水印資訊之訊號，用意是對嵌入完浮水印後的訊號段進行能量的調整，使最後的音訊品質能與其他相比較的演算法相當。

$$W^{(i)} \stackrel{\text{def}}{=} \sum_{t=0}^{B^{(i)}} \alpha_t d_t^{(i)} S^{(i)} Z^{-t} \quad (3.3.4-1)$$

$$W_N' = \sum_{i=0}^{m-1} W^{(i)} \quad (3.3.4-2)$$

$$\tilde{X} = Y + \beta W_N' \quad (3.3.4-3)$$

3.4. 浮水印萃取流程

本節將會對浮水印萃取介紹，浮水印萃取架構如圖 3.4-1 所示，而浮水印萃取的架構大致與浮水印嵌入相似，在萃取時採用與嵌入浮水印相同之互補碼基礎序列，對已嵌入浮水印資訊的音訊計算相關性，並以相關性分析以取得嵌入之浮水印位元，各層依序前述步驟，將可分別解出逐層的浮水印資訊。

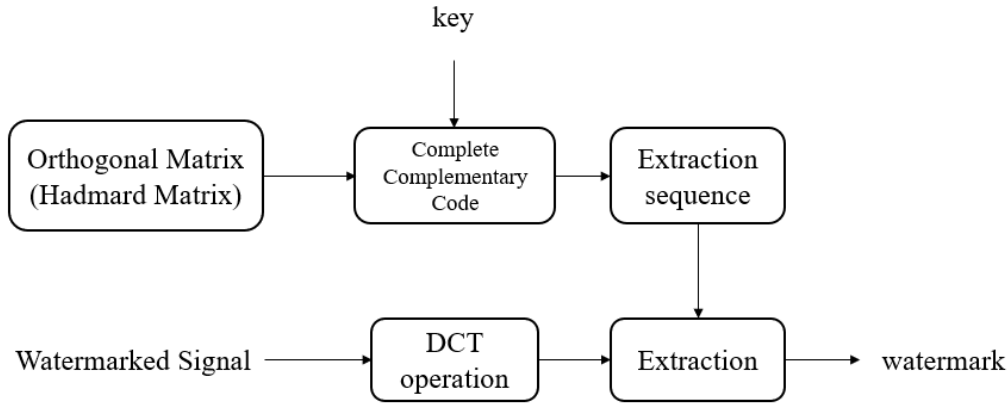


圖 3.4-1 浮水印萃取架構圖

3.4.1. 離散餘弦轉換與萃取頻段選擇

萃取浮水印時與嵌入浮水印相同，都必須將音訊訊號轉換為頻域訊號，首先我們以取樣頻率 F_s 及音訊長度為 L 個取樣點的原始音訊訊號 $x(n)$ ，利用離散餘弦轉換將音訊訊號轉換為頻域，轉換的公式如式 3.4.1-1 所示。

$$X(k) = E(m) \sum_{n=0}^{L-1} y(n) \cos\left(\frac{m\pi(2n+1)}{2L}\right), \quad m = 0, 1, \dots, L-1 \quad (3.4.1-1)$$

$$E(m) = \begin{cases} \frac{1}{\sqrt{L}}, & \text{if } m = 0 \\ \sqrt{\frac{2}{L}}, & \text{if } 1 \leq m < L \end{cases} \quad (3.4.1-2)$$

在轉換完後，根據嵌入時所使用的起始頻段 f_{low} 利用式(3.4.1-3)求出對應之離散餘弦係數位置 M_{low} 。

$$f_{step} = \frac{f_s}{L}$$

$$[M_{low}] = [\text{ceil}(\frac{f_{low}}{f_{step}})]$$

(3.4.1-3)

3.4.2. 浮水印萃取演算法

首先在萃取浮水印前，我們經由與浮水印嵌入時相同的互補碼序列產生方法，產生與嵌入浮水印時相同的偽隨機碼序列，再將各層的偽隨機序列之子碼組合成基礎序列，將基礎序列與嵌入完浮水印的訊號以式 3.4.2-1 計算乘積值，如圖 3.4.2 所示，最後再將乘積數值，以式(3.4.2-2)的訊號函數(Signal function)萃取出嵌入時的浮水印資訊，各層浮水印資訊則以相同的方法能逐一的解出。

$$R_{S^{(i)}\tilde{X}Z^{-j}} = S^{(i)}\tilde{X}Z^{-j} \quad (3.4.2-1)$$

$$d_w^{(i)} = \text{sgn}(R_{S^{(i)}\tilde{X}Z^{-w}}) \quad \text{and} \quad \text{sgn}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x \leq 0 \end{cases} \quad (3.4.2-2)$$

圖 3.4.2 浮水印萃取示意圖

在介紹完浮水印萃取演算法後，我們將證明萃取演算法，在證明解出浮水印演算法之前，將先以圖例說明完全互補碼之間的正交關係，若存在兩組互補碼分別為(++)、(+-++)，並且欲嵌入的浮水印資訊為三位元(+1,+1,+1)，首先我們以式(3.3.2-2)將浮水印嵌入，如圖(3.4.3)下三列的排列嵌入，而在解出時使用乘積萃取，亦可見圖(3.4.3)，若我們想萃取浮水印資訊的第二位元，則我們可以算出各序列的乘積

數值，最後將其相加會得到總和為(4)，將總和經過訊號函數(Signal function)，可以得到嵌入資訊為(+1)即為正確答案。接著探討不同序列的萃取問題，可見圖(3.4.4)，我們以序列(+ - + +)對於前述之嵌入序列做解出，我們以相同的乘積方法萃取，最後得到總和為 0，這也代表著不同序列並不會相互影響，這也就是互補碼之間所存在的完美正交性質。

	value		value	total
	1		-1	0
	2		2	4
	1		-1	0

圖 3.4.3 正相關互補碼萃取範例

	value		value	total
	1		-1	0
	0		0	0
	-1		1	0

圖 3.4.4 互相關互補碼萃取範例

緊接著觀察浮水印萃取的演算法，為了方便討論說明，在不考慮任何的音訊處理之下，令嵌入浮水印後的音訊訊號與第 j 個基礎序列的無位移乘積定義為 D_j ，如式(3.4.2-3)所示，其中 $\widetilde{X}_N$ 為經過與語音處理之訊號。

$$D_j = \widetilde{X}_N S^{(j)} \quad (3.4.2-3)$$

因為未考慮任何音訊處理，故我們可以將式(3.4.2-3)改寫為

$$D_j = \widetilde{X} S^{(j)} \quad (3.4.2-4)$$

經由式(3.4.2-5)，我們可以發現其中存在著原音訊訊號離散餘弦係數、嵌入的浮水印資訊訊號，分別對於基礎序列做乘積，然而這也就說明在解出的部分，不隻會有浮水印資訊訊號，還會有來自於原訊號的影響，這也間接證明了我們最初為何在

嵌入時，預先對於浮水印資訊的部分如章節 3.3.3 選擇適當比例因數。

$$\begin{aligned}
 D_j &= \beta(Y + W_N)S^{(j)} \\
 &= \beta\left(Y + \sum_{i=0}^{m-1} W^{(i)}\right)S^{(j)} \\
 &= \beta\left(Y + \sum_{i=0}^{m-1} \sum_{t=0}^{B^{(i)}} \alpha_t^{(i)} d_t^{(i)} S^{(i)} Z^{-t}\right)S^{(j)}
 \end{aligned}
 \tag{3.4.2-5}$$

接著我們可以將式(3.4.2-5)改寫為式(3.4.2-6)方便觀察

$$\beta(YS^{(j)} + \sum_{i=0}^{m-1} \sum_{t=0}^{B^{(i)}} \alpha_t^{(i)} d_t^{(i)} S^{(i)} S^{(j)} Z^{-t})
 \tag{3.4.2-6}$$

由式(3.4.2-6)可以觀察前項原訊號離散餘弦轉換係數與基礎序列的乘積，此項在比例因數(α)選擇的時候已經計算過，然而當時在選擇比例因數時，也考慮了浮水印資訊的相關性，然而對於後項的基礎序列與捲積序列乘積，可以由前文所述互補碼序列的完美正交性質，得到完美的正交，並且不受其外的基礎序列影響。藉由預先計算的前項乘積、後項互補碼的完美正交性質，在未受到任何音訊處理的影響之下，我們可以使用訊號函數計算出所嵌入的浮水印資訊位元，如式(3.4.2-7)所示，在函數中的數值便是在嵌入浮水印位元時已預先計算的數值，故可以在萃取時正確萃取浮水印資訊，最後解出公式如式(3.4.2-7)所式。

$$\text{Sign}\left(\beta\left(YS^{(j)} + \sum_{i=0}^{m-1} \sum_{t=0}^{B^{(i)}} \alpha_t^{(i)} d_t^{(i)} S^{(i)} S^{(j)} Z^{-t}\right)\right)
 \tag{3.4.2-7}$$

第四章 多層互補碼演算法結果分析及比較

本章節著重於實驗的結果與分析，在章節 4.1 將提到並介紹感知音訊品質評估 (Perceptual Evaluation of Audio Quality, PEAQ)[24]，而章節 4.2 則是說明測試環境數據與係數，最後顯示實驗的結果，章節 4.3 為演算法時間與演算法時間複雜度討論。

4.1. 感知音訊品質評估

當一個音訊需要被浮水印保護時，在嵌入浮水印的階段，也間接地必須破壞原本音訊的品質，這是無可避免的。一個相對較好的浮水印音訊須具備良好的不可感知性，換句話說浮水印音訊在聽者的聽覺感知上是與原音檔差異不大的。為了評估使用者聽覺上的情形，本論文採取國際電信聯盟(International Telecommunication Union, ITU)所提出的感知音訊品質評估(Perceptual Evaluation of Audio Quality, PEAQ)[24]做為本論文與其他演算法之客觀數據評估(Objective difference grade, ODG)。

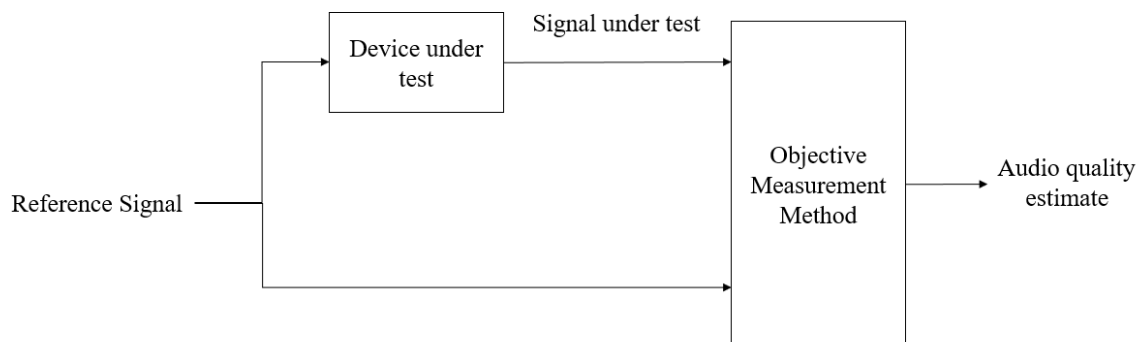


圖 4.1-1 感知音訊品質評估概念示意圖

感知音訊品質評估的估測概念如圖 4.1-1 所示，其概念即是先以一段參考訊號 (Reference Signal) 送進測試裝置 (Device under test)，其中參考訊號也就是本論文所提及的原因檔，而經過測試裝置後之音檔即為嵌入完浮水印之音訊，我們將這兩組音訊訊號經過客觀數據評估，便可以得到一個數值用以評斷音訊品質的優劣，此數值也就是客觀數據評估分數。客觀數據評估分數介於 0 到 -4 之間，分數越接近 -4 即表示音訊品質相對越差，反之亦然，而客觀數據評估分數的對照表可以參考表 4.1-1

表.4.1-1 客觀數據評估分數(ODG)對照表

分數	音訊品質
0	感覺不到失真
-1	能感覺到失真，但不惱人
-2	能感覺到失真，略為惱人
-3	能感覺到失真，惱人
-4	能感覺到失真，非常惱人

4.2. 浮水印強健性測試

4.2.1. 強健性測試環境

本論文隨機選取十種不同曲風的測試音檔作為本實驗的測試樣本，音檔如下圖所示。

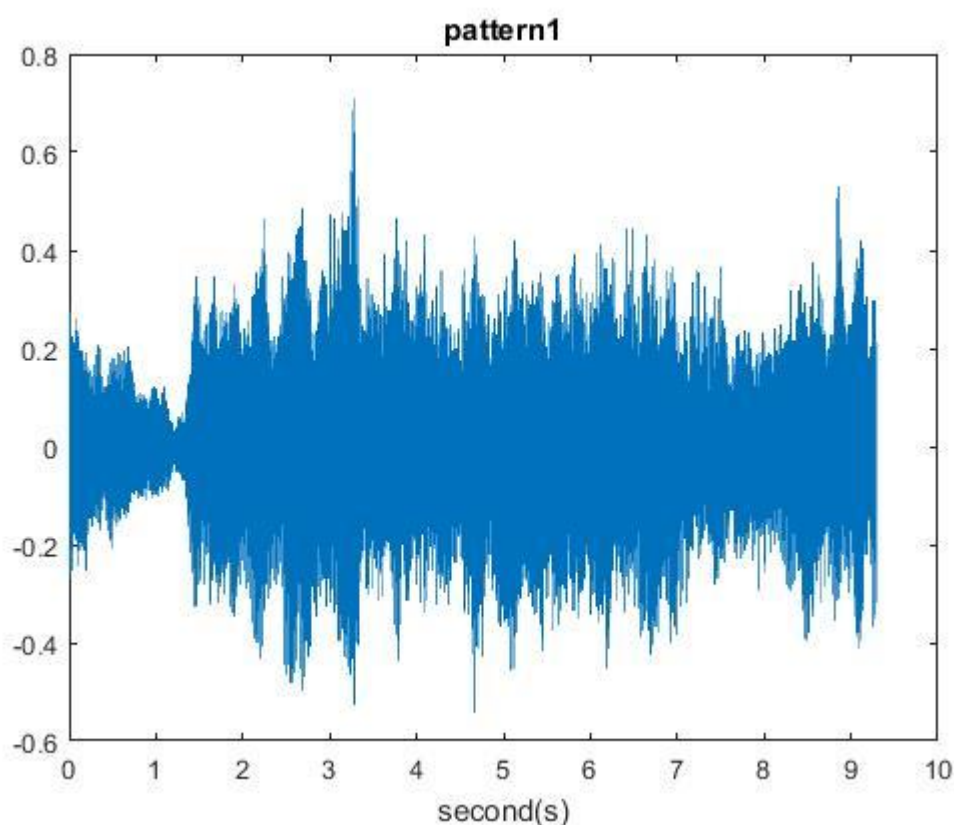


圖 4.2.1-1 強健性測試樣本一

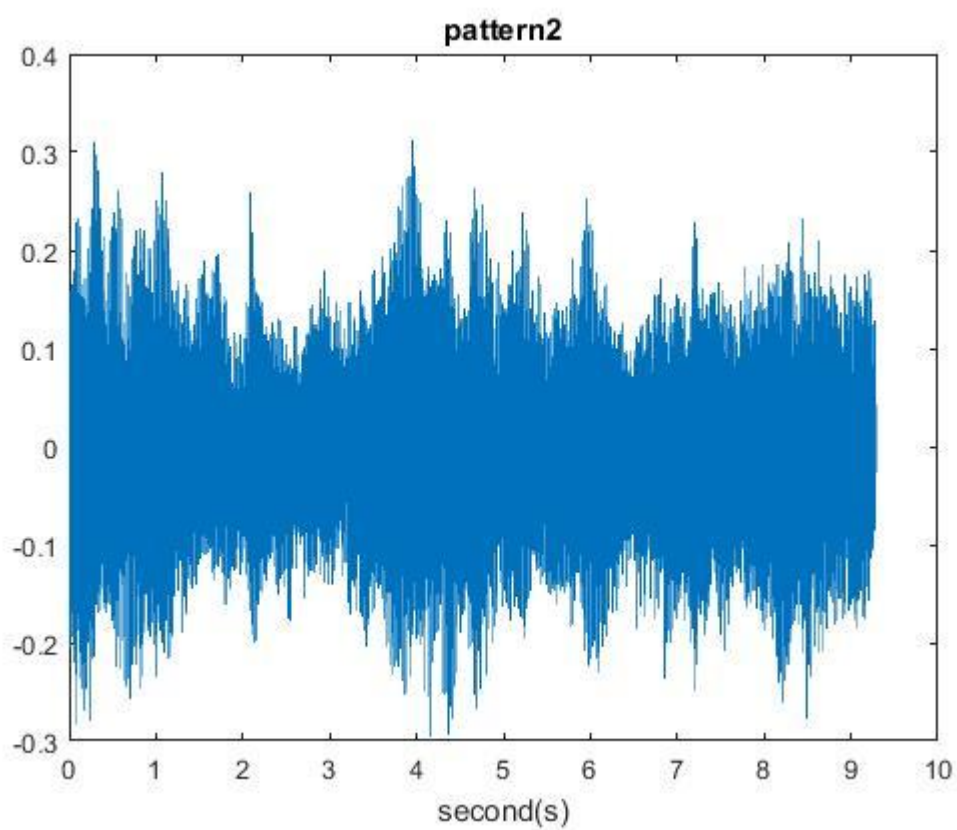


圖 4.2.1-2 強健性測試樣本二

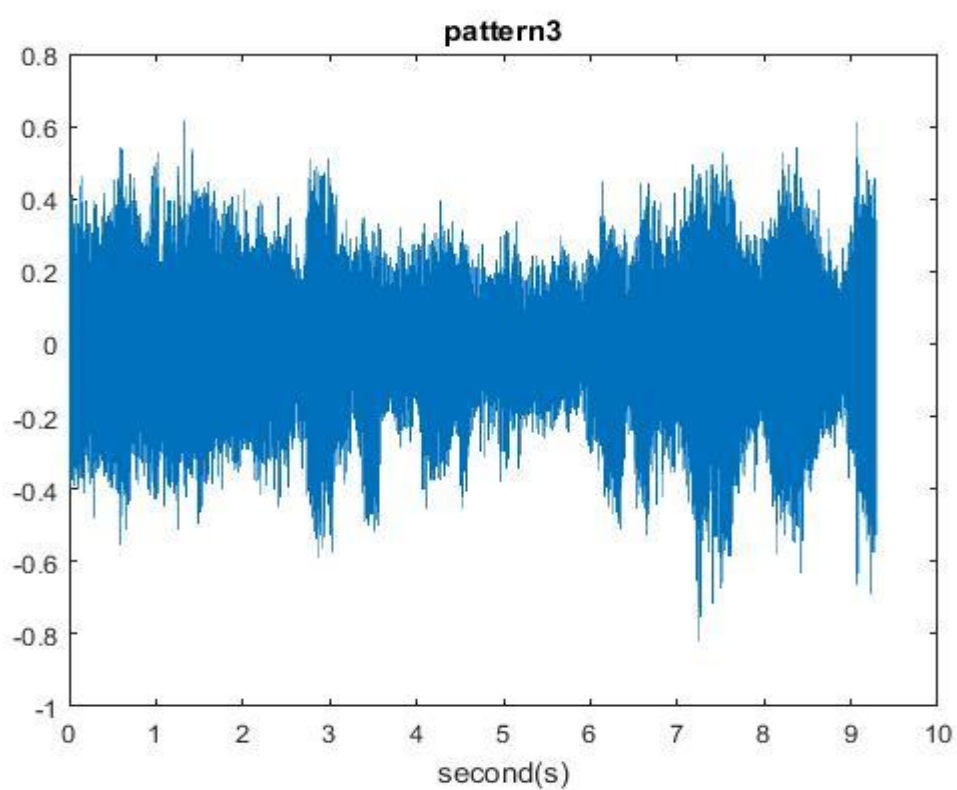


圖 4.2.1-3 強健性測試樣本三

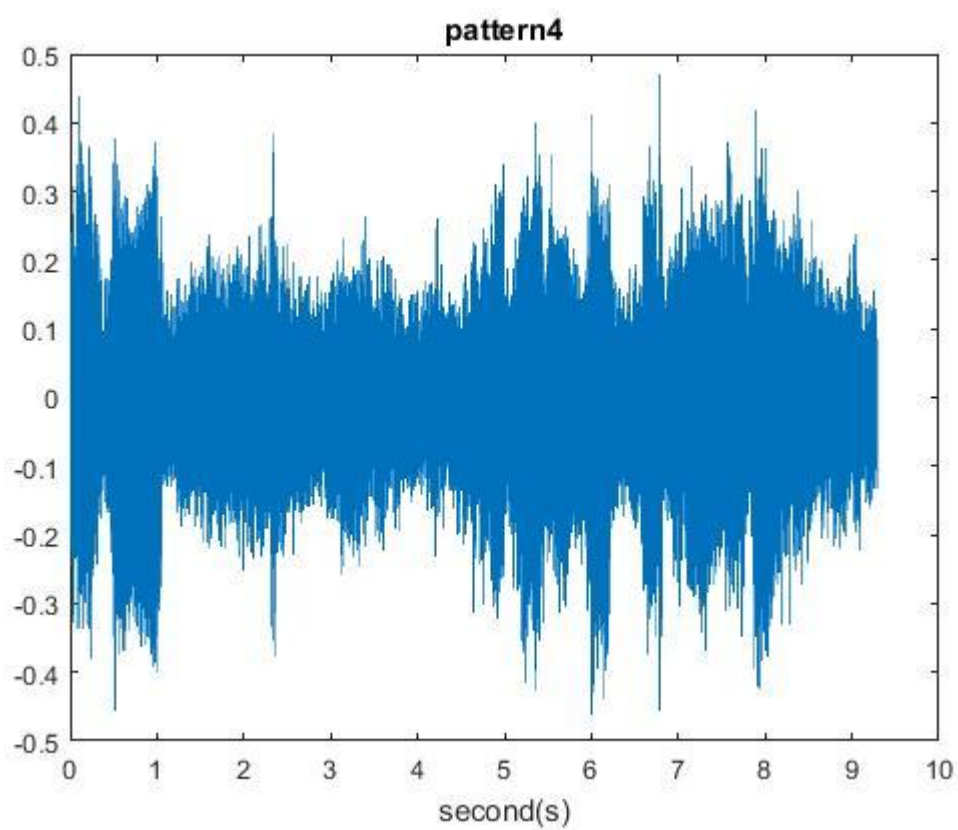


圖 4.2.1-4 強健性測試樣本四

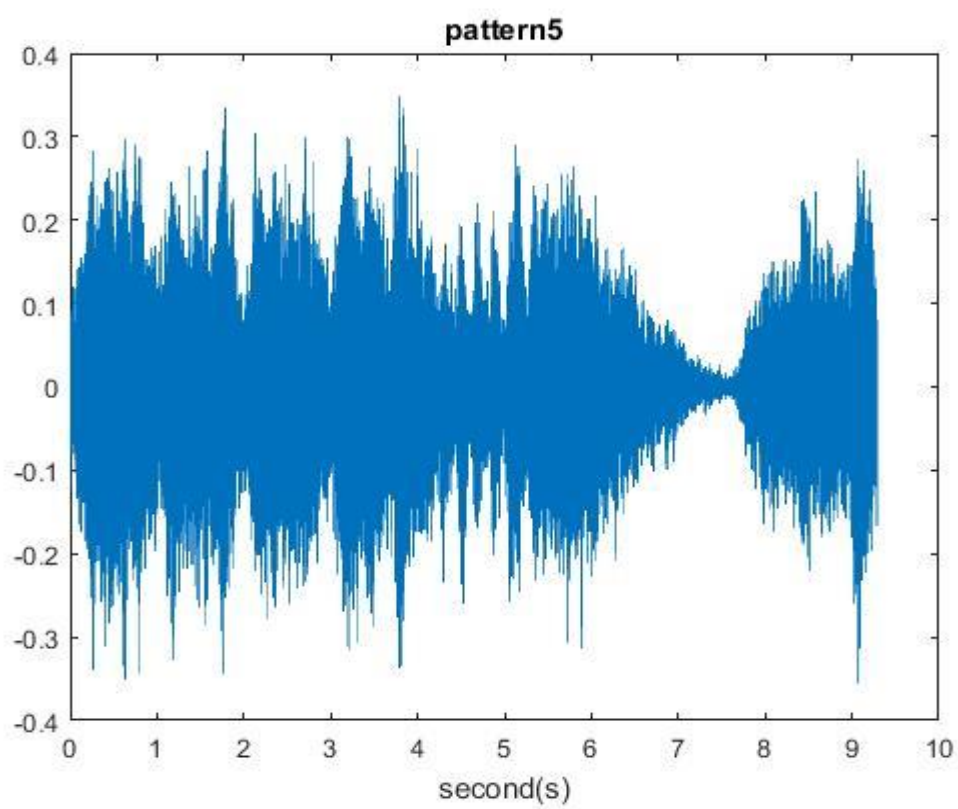


圖 4.2.1-5 強健性測試樣本五

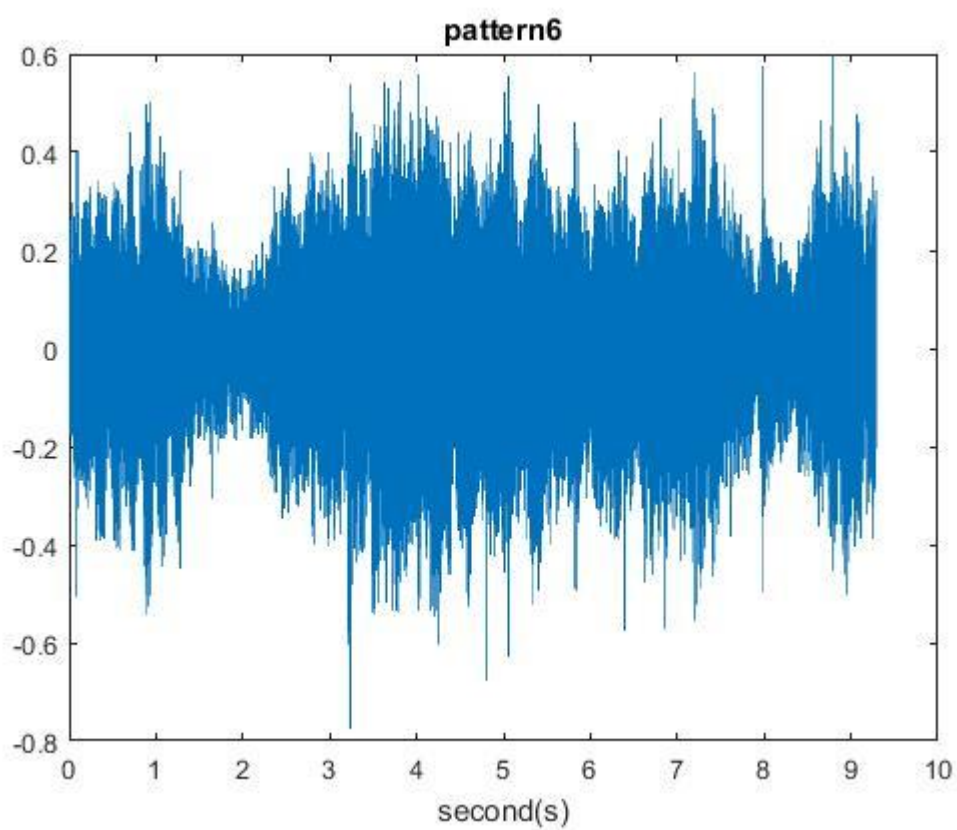


圖 4.2.1-6 強健性測試樣本六

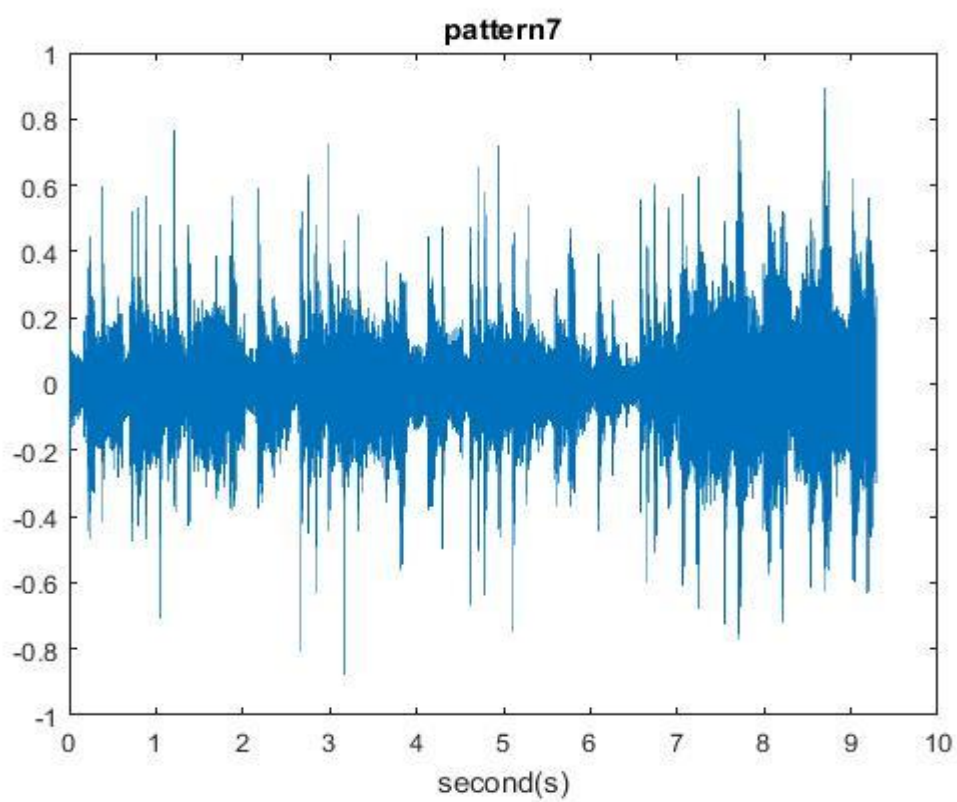


圖 4.2.1-7 強健性測試樣本七

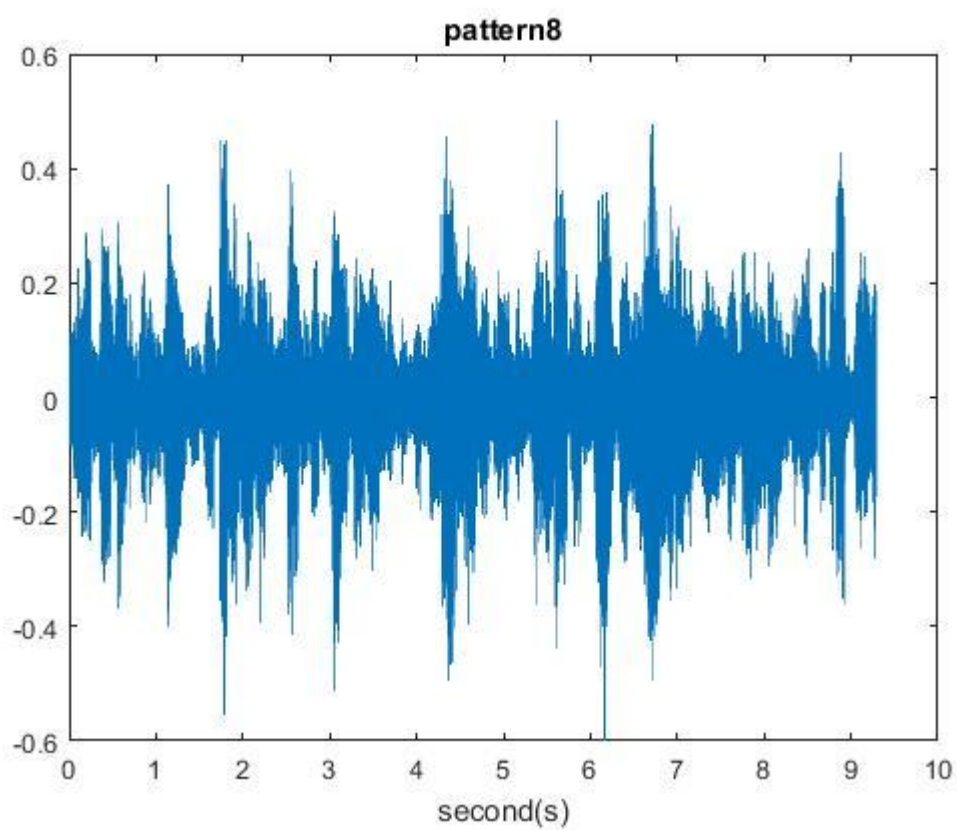


圖 4.2.1-8 強健性測試樣本八

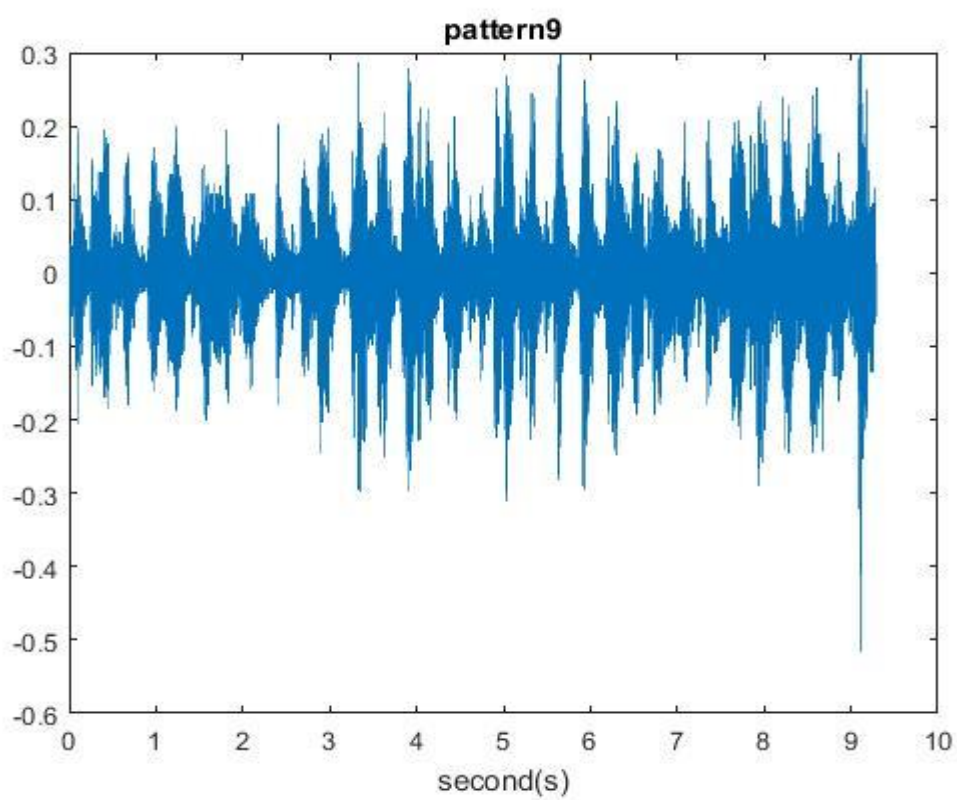


圖 4.2.1-9 強健性測試樣本九

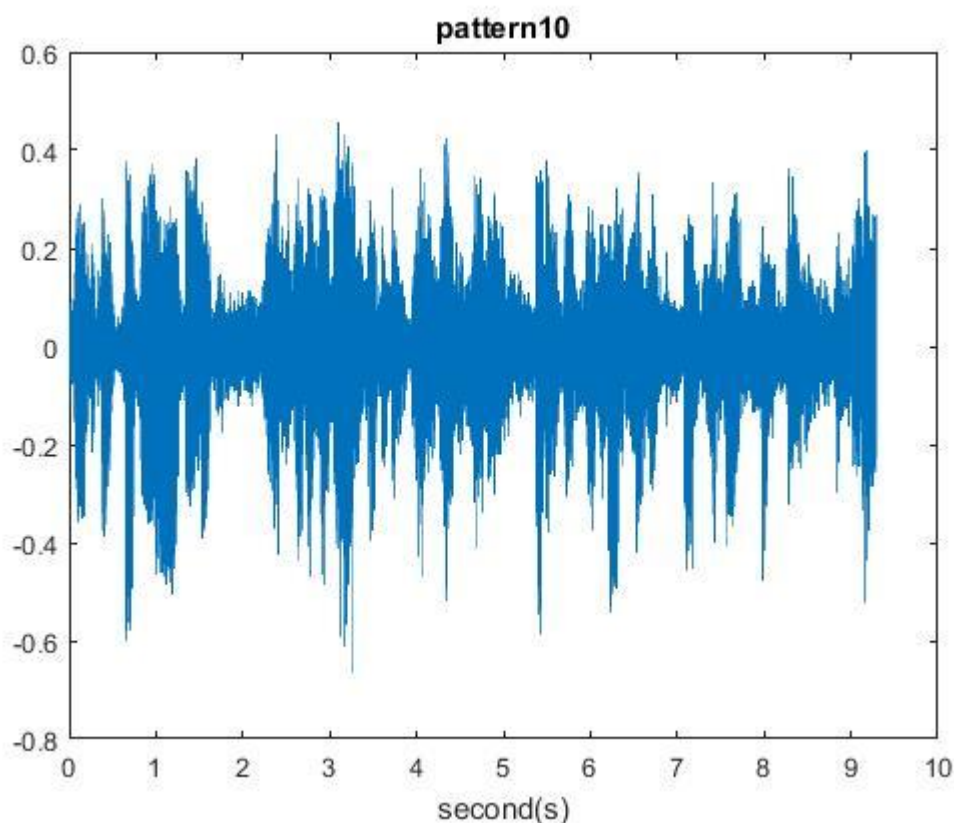


圖 4.2.1-10 強健性測試樣本十

嵌入完浮水印的音訊有可能被有意或無意地攻擊，這點由使用者的操作決定，然而就此方面的影響，本論文將使用表 4.2.1-1 的音訊處理攻擊對測試音檔進行測試，最後再將經過音訊處理後的音檔萃取出浮水印資訊，最後經由是(4.2.1-1)計算出浮水印萃取的位元錯誤率(Bit Error Ratio, BER)。

表.4.2.1-1 音訊處理攻擊方式一覽表

音訊處理方式(攻擊)	處理方式	實現方式
閉迴路 (Closed-Loop)	嵌入浮水印資訊後，不對音訊訊號做任何處理	利用 Matlab 函示庫中的 <code>awgn(signal, dB, 'measured')</code> 函數調整高斯白雜訊 dB 值。
加入高斯白雜訊 (AWGN)	加入 0dB 高斯白雜訊	
	加入 5dB 高斯白雜訊	
	加入 10dB 高斯白雜訊	
	加入 15dB 高斯白雜訊	

重新量化 (Re-quantization)	將音訊降至 8-bits 再升回至 16-bits	利用 Matlab 的 audiowrite 函示調整其中的 'BitsPerSample'。
調幅 (Amplitude)	將音訊訊號震幅調整為 1.2 倍	利用 Matlab 運算將音訊乘相對的比例。
	將音訊訊號震幅調整為 1.8 倍	
高通濾波器 (High-pass filter)	將音訊訊號經過六階截止頻率為 100Hz 的 Butterworth 高通濾波器，如圖 4.2.1-11	利用 Matlab 的函示 butter 建立高通濾波器，並且將音訊訊號經過函式 filtfilt。
低通濾波器 (Low-pass filter)	將音訊訊號經過六階截止頻率為 8000Hz 的 Butterworth 低通濾波器，如圖 4.2.1-12	利用 Matlab 的函示 butter 建立低通濾波器，並且將音訊訊號經過函式 filtfilt。
AAC 壓縮 (AAC compression)	將音訊訊號經過 AAC(128kps)壓縮	利用 Matlab 函示 audiowrite 的 AAC 壓縮並設定 'BitsRate' 的相對數值
	將音訊訊號經過 AAC(96kps)壓縮	

$$BER(\%) = \frac{\text{萃取後浮水印位元的錯誤數量}}{\text{總浮水印位元的嵌入數量}}$$

(4.2.1-1)

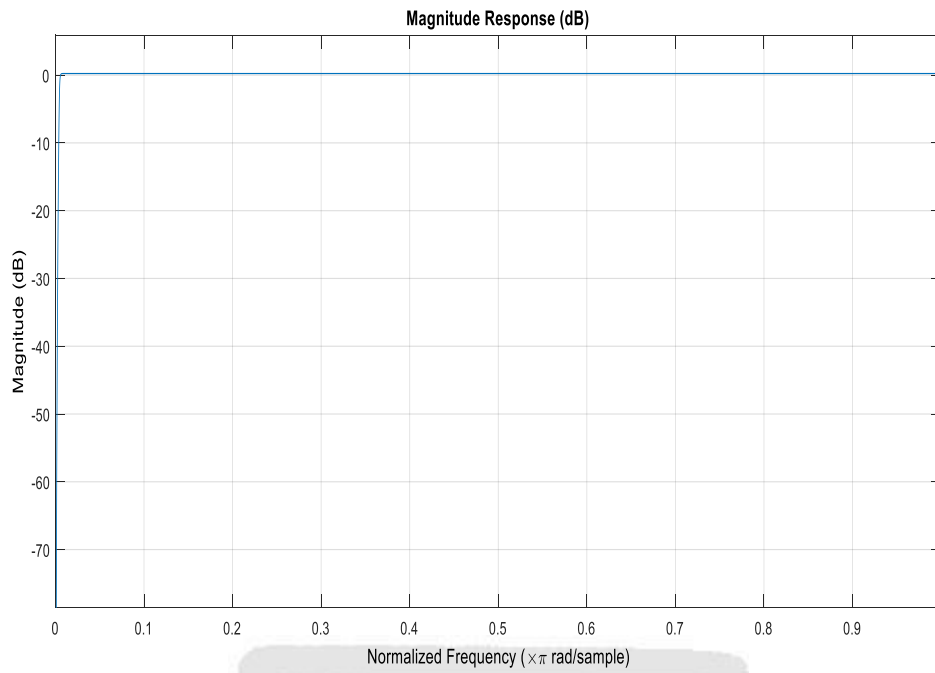


圖.4.2.1-11 六階 Butterworth 高通濾波器($f_c = 100\text{Hz}$)

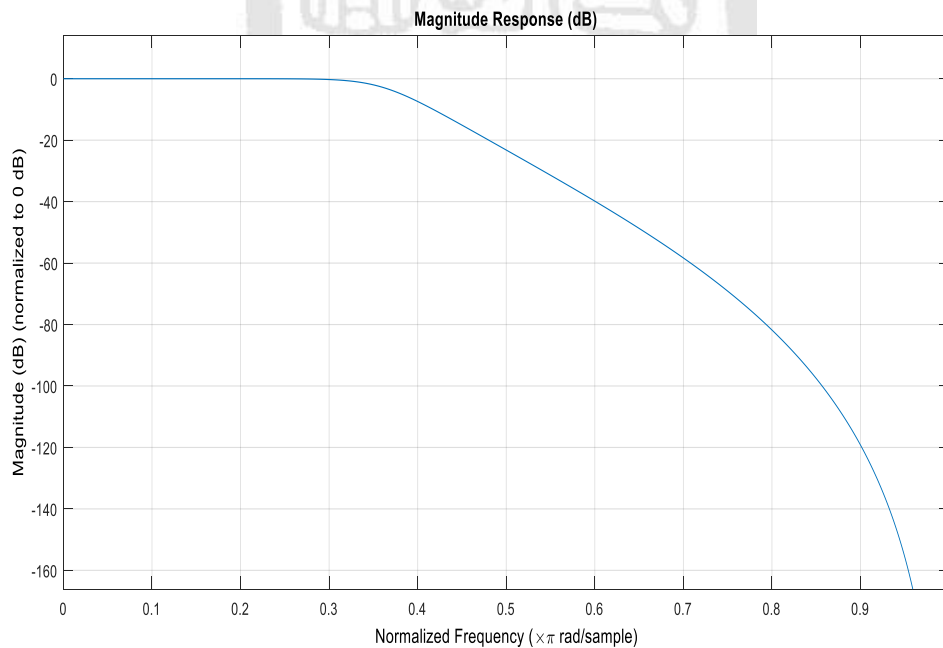


圖.4.2.1-12 六階 Butterworth 低通濾波器($f_c = 8000\text{Hz}$)

接下來則是比較個演算法的部分，而我們對於演算法的音訊品質基準，將會定為客觀數據評估 $\text{ODG} \cong -0.7$ 。在浮水印位元嵌入數目上，論文[13,14,15]三種演算法皆為嵌入 378 位元數，然而由於本演算法為多層嵌入，並且為 4 層的層數，故本演算法對於浮水印位元之嵌入數目為提升 416 位元數，對於各層之嵌入數目皆為 104 位元數。

而起始頻段選擇部分本演算法提出以不同的起始頻率能達到更好的效果，故起始頻率也會與另外三種演算法有所差異。強健性測試的環境數據可見表 4.2-2，令所提及的演算法差異可見表 4.2-3。

表.4.2-2 強健性測試環境數據表

測試環境數據	數值
客觀數據評估 (Objective difference grade, ODG)	約為-0.7
原音檔取樣頻率 (Sampling frequency)	44100Hz
原音檔解析度 (Resolution)	16 Bit
原始音檔格式	.wav
運算所使用之軟體	Matlab 2017b

表.4.2-3 強健性測試演算法差異表

測試環境數據	論文[13,14,15]		本論文演算法	
嵌入頻段範圍	f_{low}	4000Hz	起始頻段 f_{begin} 為 1000Hz 至 2000Hz 中每 100Hz 間距 ODG 數據相對最低之頻段	
	f_{Hign}	7000Hz		
嵌入總位元數	378 Bits		第一層	104 Bits
			第二層	104 Bits
			第三層	104 Bits
			第四層	104 Bits
產生的偽隨機碼序列個數 (N_p)	128 個		4 個	

4.2.2. 實驗結果比較與討論

本小節將會以章節 4.2 的測試音訊經由本實驗所提出的演算法以及[13,14,15]，並記錄每一個音檔進行強健性測試的數據結果，最後我們將以實驗數據比較，並且針對演算法度及演算法內容做進一步討論。經過強健性測試的實驗結果如下：

表.4.2.2-1 樣本一測試實驗數據

測試樣本名稱		樣本一			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.42		
	α_{lower}		0.25		
	c		0.02		
	β_{min}	0.45		0.01	
	β_{max}	15		15	
	γ_1	0.5		0.01	
	γ_2	1.5		0.012	
	$\Delta\beta$	0.2		0.01	
	β				4
	Steating frequency				1200
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-2 樣本一測試結果

測試樣本名稱	樣本一			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	0	0	0
Noise(15dB)	4.7619	0	0	0
Noise(10dB)	17.1958	0	0	0
Noise(5dB)	27.7778	1.3228	2.9101	0
Noise(0dB)	46.0318	23.2804	23.5450	1.2019
Amplitude (1.2)	0	0	0	0
Amplitude (1.8)	0	0	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	0	0	0
High-pass Filter $f_c = 100Hz$	8.2011	0	0	0
AAC (128kbps)	0	0	0	0
AAC (96kbps)	0	0	0	0

表.4.2.2-3 樣本二測試實驗數據

測試樣本名稱		樣本二			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.03		
	α_{lower}		0.15		
	c		0.005		
	β_{min}	0.6		0.01	
	β_{max}	40		30	
	γ_1	0.5		0.001	
	γ_2	1.5		0.005	
	$\Delta\beta$	0.1		0.005	
	β				3.1
	Steating frequency				1000
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-4 樣本二測試結果

測試樣本名稱	樣本二			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	0	0	0
Noise(15dB)	6.3492	0	0	0
Noise(10dB)	33.5979	1.3228	0	0
Noise(5dB)	44.9735	21.1640	17.1958	0
Noise(0dB)	48.4127	42.5926	36.2434	0.4808
Amplitude (1.2)	0	0	0	0
Amplitude (1.8)	0	0	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	0	0	0
High-pass Filter $f_c = 100Hz$	2.9101	0	0	0
AAC (128kbps)	0	0	0	0
AAC (96kbps)	0	0	0	0

表.4.2.2-5 樣本三測試實驗數據

測試樣本名稱		樣本三			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.98		
	α_{lower}		0.4		
	c		0.01		
	β_{min}	0.48		0.02	
	β_{max}	30		30	
	γ_1	0.1		0.01	
	γ_2	0.5		0.03	
	$\Delta\beta$	0.1		0.025	
	β				4.6
	Steating frequency				2000
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-6 樣本三測試結果

測試樣本名稱	樣本三			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	0	0	0
Noise(15dB)	0	0	0	0
Noise(10dB)	0	0	0	0
Noise(5dB)	6.0847	0	0	0
Noise(0dB)	20.1016	0	0	0.9615
Amplitude (1.2)	0	0	0	0
Amplitude (1.8)	0	0	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	0	0	0
High-pass Filter $f_c = 100Hz$	7.1429	0	0	0
AAC (128kbps)	0	0	0	0
AAC (96kbps)	0	0	0	0

表.4.2.2-7 樣本四測試實驗數據

測試樣本名稱		樣本四			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.65		
	α_{lower}		0.1		
	c		0.01		
	β_{min}	0.4		0.01	
	β_{max}	25		30	
	γ_1	0.1		0.01	
	γ_2	0.5		0.018	
	$\Delta\beta$	0.1		0.005	
	β				4.6
	Steating frequency				1800
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-8 樣本四測試結果

測試樣本名稱	樣本四			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	0	0	0
Noise(15dB)	0	0	0	0
Noise(10dB)	0	0	0	0
Noise(5dB)	7.1429	0	0	0
Noise(0dB)	26.4550	2.9100	0.7937	0.7212
Amplitude (1.2)	39.6825	0	0	0
Amplitude (1.8)	0	0	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	0	0	0
High-pass Filter $f_c = 100Hz$	0	0	0	0
AAC (128kbps)	0	0	0	0
AAC (96kbps)	0	0	0	0

表.4.2.2-9 樣本五測試實驗數據

測試樣本名稱		樣本五			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.27		
	α_{lower}		0.15		
	c		0.00065		
	β_{min}	0.05		0.001	
	β_{max}	25		10	
	γ_1	0.05		0.002	
	γ_2	0.2		0.005	
	$\Delta\beta$	0.05		0.002	
	β				1.95
	Steating frequency				1900
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-10 樣本五測試結果

測試樣本名稱	樣本五			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	7.6720	2.6455	0
Noise(15dB)	7.6720	7.6720	3.7037	0
Noise(10dB)	19.0476	7.6720	3.9683	0
Noise(5dB)	22.4868	11.3757	9.2593	2.1635
Noise(0dB)	36.2434	15.6085	19.8413	9.1346
Amplitude (1.2)	0	7.6720	2.6455	0
Amplitude (1.8)	0	7.6720	2.6455	0
Re-quantization	0	8.2011	2.6455	0
Low-pass Filter $f_c = 8000Hz$	0	7.6720	2.6455	0
High-pass Filter $f_c = 100Hz$	8.2011	7.6720	2.6455	0
AAC (128kbps)	8.2011	8.2011	3.7037	0
AAC (96kbps)	12.1693	8.2011	3.7037	0

表.4.2.2-11 樣本六測試實驗數據

測試樣本名稱		樣本六			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.8		
	α_{lower}		0.12		
	c		0.01		
	β_{min}	0.02		0.012	
	β_{max}	25		17	
	γ_1	0.6		0.04	
	γ_2	1.2		0.02	
	$\Delta\beta$	0.01		0.006	
	β				5.8
	Steating frequency				1000
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-12 樣本六測試結果

測試樣本名稱	樣本六			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	1.3228	0	0
Noise(15dB)	0	1.3228	0	0
Noise(10dB)	3.7037	1.3228	0	0
Noise(5dB)	10.8466	2.3810	2.3810	0
Noise(0dB)	26.5677	7.1429	3.9683	0.2404
Amplitude (1.2)	0	1.3228	0	0
Amplitude (1.8)	0	1.3228	0	0
Re-quantization	0	1.3228	0	0
Low-pass Filter $f_c = 8000Hz$	0	1.3228	0	0
High-pass Filter $f_c = 100Hz$	7.6196	1.3228	0	0
AAC (128kbps)	0	1.3228	0	0
AAC (96kbps)	0	1.3228	0	0

表.4.2.2-13 樣本七測試實驗數據

測試樣本名稱		樣本七			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.52		
	α_{lower}		0.12		
	c		0.07		
	β_{min}	0.02		0.001	
	β_{max}	10		10	
	γ_1	0.5		0.002	
	γ_2	1		0.004	
	$\Delta\beta$	0.005		0.005	
	β				3.4
	Steating frequency				1100
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-14 樣本七測試結果

測試樣本名稱	樣本七			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	1.3228	0	0
Noise(15dB)	2.1164	2.9101	0	0
Noise(10dB)	24.3386	9.2593	5.8201	0
Noise(5dB)	46.5601	24.0741	21.1640	0
Noise(0dB)	48.9418	36.7725	42.0635	1.4423
Amplitude (1.2)	0	1.3228	0	0
Amplitude (1.8)	0	2.9101	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	1.3228	0	0
High-pass Filter $f_c = 100Hz$	11.1111	1.3228	0	0
AAC (128kbps)	0	1.3228	0	0
AAC (96kbps)	0	0	0	0

表.4.2.2-15 樣本八測試實驗數據

測試樣本名稱		樣本八			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		1		
	α_{lower}		0.18		
	c		0.065		
	β_{min}	0.02		0.001	
	β_{max}	20		20	
	γ_1	0.6		0.005	
	γ_2	1		0.01	
	$\Delta\beta$	0.01		0.006	
	β				2.95
	Steating frequency				1200
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-16 樣本八測試結果

測試樣本名稱	樣本八			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	0	0	0
Noise(15dB)	0	0	0	0
Noise(10dB)	0.7967	0	0	0
Noise(5dB)	8.2011	0.7937	0	0
Noise(0dB)	22.7513	16.4021	3.4392	0.9615
Amplitude (1.2)	0	0	0	0
Amplitude (1.8)	0	0	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	0	0	0
High-pass Filter $f_c = 100Hz$	6.61376	0	0	0
AAC (128kbps)	0	0	0	0
AAC (96kbps)	0	0	0	0

表.4.2.2-17 樣本九測試實驗數據

測試樣本名稱		樣本九			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.17		
	α_{lower}		0.05		
	c		0.01		
	β_{min}	0.003		0.001	
	β_{max}	30		10	
	γ_1	0.4		0.0015	
	γ_2	0.8		0.0035	
	$\Delta\beta$	0.01		0.003	
	β				2
	Steating frequency				2000
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-18 樣本九測試結果

測試樣本名稱	樣本九			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	6.6138	0	0
Noise(15dB)	0	6.0847	0	0
Noise(10dB)	3.7037	5.8201	0	0.4808
Noise(5dB)	25.6614	12.9630	6.0847	3.1250
Noise(0dB)	41.7990	26.4550	22.4868	9.8558
Amplitude (1.2)	0	6.6138	0	0
Amplitude (1.8)	0	6.6138	0	0
Re-quantization	0	6.6138	0	0
Low-pass Filter $f_c = 8000Hz$	0	6.6138	0	0
High-pass Filter $f_c = 100Hz$	4.7619	6.6138	0	0
AAC (128kbps)	0	6.6138	0	0
AAC (96kbps)	0	4.7619	0.5291	0

表.4.2.2-19 樣本十測試實驗數據

測試樣本名稱		樣本十			
演算法		Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
實驗 數據	α_{upper}		0.58		
	α_{lower}		0.15		
	c		0.02		
	β_{min}	0.001		0.001	
	β_{max}	12		15	
	γ_1	0.5		0.005	
	γ_2	1.4		0.011	
	$\Delta\beta$	0.02		0.006	
	β				2.95
	Steating frequency				1700
	Shift index (T)				168
	Row index (m)				8
	Column index(n)				8

表.4.2.2-20 樣本十測試結果

測試樣本名稱	樣本十			
演算法	Circular Shift [13]	Gram-Schmidt [14]	Complete Complementary Code [15]	Proposed
測試攻擊	錯誤率(%)			
Closed-Loop	0	0	0	0
Noise(15dB)	0	0	0	0
Noise(10dB)	0	0	0	0
Noise(5dB)	0.7937	0.7937	0	0.4808
Noise(0dB)	12.6984	5.8201	1.0582	3.6058
Amplitude (1.2)	0	0	0	0
Amplitude (1.8)	0	0	0	0
Re-quantization	0	0	0	0
Low-pass Filter $f_c = 8000Hz$	0	0	0	0
High-pass Filter $f_c = 100Hz$	6.0847	0	0	0
AAC (128kbps)	0	3.4392	0	0
AAC (96kbps)	0	2.1164	0	0

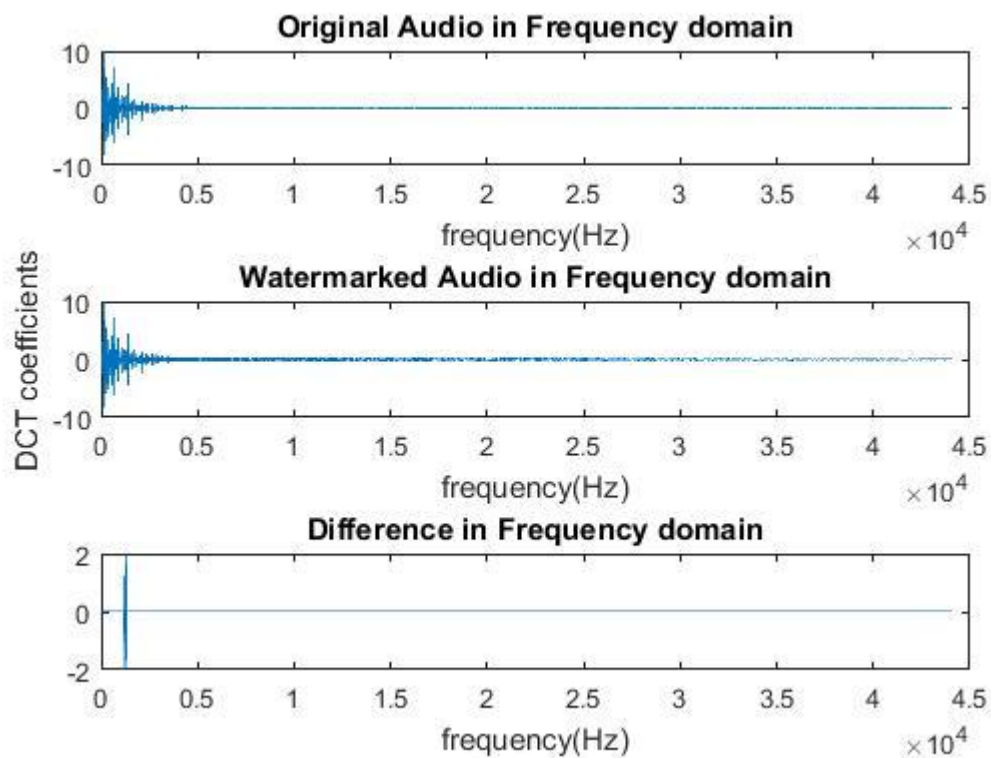


圖 4.2.2-1 樣本一嵌入浮水印頻率前後差值比較圖

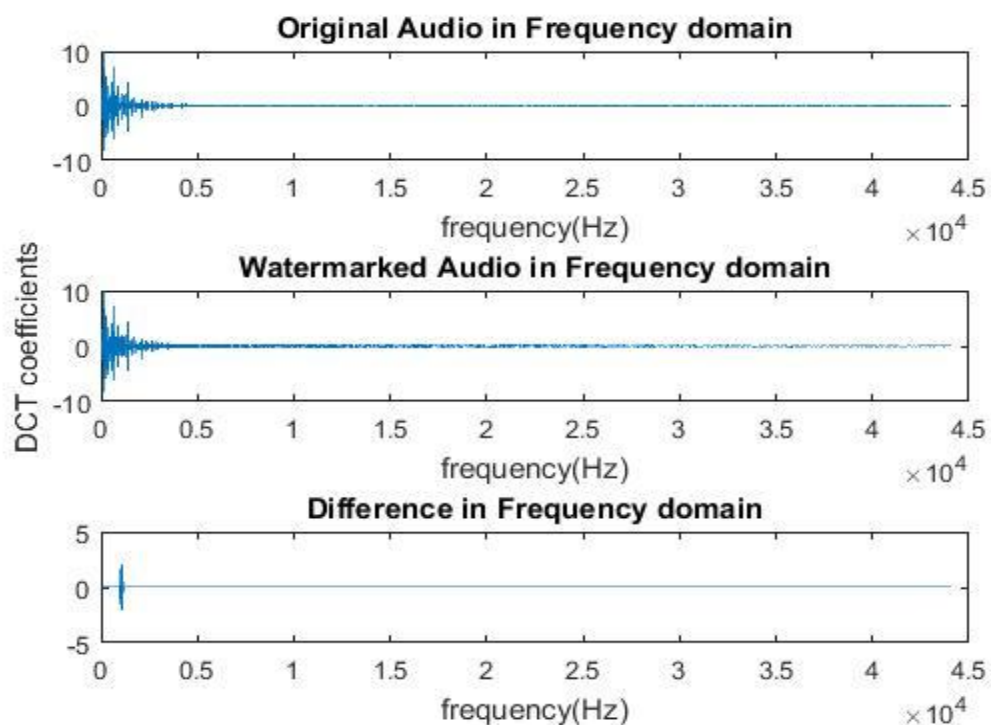


圖 4.2.2-2 樣本二嵌入浮水印頻率前後差值比較圖

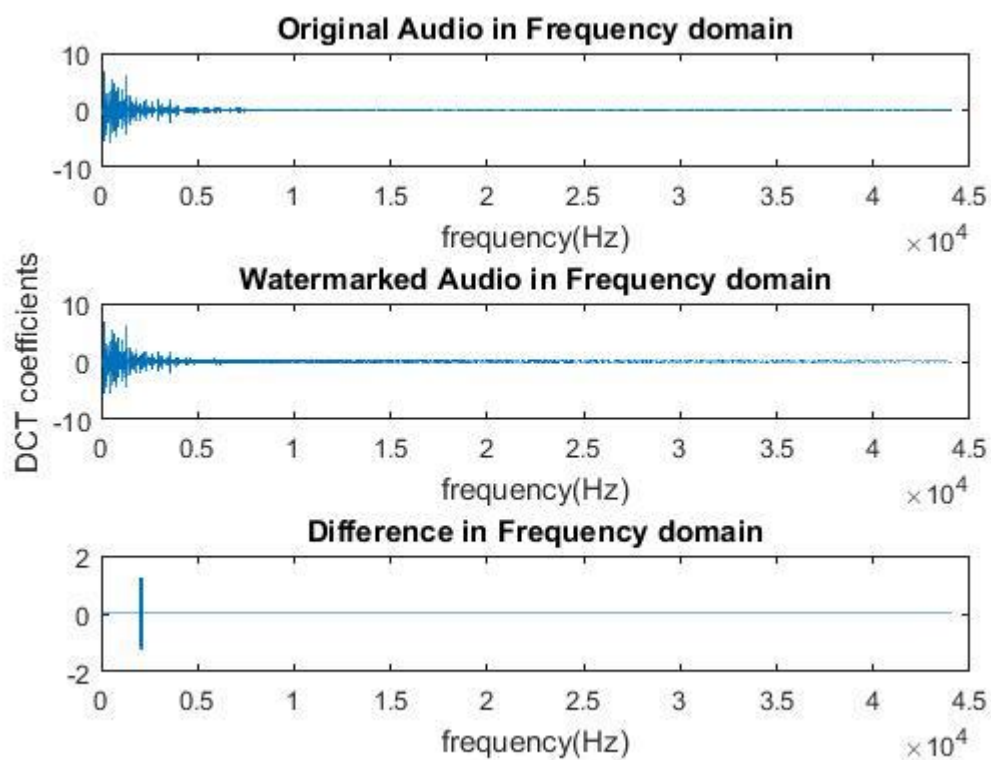


圖 4.2.2-3 樣本三嵌入浮水印頻率前後差值比較圖

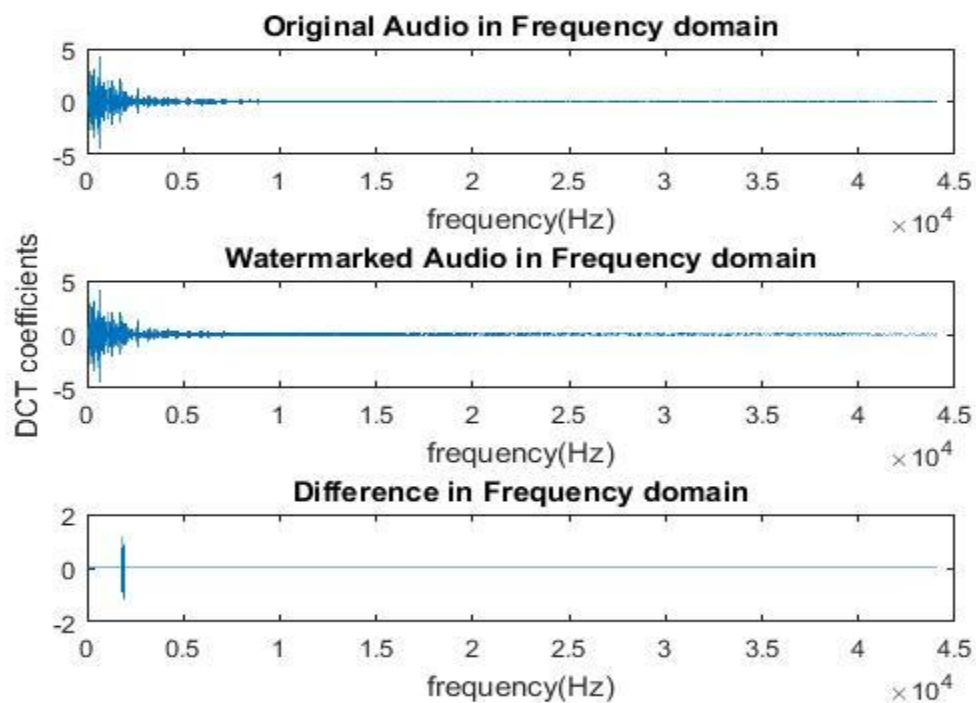


圖 4.2.2-4 樣本四嵌入浮水印頻率前後差值比較圖

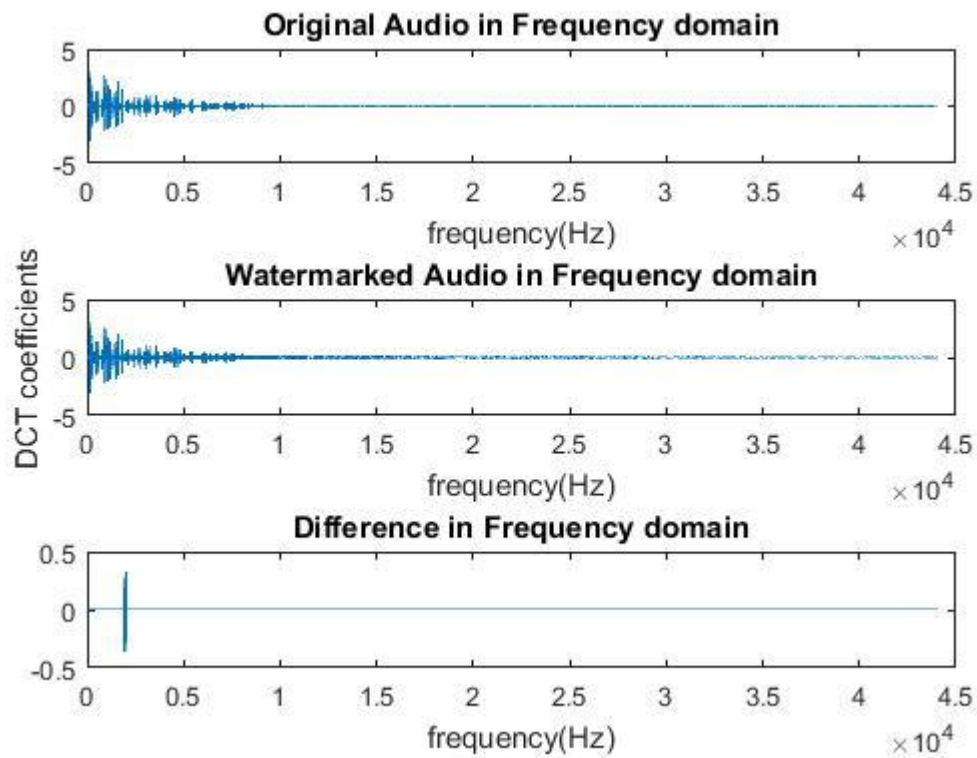


圖 4.2.2-5 樣本五嵌入浮水印頻率前後差值比較圖

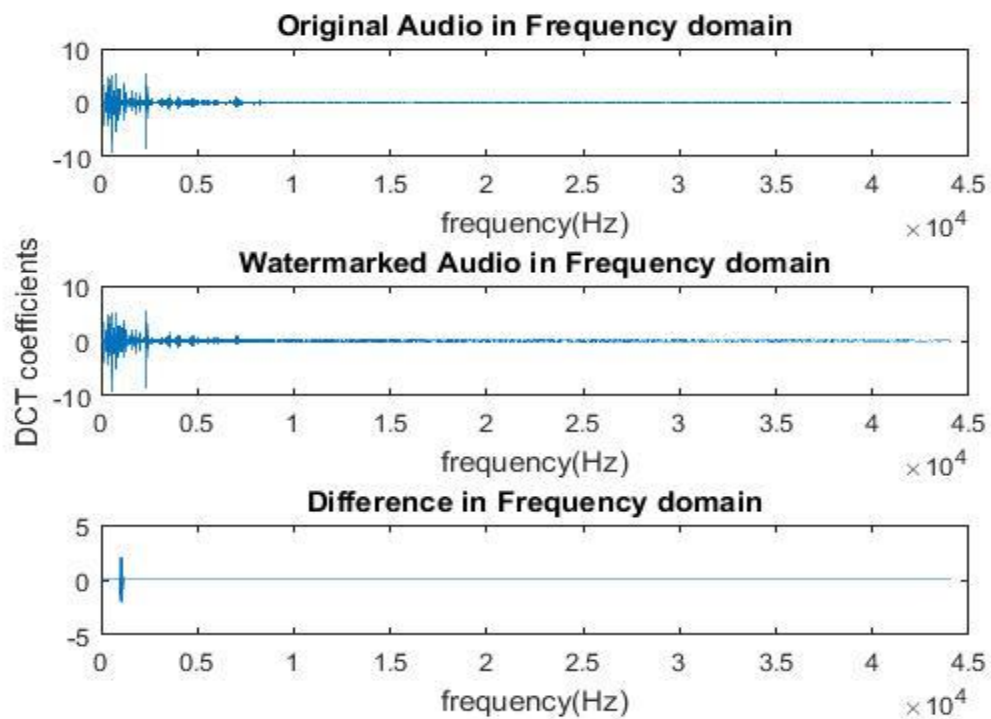


圖 4.2.2-6 樣本六嵌入浮水印頻率前後差值比較圖

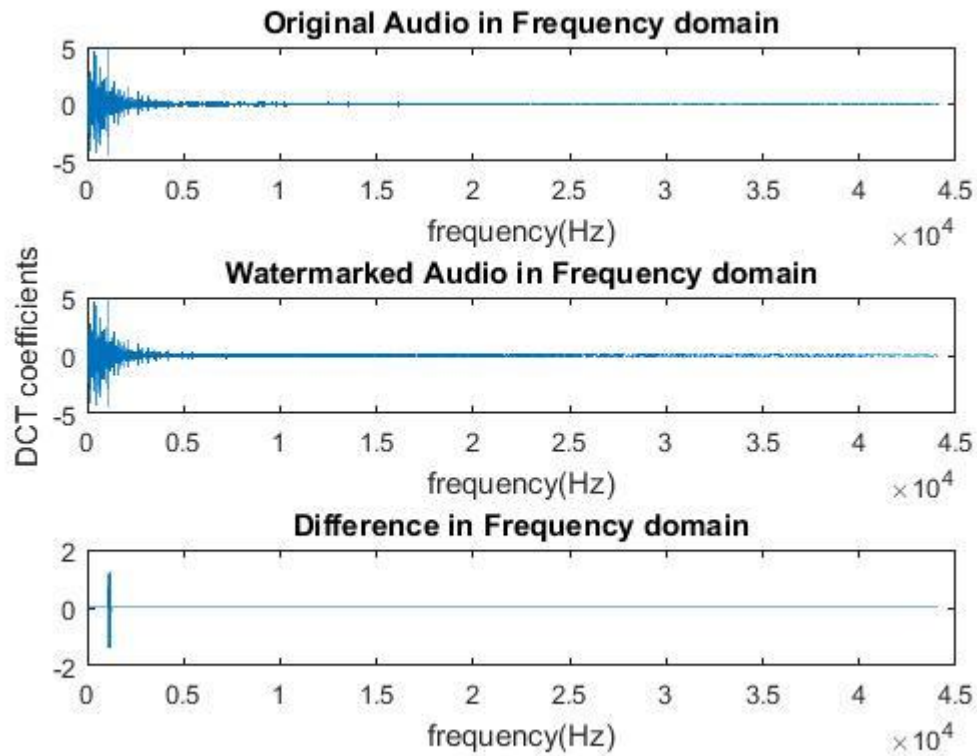


圖 4.2.2-7 樣本七嵌入浮水印頻率前後差值比較圖

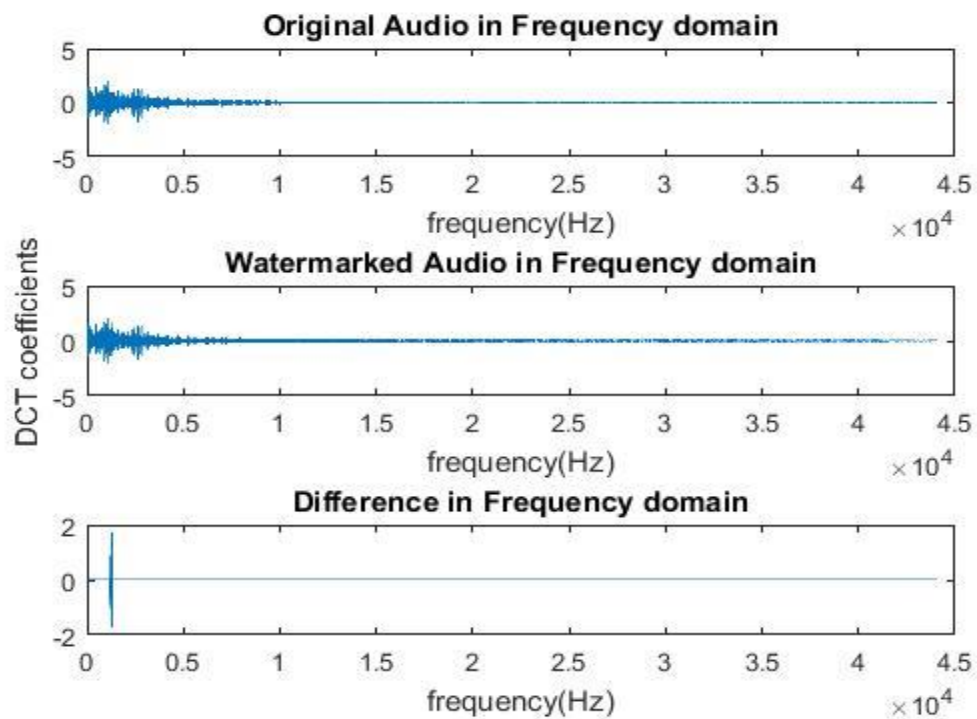


圖 4.2.2-8 樣本八嵌入浮水印頻率前後差值比較圖

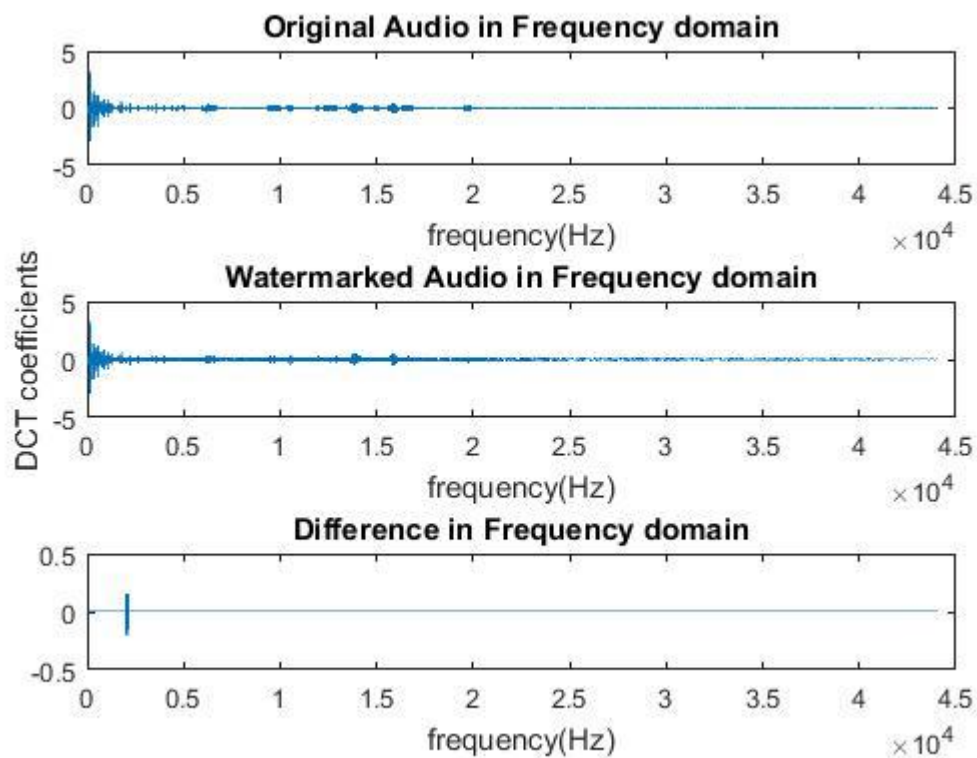


圖 4.2.2-9 樣本九嵌入浮水印頻率前後差值比較圖

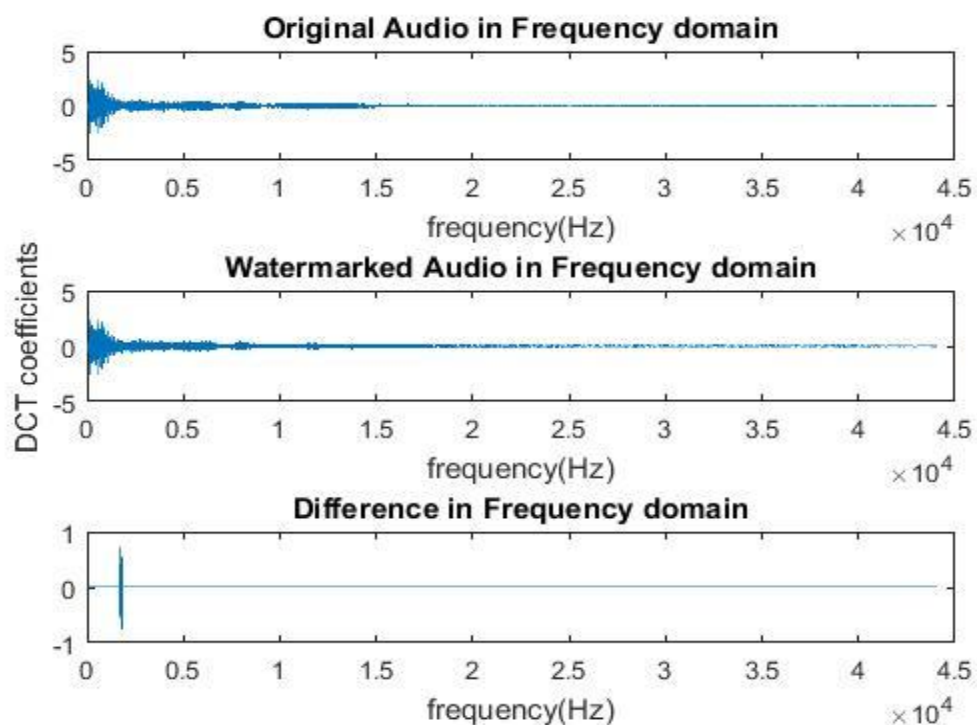


圖 4.2.2-10 樣本十嵌入浮水印頻率前後差值比較圖

從上文的數據資料可以觀察出本論文演算法經由多層浮水印嵌入，並且使用分層解出再拼湊，達到的效果比其他三種嵌入演算法更為突出，值得一提的是本論文演算法在樣本五的數據非常突出，導致此原因是因為在比例因數選擇上本論文的調整不受原音檔的限制，然而演算法[15]因為有比例因數上限值，故無法再提高比例因數，所以對於萃取率的部分也相對較為不理想。

接著我們持續觀察比例因數的部分，在 Gram-Schmidt 正交化法[14]中，為了降低嵌入時的運算量，在比例因數選擇的部分採取接近指數函數(Exponential Function)的方式，這是基於音訊經過離散餘弦轉換後的值經由絕對值轉換後，其趨近於指數函數如圖 4.2.2-11 所示，而互補碼法則是為了增加準確性，利用疊代的方式，針對每一個不同的區段，計算出適合的比例因數，也就因為如此相對的演算法就須付出較多的運算代價。本演算法所提出的比例因數選擇，跟其餘的演算法不同之處是在選擇比例因數時，藉由原音檔經過離散餘弦轉換後的係數與浮水印基礎序列計算乘積值，並且使用即將嵌入的浮水印位元做比較，計算出最佳比例因數，而比例因數是逐層逐個分析如圖 4.2.2-13，其中此圖為四層的比例因數選擇，並且各層嵌入浮水印位元為 104Bits。本演算法在比例因數選擇上本演算法的優點在於，不用設定初始值，只需藉由原音檔資訊即可判斷比例因數，在計算時間上優於互補碼法[15]，而在精確性上優於 Gram-Schmidt 正交化法[14]，最後在萃取率上優於其他三者。

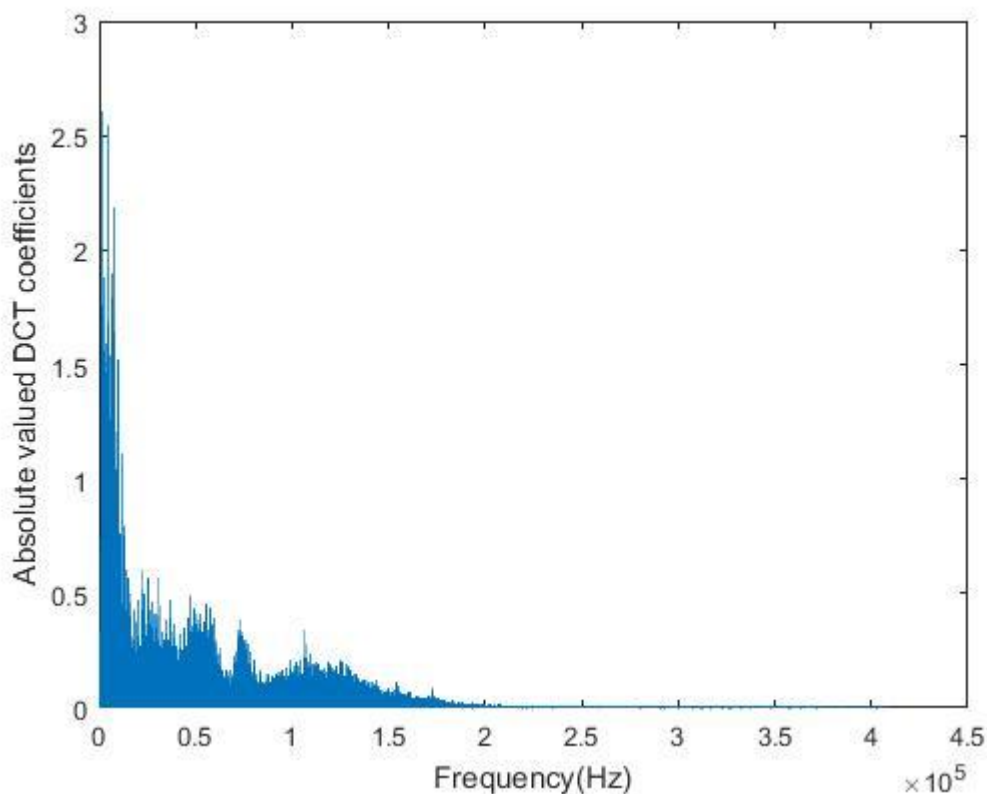


圖 4.2.2-11 將離散餘弦轉換後係數取絕對值圖

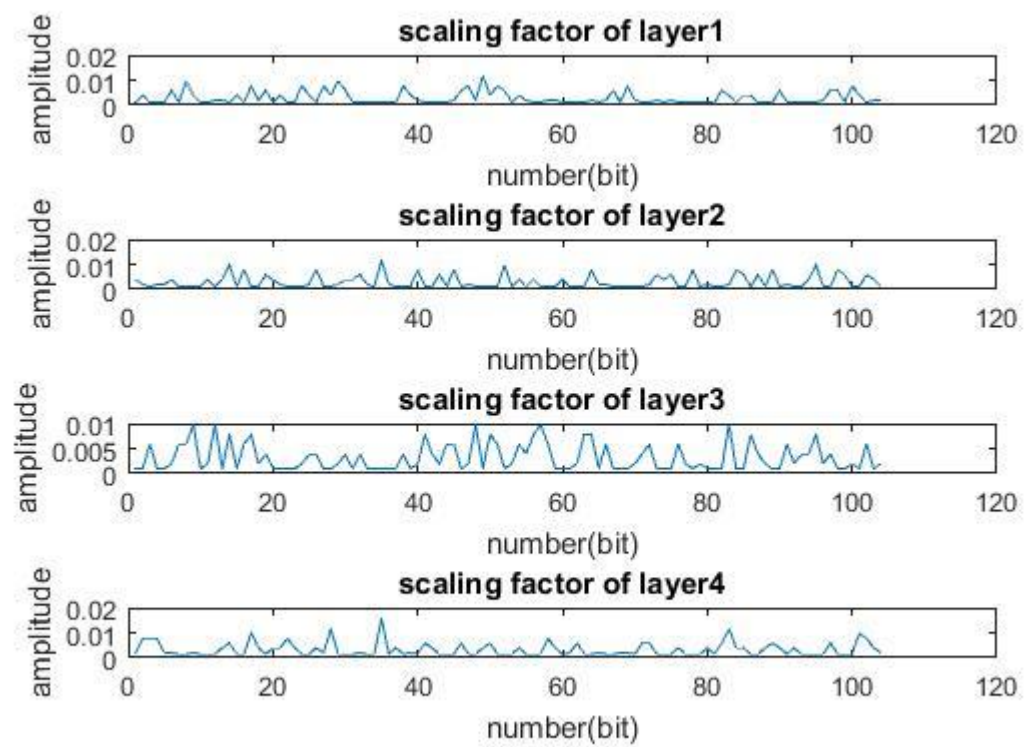


圖 4.2.2-13 本演算法之四層比例因數選擇圖

4.3. 演算法時間與演算法時間複雜度討論

本論文所提出之演算法為使用偽隨機碼序列對於浮水印資訊進行嵌入，而相對應比較的演算法使用不同的偽隨機碼序列對於浮水印做嵌入，故演算法的偽隨機碼序列產生為影響整體演算法運算的關鍵因素之一，因此我們將在本章節討論各個演算法產生偽隨機碼序列的時間複雜度，並且使用 Matlab 軟體測試執行所需要時間做為比較。

本論文所提出演算法與其他論文所提演算法之偽隨機碼序列產生主要為三種方法：(1)循環碼法(2)Gram-Schmidt(3)互補碼

首先我們觀察循環碼的產生方式，循環碼藉由對於一個長度為 n 的偽隨機碼序列，進行循環位移，則每建立一個循環碼序列所需的運算量為 n ，由此可知建立所有偽隨機碼序列則需要 $N_p \cdot n$ 的運算量，故其運算複雜度為 $O(n^2)$ 。再者，我們觀察 Gram-Schmidt 正交化法的運算量，對於要建立一個長度為 n 且數目為 N_p 的偽隨機碼序列，所需要的運算量為 $N_p \cdot n^2$ ，運算複雜度為 $O(n^3)$ 。最後則是本演算法產生偽隨機碼序列運算量的部分，然而由於本演算法在創造偽隨機碼序列時是分別由其他運算組合而成的，故我們將分別討論不同的運算。首先是在產生 Hadamard 矩陣時，由式(4.3-1)所示我們可以利用 2^{k-1} 階的 Hadamard 矩陣，產生 2^k 階的 Hadamard 矩陣，因此產生出 2^k 階的 Hadamard 矩陣的運算量為 $\log(n)$ 。

$$H_{2^k} = \begin{bmatrix} H_{2^{k-1}} & H_{2^{k-1}} \\ H_{2^{k-1}} & -H_{2^{k-1}} \end{bmatrix} \quad (4.3-1)$$

接著則是使用產生出的 Hadamard 矩陣經由式(2.3.3-3)產生 n 個長度為 n^2 的序列，故需要 $(\sqrt{n^3})$ 的運算量，接著我們將產生出的序列經由式(2.3.3-5)產生 n^2 個長度為 n^2 的序列，故需要 $(\sqrt{n^4})$ 的運算量。因此經由互補碼產生出的偽隨機碼序列運算複雜度則為 $O(n^2)$ ，最後我們將所有方法演算法時間複雜度統整為表.4.3-1

表.4.3-1 三種偽隨機碼序列產生演算法之時間複雜度比較表

方法	循環碼法	Gram-Schmidt 正交化法	互補碼法
時間複雜度	$O(n^2)$	$O(n^3)$	$O(n^2)$

接著我們將使用 Matlab 軟體測試執行所需要時間，我們將三種方法皆產生 128×512 的偽隨機碼序列，而 Matlab Benchmark 數據圖與相對執行速度直方圖如圖 (4.3-1)、(4.3-2) 所示，最終由於 Matlab 軟體的第一次執行時間會較為長，故我們將不採用第一次運算執行時間，而是取用後十次的執行時間做為平均並將比較列於表 4.3-2 中。

Computer Type	LU	FFT	ODE	Sparse	2-D	3-D
Windows 7, Intel Xeon E5-1650 v3 @ 3.50 GHz	0.1228	0.1090	0.0615	0.0887	0.1966	0.2270
iMac, OS X 10.10.5, Intel Core i7 @ 3.4 GHz	0.1425	0.1444	0.1041	0.1102	0.3964	0.3219
Windows 10, Intel Xeon X5650 @ 2.67 GHz	0.2550	0.1809	0.1112	0.1490	0.4603	0.3478
This machine	0.1474	0.2591	0.0935	0.1228	0.4635	0.7709
Linux, Intel Xeon W3550 @ 3.07 GHz	0.2528	0.1790	0.1762	0.1540	0.6825	0.6712
Surface Pro 3, Windows 8.1, Intel Core i5-4300U @ 1.9 GHz	0.4704	0.2599	0.1365	0.2163	0.9987	0.8042
MacBook Pro, OS X 10.11.2, Intel Core i5 @ 2.6GHz	0.2472	0.1862	0.0760	0.1217	2.0229	1.7001
Linux, Intel Core2 Quad Q6600 @ 2.40GHz	0.6738	0.4044	0.2202	0.3244	0.9605	0.8179

Place the cursor near a computer name for system and version details. Before using this data to compare different versions of MATLAB, or to download an updated timing data file, see the help for the bench function by typing "help bench" at the MATLAB prompt.

圖 4.3-1 Matlab Benchmark 數據圖

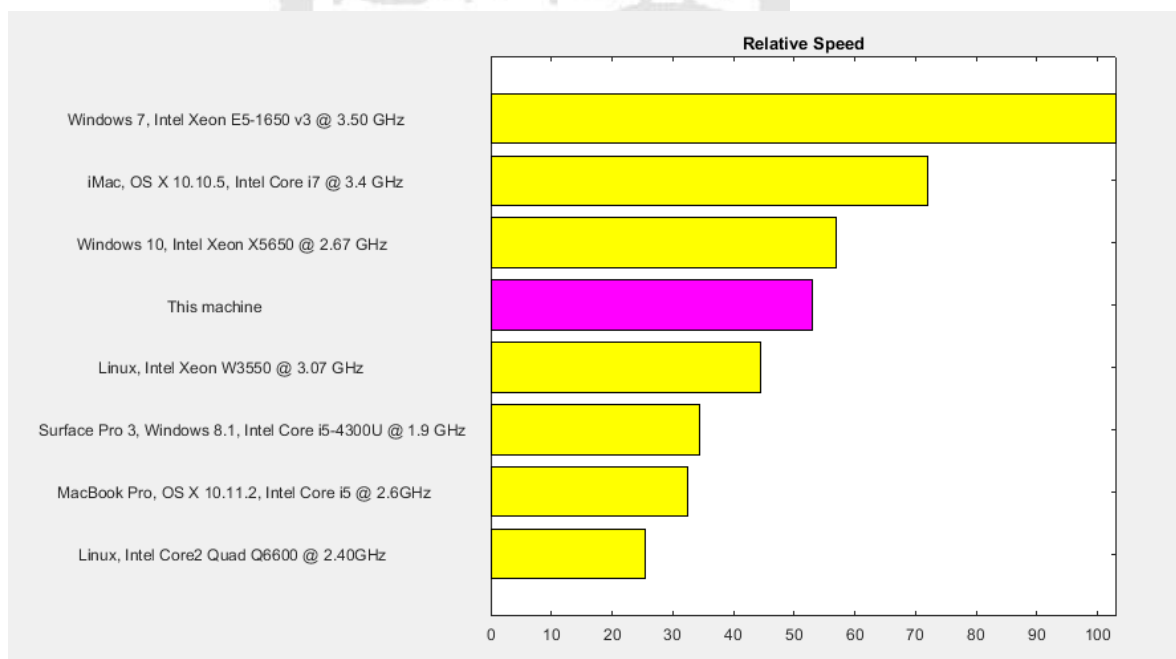


圖 4.3-2 Matlab Benchmark 相對執行速度直方圖

表.4.3-4 偽隨機碼序列實際運算時機比較表

方法	循環碼法	Gram-Schmidt 正交化法	互補碼法
時間(秒)	0.0157	4.6171	0.077

然而影響演算法的因素不只有偽隨機碼序列產生時間，在本演算法中有提及比例因數的選擇，比例因數的選擇也將對演算法產生影響，故本論文也對於比例因數的選擇使用 Matlab 軟體進行測試並列出所須執行時間，本測試的樣本選擇為樣本一並且去除第一次的執行時間，將後十次的執行時間做平均。執行時間的比較表如表 4.3-5 所示。

表.4.3-5 比例因數選擇實際運算時機比較表

方法	Method in [13]	Method in [14]	Method in [15]	Proposed
時間(秒)	0.0117	0.0007	14.689	0.2539

可以觀察表 4.3-4 中，偽隨機碼序列產生時間雖然高於循環碼，不過卻遠低於 Gram-Schmidt 正交化法，而在表 4.3-5 的部分可以觀察出，比例因數的選擇雖然高於論文[13]、[14]，但卻是遠低於論文[15]之演算法。值得一提的是本研究所提出之偽隨機碼與論文[15]相似，但在演算法使用上本演算法只需使用 4 組長度為 512 的偽隨機序列，而論文[15]之演算法則必須產生出 128 組長度為 512 的偽隨機序列，可以就此得知本研究之演算法在產生偽隨機序列上也會優於論文[15]。

第五章 結論與未來展望

本論文提出藉由超級互補碼演算法所產生的彼此正交偽隨機碼序列，經由特定方式排列並且就由位移嵌入浮水印資訊，再藉著原音檔與浮水印影響計算，取得各層個位元的嵌入比例因數，使浮水印位元嵌入於音檔之中，並且嵌入完後之音檔經過音訊處理(高斯白雜訊(Adding white Guassian noise)、重新量化(Re-quantization)、調幅(Amplitude scaling)、高、低通濾波器(Filtering)、AAC 壓縮)後，有最佳的浮水印資訊萃取率。在比例因數選擇的部分運算複雜度低於互補碼法[15]，而在偽隨機碼序列的產生上比循環碼方式演算法運算時間高，但遠低於 Gram-Schmidt 正交化法[14]，而產生的序列數量遠低於互補碼法[15]，總結以上優點，最後在萃取率的部分也較佳，除此之外，本演算法有另外一個優勢是因為本演算法是多層嵌入的，所以需要被保護之音檔，可以被多個使用者所擁有，這是其他演算法所無法達成的。本演算法在安全性上以及還原度上皆有相當的程度，故本演算法為一個有所提升的演算法。

由於本論文的音訊訊號處理為同步訊號處理，例如高斯白雜訊(AWGN)、高通濾波器(High-pass filter)、低通濾波器(Low-pass filter)等，尚未考慮非同步音訊處理對於浮水印演算法的影響，而非同步音訊處理則例如：抖動(Jitter)、剪裁(cropping)、…等。非同步音訊訊號處理對於浮水印解出正確率會有嚴重的影響，近期學者也有對於此部分討論，在未來將會針對此部分進行深入的研究，使得展頻浮水印對於音訊訊號處理方面有更進一步的發展。

參考文獻

-
- [1] S.P. Mohanty, Digital watermarking: a tutorial review. Technical Report, University of South Florida (1999) [Online], <http://www.cs.unf.edu/smohanty/research/Reports/Mohanty/WatermarkingSurvey1999.pdf>
 - [2] L. Boney, A. H. Tewfik and K. N. Hamdy, "Digital watermarks for audio signals," *Proceedings of the Third IEEE International Conference on Multimedia Computing and Systems*, Hiroshima, Japan, pp. 473-480, 1996.
 - [3] D. Lam, "Audio watermarking". COMPSYS401A Project, The University of Auckland, 2003
 - [4] P. Bassia, I. Pitas and N. Nikolaidis, "Robust audio watermarking in the time domain," in *IEEE Transactions on Multimedia*, vol. 3, no. 2, pp. 232-241, Jun 2001.
 - [5] W. Bender, D. Gruhl, N. Morimoto, A. Lu, Techniques for data hiding. IBM Systems Journal. vol. 35, nos 3&4, pp. 313-336, 1996.
 - [6] Ye Wang, "A new watermarking method of digital audio content for copyright protection," *Signal Processing Proceedings, 1998. ICSP '98. 1998 Fourth International Conference on*, Beijing, China, vol. 2, pp. 1420-1423, 1998.
 - [7] P. Bassia, I. Pitas and N. Nikolaidis, "Robust audio watermarking in the time domain," in *IEEE Transactions on Multimedia*, vol. 3, no. 2, pp. 232-241, Jun 2001.
 - [8] Ye Wang, "A new watermarking method of digital audio content for copyright protection," *Signal Processing Proceedings, 1998. ICSP '98. 1998 Fourth International Conference on*, Beijing, China, vol. 2, pp. 1420-1423, 1998.
 - [9] Hyen O Oh, Jong Won Seok, Jin Woo Hong and Dae Hee Youn, "New echo embedding technique for robust and imperceptible audio watermarking," *2001 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No. 01CH37221)*, Salt Lake City, UT, vol.3, pp. 1341-1344, 2001.
 - [10] P. Zhang, S. Xu and H. Yang, "Robust and transparent audio watermarking based on improved spread spectrum and psychoacoustic masking," *2012 IEEE International Conference on Information Science and Technology*, Hubei, pp. 640-643, 2012.
 - [11] H. S. Malvar and D. A. F. Florencio, "Improved spread spectrum: a new modulation technique for robust watermarking," in *IEEE Transactions on Signal Processing*, vol. 51, no. 4, pp. 898-905, Apr 2003.
 - [12] Y. Chen, G. Gao, Y. Liu and B. Lv, "A cross spread spectrum audio watermarking robust to acoustic transmission," *2017 36th Chinese Control Conference (CCC)*, Dalian, pp. 5025-5030, 2017.
 - [13] Y. Xiang, I. Natgunanathan, Y. Rong and S. Guo, "Spread Spectrum-Based High Embedding Capacity Watermarking Method for Audio Signals," in *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 23, no. 12, pp. 2228-2237, Dec. 2015.
 - [14] Y. Xiang, I. Natgunanathan, D. Peng, G. Hua and B. Liu, "Spread Spectrum Audio Watermarking Using Multiple Orthogonal PN Sequences and Variable Embedding Strengths and Polarities," in *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 3, pp. 529-539, March 2018.
 - [15] 徐詠翔, "利用互補碼方式之展頻音訊浮水印演算法," 台灣, 國立成功大學電機工程學系研究所, 2018.
 - [16] G. Strang, "Introduction to linear algebra," fifth Edition, Wellesley-Cambridge Press, 2016.

- [17] M. Golay, "Complementary series," in *IRE Transactions on Information Theory*, vol. 7, no. 2, pp. 82-87, April 1961.
- [18] R. Turyn, "Ambiguity functions of complementary sequences (Corresp.)," in *IEEE Transactions on Information Theory*, vol. 9, no. 1, pp. 46-47, Jan 1963.
- [19] N. Suehiro and M. Hatori, "N-shift cross-orthogonal sequences," in *IEEE Transactions on Information Theory*, vol. 34, no. 1, pp. 143-146, Jan 1988.
- [20] 林金賢, "互補碼分碼多工系統之研究," 台灣, 國立中山大學電機工程學系研究所, 2003.
- [21] J. Hadamard, "Résolution d'une question relative aux déterminants," *Bulletin des Sciences Mathématiques*, vol. 17, pp. 240-246, 1893.
- [22] J.J. Sylvester. *Thoughts on inverse orthogonal matrices, simultaneous sign successions, and tessellated pavements in two or more colours, with applications to Newton's rule, ornamental tile-work, and the theory of numbers.* [Philosophical Magazine](#), 34. pp.461-475, 1867
- [23] Kojima T, Oizumi A, Okayasu K, Parampalli U. An audio data hiding based on complete complementary codes and its application to an evacuation guiding system. *Signal Design and Its Applications in Communications, The Sixth International Workshop*. pp.118-121. 2013.
- [24] Rec.B. S. 1387: *Method for objective measurements of perceived audio quality*, Rec.B. S. 1387, International Telecommunications Union, Geneva, Switzerland, 2001.

